

有界示向误差下无线电干扰源的 覆盖认证搜索与快速清除

研究与复核稿：官方模拟器测试已按要求搁置，非可直接提交版本

摘要

无线电干扰源数量、接收半径以及定向源的发射方向均未知，因而快速定位不仅是多条示向线求交，还必须解决误差约束、全区域发现与正确停止三个问题。本文采用“确定性覆盖保证正确性、观测驱动调度降低耗时”的统一框架。

针对问题一，将每次示向度转为正向楔形半平面约束，给出空集、有界和无界情形的判别及顶点对直径算法。构造由合法测向观测实现的边长 36 米等边三角形反例，证明直径为区域直径的圆不一定覆盖区域；以最小包围圆半径代替区域直径作为一次清除判据。

针对问题二，利用首次接收到信号隐含的接收半径下界，推导无需已知目标距离的三圆交候选区域。以交会夹角与距离共同刻画测向条件，并以候选点的后验可行域包围圆半径进行采样极小极大评分，区分可证明的接收安全性与启发式的最优性。

针对问题三，先构造 900 米覆盖的七点骨架并证明该覆盖半径下七点达到点数下界，再将六点环半径收缩为 1200 米，利用精确覆盖公式保留 31.10 米接收余量；收缩环旨在减少行程，实际总耗时收益仍需由同口径合成实验检验。针对问题四，用含圆域外点的 31 点三角格及凸包半平面性质保证任意发射方向不漏检，构造逐点不可删反例，并从实际位置滚动优化剩余骨架顺序。两问均维护逐频道完成证书；局部定位保留有限测向和完整光学覆盖后备。

在明确声明分布与固定地点误差的本地环境中，基础对照执行 4430 次运行；另用 3080 次隔离种子的运行评估本轮增强。每问 500 个新随机世界上，Q3 平均单源总耗时为 275.68 秒，较同世界基础集成策略降低 12.48%；Q4 为 797.76 秒，降低 1.72%。所列运行均全清；静态 2-opt 和区间自适应测点因完整耗时未改善而未被默认采用。所有结果均非官方成绩；正式日志和参赛队人工核验仍缺失。

关键词：有界误差；交会定位；最小包围圆；覆盖证书；主动测向；定向源搜索

1 问题理解、目标与证据边界

1.1 四问之间的依赖关系

题目给定半径 1800 米的目标圆域，最多 20 个频道、每频道至多一个静止干扰源；在问题三、四中目标总数为 10 至 16 但不向程序公开。每次检测只报告无信号、近源或带误差的示向度，没有场强数值、距离或源数接口 [1]。因此不能从信号强弱反推距离，不能把 20 个频道同时检测，也不能把“已发现列表清空”当成全清。

问题一回答怎样描述和计算定位不确定性；问题二回答下一次测向应走向何处；问题三在全向条件下把发现、定位、清除和调度结合；问题四进一步解决从发射背面完全看不到信号的困难。本文不采用需要训练真值标签的端到端学习，也不对未知目标先验进行未经说明的概率化。主模型只用题目给定的确定界，概率分布仅用于可复现的合成实验。

1.2 优化目标和两种时间

设总行程为 L ，检测次数为 M ，实际切换频道次数为 S ，失败清除次数为 F ，成功清除数为 K 。 N 表示该世界中隐藏的真实源总数，只在退出后的评价器中使用； K 表示已收到成功清除响应的频道数。由附件计时规则，虚拟任务时间为

$$T = \frac{L}{5} + 5M + S + 3F + 5K. \quad (1)$$

式中距离单位为米、时间单位为秒；成功清除已含 3 秒光学精确定位与 2 秒激光清除，不能另加一次 3 秒。/clear 中的频道只指定目标，不改变测向机频道。

优化是字典序的：首先对每个合法世界保证最终全清，其次在这一约束下减小 T 。评价量为清除率 K/N 和每世界单位源数时间 T/N ；本文列示的完整清除运行满足 $K = N$ ，此时 $T/K = T/N$ 。另记程序现实运行时间 T_{real} 。现实 20 分钟限制、测试窗口与虚拟 100 小时限制不是同一个量，不能令程序为每次虚拟 5 秒检测真的等待 5 秒。

由于目标状态未知且实际反馈尚未出现，不能预先给出一个对所有世界同时最短的固定路径。本文的覆盖和清除安全性有解析证明，调度效率则由同口径对照实验支持，不声称全局时间最优。

1.3 当前证据等级

原题、两份附件及本届官方规则是题意来源；几何命题是解析推导；结果文件是本机程序实际输出。本文中的合成实验严格标记为非官方。官方模拟器当前不能运行，已按用户要求搁置，不继续尝试启动，也不编造题目表 1 所需的案例编码和六份加密日志。研究稿正文与代码可以复核，但不能因编译成功或合成全清而宣称已经具备正式提交资格。

2 符号、假设和统一信息状态

表 1: 主要符号及来源

符号	含义	来源或单位
$\mathcal{D}$	$\{g : \ g\ \leq 1800\}$, 源所在区域	题目, 米
c, g_c	频道编号、该频道源坐标	$c \in \{1, \dots, 20\}$
R_c	源的有效接收半径	未知, $[1000, 1500]$ 米
n_c	定向源发射半平面的单位法向	未知, 固定
s_i, θ_i	第 i 次检测点及示向度	可观察量
α	示向度容差	理论 1° , 实现 1.005°
$\mathcal{P}_c$	包含真实源的保守位置多边形	在线维护
$D(\mathcal{P}), r_*(\mathcal{P})$	直径、最小包围圆半径	米
ρ_{opt}	规划使用的光学覆盖半径	19.9 米
$V_c \subseteq V$	已完成的骨架索引与完整骨架索引集	每频道单独记录

除题目条件外, 仅作以下实现约定:

1. 示向度接口保留两位小数。为兼容先加入 $\pm 1^\circ$ 误差再舍入的过程, 计算使用 1.005° 容差; 这是额外的保守余量, 不改变题面误差定义。
2. 以外切正 180 边形近似目标圆盘, 绝不使用内接近似删除边界可能源。该外切多边形的径向过估计为 $1800(\sec(\pi/180) - 1)$, 小于 0.275 米。
3. 为控制数值误差, 一次保证清除的计算半径设为 19.9 米, 小于设备允许的 20 米。几何半平面向外添加极小浮点余量, 约束矛盾时报告错误而不擅自丢弃旧观测。
4. 源静止、频道唯一、方向固定和未清除前持续发射均来自题意。不同地点误差的具体分布及独立性没有给定; 重复原地测向不得被当作独立样本平均降噪。

定向源的可检测条件写作

$$\|s - g_c\| \leq R_c, \quad n_c^\top (s - g_c) \geq 0. \quad (2)$$

全向源没有第二个条件。距离不超过 5 米且可检测时返回 `near`, 此时原地清除必定成功。光学清除只要求 $\|s - g_c\| \leq 20$, 与式(2)的方向条件无关。

完整隐藏状态可以表示为各频道的存在性、位置、类型、接收半径和方向。程序不读取这个隐藏状态, 而只维护位置多边形、正测向记录、失败清除记录、清除成功集合及逐频道骨架检测记录。这种保守的信息压缩可能放弃部分负观测信息, 但不会因此误删真实源。

3 问题一：交会区域直径与清除判据

3.1 从示向度到正向半平面交

定义 $u(\theta) = (\cos \theta, \sin \theta)$, $u_i^\pm = u(\theta_i \pm \alpha)$, 二维叉积为 $\text{cross}(a, b) = a_x b_y - a_y b_x$ 。一次观测对应

$$W_i = \left\{ g : \begin{array}{l} \text{cross}(u_i^-, g - s_i) \geq 0, \\ \text{cross}(u_i^+, g - s_i) \leq 0, \\ u(\theta_i)^\top (g - s_i) \geq 0 \end{array} \right\}. \quad (3)$$

前两个半平面在 $\alpha > 0$ 时已经给出正向楔形, 第三个条件显式排除精确零误差情形下的背向射线。使用正弦、余弦表达使 359.9° 和 0.1° 之间的跨零问题无需分支处理。

原始交会区域为 $P = \bigcap_i W_i$ 。若题意需要同时利用已知目标区域或距离上限, 可再取交集; 但添加先验后的区域不是原始无限楔形交, 不能混淆二者的直径。

3.2 空集、无界集和有界多边形的算法

将全部约束写成 $Ag \leq b$ 。一个透明、易核查的实现如下。

1. 解二维可行性线性规划。无可行解时报告观测或条件矛盾, 不返回虚假的零直径。
2. 分别最大化及最小化 x, y 。任一坐标目标无界, 则非空区域无界, 直径为 $+\infty$; 四个坐标上下界都有限则区域有界。
3. 对每对非平行边界直线求交, 保留满足全部半平面约束的交点; 去除数值重复点, 按凸边界排序。
4. 求所有顶点对距离的最大值。单点直径为 0, 线段直径为两端点距离。

配套 `intersection.py` 直接完成上述分类, 不用任意大包围盒冒充原区域的有界性。若约束数为 m , 直线交点枚举并逐个检验的朴素复杂度为 $O(m^3)$; 通常只需少数测向, 足够迅速。大规模情形可改为按极角排序的半平面交和旋转卡壳, 几何阶段降至 $O(m \log m)$, 但这不是本题在线计算的瓶颈。

结论 1 (直径的顶点刻画). 若 $P = \text{conv}\{v_1, \dots, v_n\}$ 非空、有界, 则

$$D(P) = \max_{i,j} \|v_i - v_j\|. \quad (4)$$

证明. 任取 $p = \sum_i a_i v_i$ 与 $q = \sum_j b_j v_j$, 其中权重非负且各自和为 1, 有

$$\|p - q\| = \left\| \sum_{i,j} a_i b_j (v_i - v_j) \right\| \leq \sum_{i,j} a_i b_j \|v_i - v_j\| \leq \max_{i,j} \|v_i - v_j\|.$$

反向不等式由各顶点本身属于 P 得到。□

3.3 直径圆不能总是覆盖：一个真实可实现的反例

边长为 D 的等边三角形直径就是 D , 最小包围圆半径却是 $D/\sqrt{3} > D/2$ 。这不仅是任意多边形反例, 还能由本题 $\pm 1^\circ$ 测向直接生成。

取 $D = 36$, $A = (0, 0)$ 、 $B = (36, 0)$ 、 $C = (18, 18\sqrt{3})$ 。依次对三条逆时针有向边取单位方向 e_j , 令

$$S_j = V_j - L e_j, \quad L = 18\sqrt{3} \cot 2^\circ - 18. \quad (5)$$

测向读数取边方向加 1° , 即 $1^\circ, 121^\circ, 241^\circ$ 。每个楔形下边界沿对应边、上边界经过对角顶点；三个楔形都包含该三角形, 而三条下边界的内半平面交恰为该三角形。所有三角形点距对应检测站小于 1000 米, 故可取合法接收半径 1000 米。

实际计算得到直径约 36.000000 米, 最小包围圆半径约 20.784610 米。于是即使 $D < 40$, 仍不一定能在某一点以 20 米光学半径保证一次清除。

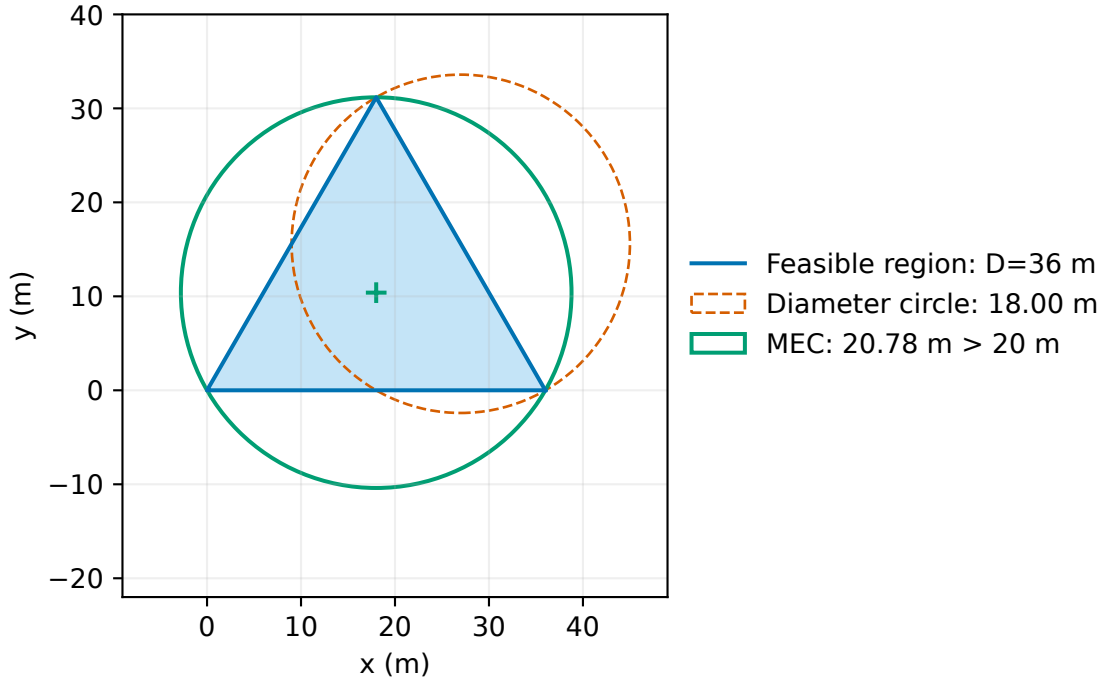


图 1: 直径为 36 米的可实现三角形反例。任何半径 18 米的圆都无法覆盖三顶点；正确包围半径约 20.785 米。

3.4 用最小包围圆直接连接定位与清除

定义

$$r_*(P) = \min_z \max_{p \in P} \|p - z\|, \quad z_*(P) = \arg \min_z \max_{p \in P} \|p - z\|. \quad (6)$$

圆盘是凸集, 包含多边形所有顶点即包含整个多边形。最小包围圆由一个、两个或三个边界支持点确定, 可用随机增量支持点算法求解 [2]。本文实现对近共线三点重算高精度行列式, 并在最终半径上保留向外余量。

平面 Jung 界为

$$\frac{D(P)}{2} \leq r_*(P) \leq \frac{D(P)}{\sqrt{3}}. \quad (7)$$

两点支持时左端取等；三点支持时圆心在支持三角形内, 三段圆心角中最大的一段至少 120° , 对应弦长至少 $\sqrt{3}r_*$, 得到右端。等边三角形取右端等号。因此:

- $r_*(P) \leq 20$ 是存在一次保证清除点的充要几何条件；执行点取 $z_*(P)$ 。
- 仅使用直径时， $D(P) \leq 20\sqrt{3}$ 是充分条件， $D(P) > 40$ 则必不可能一次保证覆盖。
- $20\sqrt{3} < D(P) \leq 40$ 时必须进一步算包围圆，不能凭直径直接清除。

3.5 在线使用的保守多边形

在问题三、四中，每次有效示向度还说明 $\|g - s_i\| \leq 1500$ 。为保持纯线性裁剪，可使用安全松弛

$$u(\theta_i)^\top (g - s_i) \leq 1500. \quad (8)$$

这不是把接收半径错误设为 1500，而是利用其已知上界。若初始外切多边形为 A_{180} ，则维护

$$\mathcal{P}_c = A_{180} \cap \bigcap_{i \in I_c} (W_i \cap \{g : u(\theta_i)^\top (g - s_i) \leq 1500\}). \quad (9)$$

真实源初始在 A_{180} 内，每条新增约束也包含它，故归纳得到 $g_c \in \mathcal{P}_c$ 。一次观测裁剪四个半平面， m 次观测最多形成 $180 + 4m$ 条边，复杂度 $O(180m + m^2)$ 。空交集是需要定位原因的异常，而不是允许重置状态的信号。

4 问题二：第二检测点的安全候选区域与选点策略

4.1 单次测向为什么没有唯一最优第二点

只有角度而没有距离时，目标位于长而窄的扇形中。若第二点沿首次示向方向移动，两条示向线可能几乎平行或反平行，径向误差难以压缩。若仅为了接近 90° 交会而绕行很远，则会增加移动时间，未必有利于题目总目标。

令 $r_i = \|g - s_i\|$ 、 $u_i = (g - s_i)/r_i$ 、 $n_i = (-u_{iy}, u_{ix})$ 。对小误差线性化，有

$$|n_i^\top \delta g| \leq \alpha r_i. \quad (10)$$

两条误差带交形成的平行四边形面积近似为

$$A \approx \frac{4\alpha^2 r_1 r_2}{|\sin \beta|}, \quad (11)$$

其中 β 为两条真实视线夹角， α 以弧度计。固定距离时， 90° 交会最有利；距离也变化时，应同时考虑缩短 r_2 、避免共线及移动代价。该式是局部解释，不能代替精确楔形裁剪。

若额外采用独立高斯角误差的辅助似然，才可写出 Fisher 信息 $J = \sigma^{-2} \sum_i n_i n_i^\top / r_i^2$ 及 $\det J = \sin^2 \beta / (\sigma^4 r_1^2 r_2^2)$ 。没有这种似然假设时，这至多是加权最小二乘的局部信息代理，不能当成题目提供的真实误差统计模型。最终安全性仍由式(9)保证。

4.2 利用首次已接收事实推导三圆交

取第一次检测点 S_1 为局部原点，首次读数方向为 u ，法向为 n ，第二点写为

$$S_2 = S_1 + tu + bn. \quad (12)$$

若可能真实距离为 r ，首次已经接收说明

$$R \geq \max(1000, r), \quad (13)$$

而不仅仅是 $R \geq 1000$ 。设 F_1 为当前全部可能源位置，则保证全向源在第二点仍有信号或返回近源的精确鲁棒条件为

$$S_2 \in \bigcap_{g \in F_1} B(g, \max(1000, \|g - S_1\|)). \quad (14)$$

用完整扇形 $r \in [0, 1500]$ 、角差 $\varphi \in [-\alpha, \alpha]$ 覆盖真实 F_1 ，可得闭式充分候选域。

结论 2 (三圆交接收保证). 令 $u_{\pm} = u(\theta_1 \pm \alpha)$ 。以下候选域内的任一点，对整个放宽扇形都能保证全向接收：

$$\mathcal{C} = S_1 + [B(0, 1000) \cap B(1000u_-, 1000) \cap B(1000u_+, 1000)]. \quad (15)$$

对这一完整扇形，式(15)也恰好是式(14)的交集；若再利用圆域等更强先验，它可能不是最大的安全候选域。

证明. 写 $q = S_2 - S_1$ 。必要性由 $r = 0$ 和 $r = 1000, \varphi = \pm\alpha$ 给出。反之，两端圆约束意味着 $q^T u_{\pm} \geq \|q\|^2 / 2000 \geq 0$ 。小于 180° 的中间短弧上， $q^T u_{\varphi}$ 的最小值在端点，因此对所有允许角度都满足同样下界。若 $r \leq 1000$ ，距离 $\|q - ru_{\varphi}\|$ 关于 r 是凸函数，两端 $r = 0, 1000$ 均不超过 1000，内部也不超过 1000；若 $r \geq 1000$ ，

$$\|q - ru_{\varphi}\|^2 - r^2 = \|q\|^2 - 2rq^T u_{\varphi} \leq 0.$$

故距离不超过 $\max(1000, r)$ ，由首次接收所隐含的半径下界即可保证可检测。 $\square$

4.3 显式候选区域与可执行搜索

在 (t, b) 局部坐标中，三圆交等价于

$$\begin{cases} t^2 + b^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b - 1000 \sin \alpha)^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b + 1000 \sin \alpha)^2 \leq 1000^2. \end{cases} \quad (16)$$

令

$$b_{\max}(t) = \min \left\{ \sqrt{1000^2 - t^2}, \sqrt{1000^2 - (t - 1000 \cos \alpha)^2} - 1000 \sin \alpha \right\}. \quad (17)$$

根号有定义且 $b_{\max} \geq 0$ 时， $0 < t < 1000$ 、 $0 < |b| \leq b_{\max}(t)$ 形成首次方位两侧的横向候选区。排除 $b = 0$ 用于避免把共线点作为唯一方案，而非禁止为了接近目标沿轴行进。

程序按下式构造 42 个候选：

$$\begin{aligned} t &\in \{125, 250, 375, 500, 625, 750, 875\} \text{ 米}, \\ b &= \pm \kappa b_{\max}(t), \quad \kappa \in \{1/4, 1/2, 3/4\}. \end{aligned} \quad (18)$$

对每个候选模拟允许的第二读数，计算新多边形最小包围圆；优先选择采样最坏半径较小的点，可再以本步移动和检测代价打破并列。所有这些点的接收安全性来自定理2，不是来自采样。

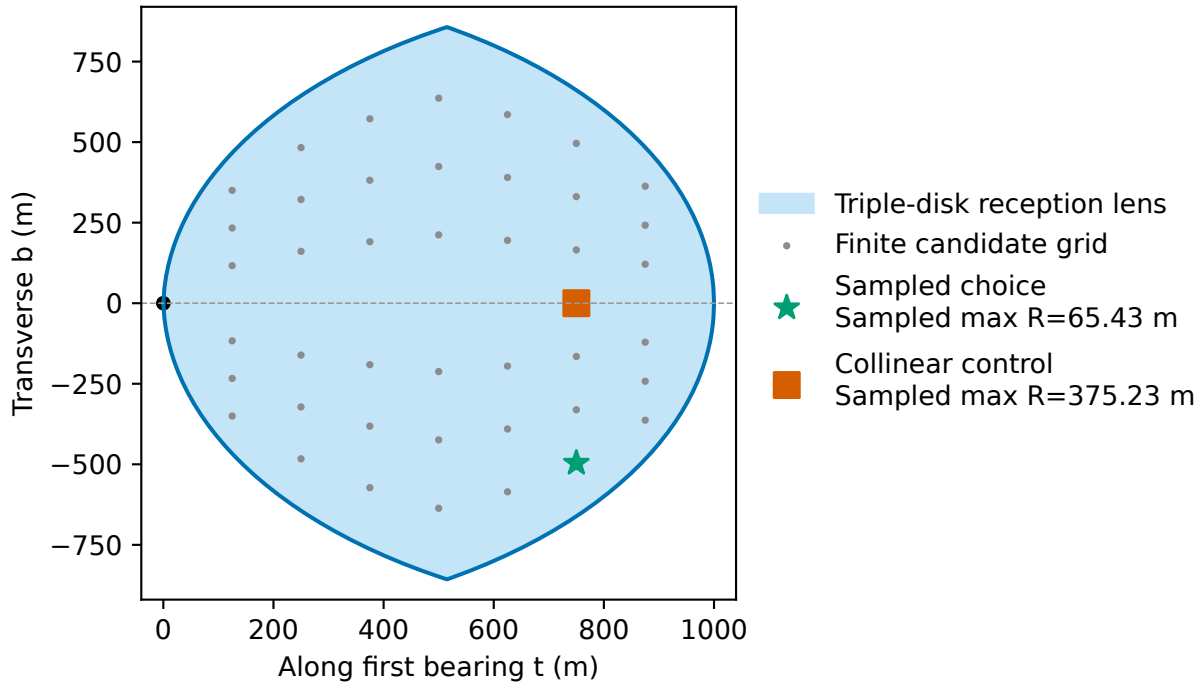


图 2: 第二点的三圆交候选区域。横向基线改善交会条件，但候选排序仍需兼顾移动耗时和未知径向距离。

在 $S_1 = (0, 0), \theta_1 = 0$ 的归一化示例中，取 60 个径向位置、3 个首次真实角差和 3 个第二次测向误差进行比较。程序得到采样最佳几何候选约为 (750, -496.078) 米，采样最坏包围半径约 65.426 米；共线对照 (750, 0) 的对应值约 375.231 米。两者本步耗时分别约 184.844 秒和 155 秒。这说明增加约 29.844 秒移动检测成本可显著改善此例的径向辨识，但第二次测向后仍未必能一次清除。

该示例是候选域与评分算法的演示，不是题目给定正式案例，也不是连续最坏情形或总时间的全局最优解。二维方位误差的连续极值、未知距离分布及后续代价若未严格处理，不能将网格最小值写成“最优定位策略”。

5 局部定位：快速测向与有限光学兜底

5.1 正常测向路径

针对已确认存在的频道，反复计算 (z_*, r_*) ：

1. 当前点到全部多边形顶点距离都不超过 19.9 米时，直接原地保证清除；否则若 $r_* \leq 19.9$ ，移动到 z_* 保证清除。
2. 若 $r_* \leq 38$ 米，可先在 z_* 进行一次光学尝试。失败仅花 3 秒且记录为失败，不将其写成保证成功；随后使用全区域覆盖，避免为一个很小区域继续绕远测向。

3. 设最近一次正测向点为 s^+ ，从 s^+ 指向 z_* 的单位方向为 u 、法向为 n ，候选测点取 $z_* \pm \delta n$ ，其中 $\delta = \min(120, \max(25, 0.22r_*))$ 米。优先较近且未重复正读数的位置。
4. 对实际返回的正示向度继续裁剪；`near` 立即原地清除。每次局部定位最多使用 6 个新测向点，随后转入光学覆盖。

这里的 25、120、38、0.22 和 6 是公开的工程策略参数，而非从题面推导的自然常数。它们影响效率，不影响后备覆盖的正确性。问题二给出的是保证全向接收的完整候选域；本节采用更直接的中心横向点以降低重复评分开销，**没有声称每个在线测点都属于三圆交或总能保持信号**，而是对失去信号设置了可靠处理。

5.2 定向源丢信号后的有限恢复

问题四中，新的点可能越过发射半平面的边界。设失败点相对最近正测向点的分解为 $q^- - s^+ = au + bn$ ，下一点建议取

$$q^{\text{next}} = s^+ + \frac{1}{2}(au - bn). \quad (19)$$

其含义是向已知能接收的位置退回，同时尝试另一侧。新正测向可收缩位置多边形并更新 s^+ ；持续无信号则继续有限恢复，直到达到 6 次测向上限。

式(19)只是利用正负观测进行选点的启发式，不据此删除任何可能源位置，也不假称一定重获信号。它的价值通过最低接收半径、朝外边界源和聚集源的消融实验评价；最终清除保证不依赖恢复成功。

5.3 覆盖整个可行多边形，而非只覆盖顶点

若无法继续有效测向，取多边形的一条直径方向为局部 x 轴，将 P 包含在投影矩形中。把矩形划分为边长分别不超过 $\sqrt{2}\rho_{\text{opt}}$ 的网格单元，在每个单元中心放置半径 $\rho_{\text{opt}} = 19.9$ 米的光学清除圆盘。每个单元半对角线不超过 ρ_{opt} ，故

$$P \subseteq \bigcup_{j=1}^J B(q_j, \rho_{\text{opt}}). \quad (20)$$

注意多个圆盘的并集一般不凸，覆盖所有顶点并不能推出覆盖内部；本文覆盖的是整个外包矩形，因此不存在内部孔洞。

按蛇形访问这些中心，可把起点循环移动到距离当前位置最近的中心；这样最多增加一次跨行程的长跳。每个失败清除只排除该清除圆盘里的目标可能性，不把剩余非凸集合强行当成一个更小的凸多边形。实际源始终在 P 内，故尝试完全部中心之前必有一次成功，随后立即停止该频道的搜索。

5.4 默认配置的有限动作与虚拟时间上界

本节限于本文最终增强策略：问题三用原点及半径 1200 的六点环、最近邻调度；问题四用 990 米 31 点骨架、滚动 2-opt 余点排序；均用固定横向候选，每任务至多 6 次

测向。假定接口正常、误差模型成立且无现实时间中断。证明只用骨架范数界，不依赖访问顺序；不声称全局最优，不涵盖自适应候选或任意参数配置。距离单位均为米。

可行域尺寸与 216 单元证书。 记真源为 g ，最近正示向点为 s ， $\alpha = 1.005^\circ$ 。示向轴坐标满足 $0 \leq x \leq 1500$ 、 $|y| \leq x \tan \alpha$ 。计入显式外扩，令 $\varepsilon = 10^{-7}$ ，取

$$X = 1500 + 2\varepsilon, \quad W = 2((1500 + \varepsilon) \tan \alpha + \varepsilon / \cos \alpha).$$

于是纵向长度至多 X ，横向宽度至多 $W < 53$ ， $\sqrt{X^2 + W^2} < 1501$ 。最小包围圆圆心 $z \in P$ ，故 $\text{diam}(P) < 1501$ 且 $\|z - s\| < 1501$ 。正测点范数至多 3300、初始外切多边形范数小于 1801，使相关 `_padding` 为 $10^{-9} < \varepsilon$ ；此为几何与显式外扩的解析界，不是全部浮点运算的形式化认证。

单点域直接用一圆覆盖。否则设直径端点差为 (a, b) 、 $d = \sqrt{a^2 + b^2} > 0$ 。任意点差满足 $|\Delta x| \leq d$ 、 $|\Delta y| \leq W$ 、 $|b| \leq W$ ，故直径法向投影差 $|-b\Delta x + a\Delta y|/d \leq 2W$ 。计入投影外扩，直径对齐矩形仍长小于 1501、宽小于 106，对角线小于 1505。令 $h_- = \sqrt{2}(19.9 - 8\varepsilon)$ ，不大于代码单元边长上限，则

$$J \leq \left\lceil \frac{1501}{h_-} \right\rceil \left\lceil \frac{106}{h_-} \right\rceil = 54 \times 4 = 216 \quad (21)$$

个半径 19.9 的圆盘覆盖整个矩形。蛇形相邻中心距至多 $\sqrt{2} 19.9$ ，循环移位至多增加一条矩形内跳跃，故

$$L_{\text{snake}} \leq (216 - 1)\sqrt{2} 19.9 + \sqrt{1501^2 + 106^2} < 7570. \quad (22)$$

每个覆盖中心 q_{opt} 与 g 均在同一矩形中，故 $\|q_{\text{opt}} - g\| < 1505$ 。

恢复点必须以最近正测点为中心估计。 固定候选 $q_0 = z \pm \delta n$ 满足 $n \perp (z - s)$ 、 $\delta \leq 120$ ，因此 $\|q_0 - s\| \leq A := \sqrt{1501^2 + 120^2} < 1506$ 。在获得新正示向前，所有恢复仍使用同一个 s 。设 R_k 为相应轴向反射，则

$$q_{k+1} - s = \frac{1}{2}R_k(q_k - s), \quad \|q_{k+1} - s\| = \frac{1}{2}\|q_k - s\|.$$

反射只保持到 s 的距离，不能把失败点的源距与 1500 取平均。故连续恢复有 $\|q_k - s\| \leq 2^{-k}A$ ；新正示向更新 s 后重用首点界。由 $\|s - g\| \leq 1500$ ，全部局部测向点满足

$$\|q - g\| \leq B := 1500 + A < 3006, \quad \|q\| \leq 1800 + B < 4806. \quad (23)$$

圆心源距小于 1501，覆盖中心源距小于 1505，两者范数亦小于 4806。测向间连线至多 $2B < 6020$ ；额外圆心清除接入至多 $B + 1501 < 4520$ ；光学接入至多 $B + 1505 < 4520$ ，若紧接圆心试清则至多 $1501 + 1505 < 4520$ 。

任务进出、动作计数与总界。 每任务至多 6 次测向，各提前返回分支均成功清除或转入有限光学覆盖；真源在覆盖域内，故每个正常完成任务必清除一新源，任务数至多 16。成功终点距该源至多 20、范数至多 1820；共享补测及其 `near` 清除不移动。这处理了从另一源清除点进入下一任务的情形。

骨架点范数至多 $990\sqrt{7} < 2620$ 。集成策略整次骨架扫描不插入局部移动，故至多 31 次骨架抵达，出发于原点、骨架或成功清除点，每段小于 5240。任务首个动作由范数至多 2620 的任务外位置进入范数小于 4806 的局部点，入口小于 7500。此后至多 5 条测向连线、一次额外圆心清除移动（保证清除或小区域试清）、一次光学接入及一条蛇形，故

$$L \leq 31 \times 5240 + 16(7500 + 5 \times 6020 + 4520 + 4520 + 7570). \quad (24)$$

若首动作即圆心清除或光学首点，入口已支付该段，多余预算仅为放宽。任务至下一骨架的退出段已计入 31 次骨架抵达；无需返回原锚点。

骨架至多 31×20 次测向；共享只在任务结束后调用，至多 16 次、每次至多 16 源；局部至多 16×6 次。放宽共享预算得到

$$M \leq 31 \times 20 + 16 \times 16 + 16 \times 6 \leq 31 \times 20 + 31 \times 16 + 16 \times 6 = 1212.$$

每源无信号后试清至多 6 次、小区域试清至多一次、覆盖至多 216 次，另留一次给非覆盖成功清除（骨架、共享、局部 near 或保证清除）；成功后不再处理，故 $C \leq 16(216 + 6 + 1 + 1) = 3584$ 。移动速度为 5 米/秒，测向含换频至多 6 秒，清除至多 5 秒，因而

$$T \leq \frac{31 \times 5240 + 16(7500 + 5 \times 6020 + 4520 + 4520 + 7570)}{5} + 1212 \times 6 + 3584 \times 5 < 100 \times 3600 \text{ s}. \quad (25)$$

这是虚拟时间保守界，不是预期成绩；现实 20 分钟限制须由客户端期限机制另行遵守。

网络故障可能耗尽现实时间，任何算法都不能对无限网络延迟保证 20 分钟内完成。客户端必须按真实剩余时间停止并报告未完成，不能将这类异常伪装成全清；本文尚未验证官方接口条件下的现实耗时。

6 问题三：七点覆盖与观测驱动调度

6.1 七点发现骨架的解析构造

取

$$p_0 = (0, 0), \quad p_{k+1} = 900\sqrt{3} \left(\cos \frac{k\pi}{3}, \sin \frac{k\pi}{3} \right), \quad k = 0, \dots, 5. \quad (26)$$

环半径约 1558.846 米，而覆盖圆半径只需 900 米。

结论 3 (全向七点不漏检). $\mathcal{D} \subseteq \bigcup_{k=0}^6 B(p_k, 900)$ 。因有效接收半径至少 1000 米，在每点检测全部未清除频道必能发现每个尚存的全向源。

证明. 若 $r = \|g\| \leq 900$ ，原点即可覆盖。若 $900 \leq r \leq 1800$ ，存在一个环点与 g 的极角差不超过 30° ，因此

$$\|g - p_k\|^2 \leq r^2 + 3 \times 900^2 - 3 \times 900r.$$

右端是凸二次函数，在 $[900, 1800]$ 上的最大值取在端点，均等于 900^2 。圆周边界与环点夹角平分线同样包含在证明内。□

6.2 每频道的完成证书

为每个频道保留以下状态：已确认清除、已发现但未清除、尚未确定。逐频道记录实际检测过的骨架索引。若某未清除频道在全部七个点都返回无信号，则由覆盖定理可认证该频道无源；若曾经检测到源，则必须获得成功清除反馈，不能因后来无信号而把它当成无源。

因此正确停止条件是：20 个频道各自都已清除或已获得完整覆盖的无源证书。另一个合法提前停止条件是已确认清除了 16 个不同频道，因为这达到题给总数上界。清除 10 个、长时间没有新源、已知目标队列为空均不足以停止。

全向无信号还可排除检测点 1000 米范围内的该频道源，但当前实现不把这种非凸位置减法强塞进凸多边形；它主要用无信号完成骨架证书。虽然可能多走一些路，解释与安全边界更清楚。

6.3 三个可复现的调度层次

基线 A (baseline)：最近未访问骨架点依次扫描频道，一检测到源就离开骨架点完成该源定位清除，再返回继续剩余频道。它简单且有完整覆盖保证，但多源时往返多。

策略 B (batched)：在同一骨架点先完成所有未清除频道的检测，合并维护已发现源。每步在访问新骨架与清除已知源之间做滚动决策。目标任务的分数取当前位置到其包围圆中心的距离；骨架任务分数取移动距离加 $30n_u$ ，其中 n_u 为未清除频道数，30 米等价于一次 6 秒检测切换的移动成本。该加项是扫描费用近似，不是精确价值函数。

策略 C (integrated)：在 B 基础上，完成某源清除后利用已有停靠点为其他已知源补充测向。仅对区域尚大、可能处于 1500 米接收范围且当前位置未曾检测过的频道补测；历史位置按精确坐标去重，不把相距小于 80 米的不同地点误当作相同电磁环境。成功清除频道立即从后续扫描中剔除。

A 到 B 的差异同时包括批量扫描和目标顺序选择，不能把其收益全叫作“共享观测收益”。B 和 C 使用相同覆盖、局部定位和任务评分，只有补测共享开关不同，因此可用于共享机制消融。额外补测仍花 5 或 6 秒，个别案例可能增时，不能声称总是占优。

7 问题四：任意发射方向下的 31 点覆盖

7.1 为什么不能沿用圆内全向搜索

在圆域边界取一个向外发射的源，圆内绝大多数检测点都在其背面，即使距离远小于 1000 米也无信号。故“每个位置附近都有检测点”只保证全向覆盖，不保证定向覆盖。题目允许机器人提交圆域外坐标，必须利用这一条件。

7.2 三角格和半平面凸包证书

取 $s = 990$ 米，基向量 $a = (s, 0)$ 、 $b = (s/2, \sqrt{3}s/2)$ ，保留

$$\mathcal{V} = \{ia + jb : i, j \in \mathbb{Z}, \|ia + jb\| \leq 1800 + s\}. \quad (27)$$

任意 $g \in \mathcal{D}$ 位于一个边长 s 的基本闭等边三角形内。三个顶点都距 g 不超过 s ，范数不超过 $1800 + s$ ，故均在 $\mathcal{V}$ 中。若 $g = \sum_i \lambda_i v_i$ ， $\lambda_i \geq 0$ 且和为 1，则任意发射法向 n 满足

$$\sum_i \lambda_i n^T (v_i - g) = 0. \quad (28)$$

因而不可能所有正权重顶点都位于严格背面，至少一个点满足 $n^T (v_i - g) \geq 0$ 并处于 1000 米以内，必有信号。源落在格边时利用含边界的发射规则；源正好位于圆域内的格点时，其六个邻点均被保留，任意朝向至少有一个非零邻点在前方。

格点范数平方为 $s^2(i^2 + ij + j^2)$ 。默认截断相当于 $i^2 + ij + j^2 \leq 961/121 < 8$ ，出现的范数层只有 0, 1, 3, 4, 7，点数分别为 1, 6, 6, 6, 12，总计 31；其中 13 点在目标圆内，18 点在圆外。不能把这 18 点投影回圆域。

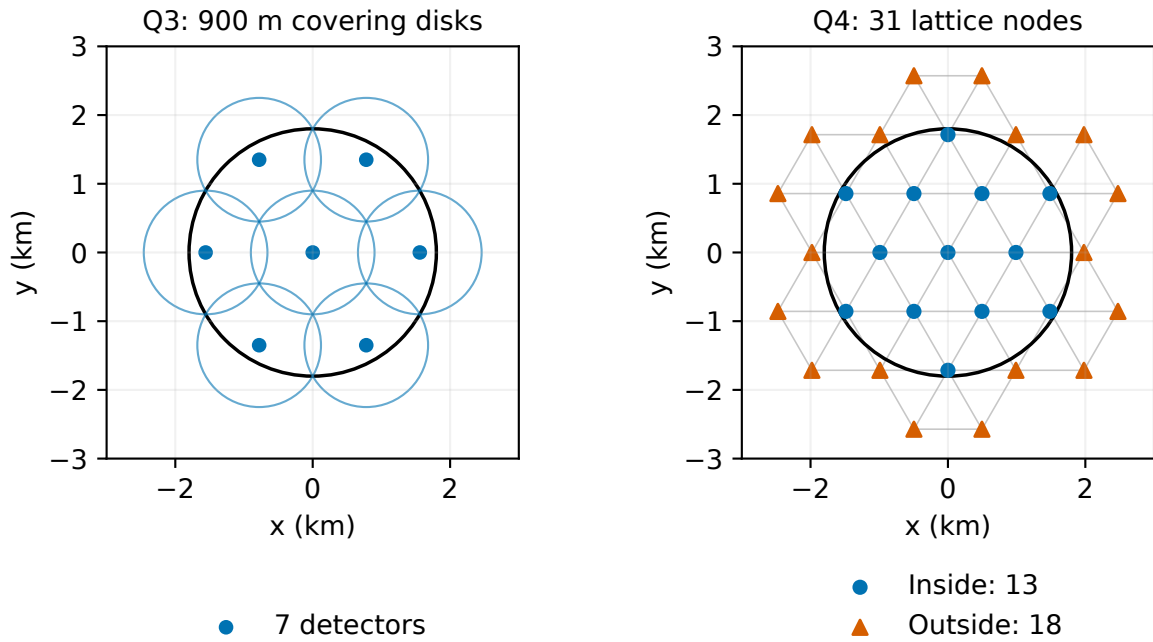


图 3: 全向七点覆盖与定向 31 点覆盖。右图圆外格点是方向覆盖证书的一部分，而非可删除的冗余越界点。

7.3 比“恰好在边界上”更强的数值余量

默认点集还能给出严格正方向余量。给定 g, n ，令 $h = g + 5n$ 。包含 h 的基本三角形顶点满足 $\|v - g\| \leq 995$ 且 $\|v\| \leq 2795$ 。因 $(2795/990)^2 < 8$ 且范数层是整数，其层号至多 7，仍全在原 31 点中。它们的凸组合满足

$$\sum_i \lambda_i n^T (v_i - g) = 5. \quad (29)$$

因此至少一个顶点正投影不小于 5 米、距离不大于 995 米。即使坐标产生不超过 1 米的欧氏扰动，该点仍在严格前方且距离不超过 996 米。这避免只依赖理想实数下的发射边界相等式。实际双精度坐标误差远小于这一余量。

7.4 负观测的正确语义

问题四中的 `no_signal` 可能由无源、超出半径或处于背面导致，不能像全向情形那样直接删除 1000 米圆盘。若坚持更精细推断，应维护 (g, R, n) 的联合可行域：正观测要求距离及正半平面条件；当 $\|s - g\| \leq 1000$ 时，负观测可推出 $n^T(s - g) < 0$ ，但位置与方向不可分离处理。

当前实现选择不增加这一高维状态维护，而以正测向多边形、有限失信号恢复与完整光学覆盖完成局部任务。这是主动保持保守性，不是忽略定向条件：31 点发现定理和方向无关的光学清除正好给出了两个端点保障。

搜索调度、逐频道完成判据及 16 源上界提前停止均与问题三相同，只将七点骨架换为 31 点，并启用式(19)。骨架检测最多 $31 \times 20 = 620$ 次；这不包含局部检测，但已清除频道不再扫描。

8 统一求解流程、下界与策略筛选

8.1 部分可观测状态机与证书不变量

将机器狗搜索过程表示为部分可观测状态机。连续状态为位置 q_t 、各频道定位多边形和观测历史，离散状态为测向机当前频道、已接受动作日志及频道证书。记频道 c 的状态为

$$Z_c(t) \in \{U, D, C\},$$

其中 U 表示尚未发现， D 表示已发现但尚未确认清除， C 表示已确认清除；无源认证另由 $\text{Cert}_c = N$ 表示。全局状态为

$$X_t = (q_t, k_t, \{Z_c(t), V_c(t), P_c(t)\}_{c=1}^{20}, A_t),$$

其中 k_t 是测向机频道， V_c 是已完成的骨架检测点集合， P_c 是正示向形成的保守多边形， A_t 仅含已接受的请求及响应。令 V 表示当前题型的完整骨架：问题三为 $V_3 = \{0, \dots, 6\}$ ，问题四为 $V_4 = \{0, \dots, 30\}$ ；证书中的 V 按当前问题取 V_3 或 V_4 。

隐藏真实状态 ξ_c 可能为空，也可能包含位置、接收半径和定向方向；程序绝不读取 ξ_c 。令 $F_c(t)$ 为与该频道截至 t 时刻全部已接受响应相容的联合状态集合。我们的更新规则满足

$$\xi_c \in F_c(0) \implies \xi_c \in F_c(t), \quad F_c(t+1) \subseteq F_c(t). \quad (30)$$

正示向只加入真实误差楔形及安全距离帽；`no_signal` 在问题四中不单独删去位置圆盘；失败清除只记录失败位置。因而即便效率较低，式(30)仍不被启发式路线破坏。

频道完成证书定义为

$$\text{Cert}_c(t) = \begin{cases} C, & \text{收到该频道成功清除响应,} \\ N, & Z_c(t) = U, V_c(t) = V, \text{ 且所有检测均为有效 no_signal,} \\ U \text{ 或 } D, & \text{其他情况.} \end{cases} \quad (31)$$

这里 D 状态一旦出现就不能被后续无信号改写为 N 。因为一个已知存在的源不会因改变检测点而消失，且问题三、四的覆盖证书分别保证全向源或任意方向定向源至少被检

测一次，故

$$\forall c, \text{ Cert}_c \in \{C, N\}$$

是安全停止条件；确认清除 16 个不同频道是利用题目上界的另一种安全停止条件。这个定义比“当前待处理队列为空”更强，也比依赖字典中是否存在某个轨迹更稳定。

动作状态还需满足协议原子性：只有响应完整且 `accepted=true` 时，才推进 X_t ；截断响应重试时保留原请求编号和请求字节；不确定动作禁止发送不同的新动作。若已接受动作到达虚拟时间上限，先保留该动作的观测结果，再检查已有证书；若证书未完成则返回“不完整超时”，绝不再发送 `/exit`。这一点由 `SessionTerminated`、`terminal_reason` 和独立终端回归脚本共同实现。

8.2 正确性风险与时间代价分解

将风险拆为发现、定位、清除和停止四类：

$$R_D, R_L, R_C, R_S.$$

对于解析保证范围，要求对每一个合法世界均有

$$R_D = R_L = R_C = R_S = 0. \quad (32)$$

覆盖点集和凸包半平面证书解决 R_D ，保守多边形和完整光学外包矩形解决 R_L ，接口返回 `success` 才更新清除状态解决 R_C ，式(31)解决 R_S 。有限合成试验只能检查实现是否符合这些条件，不能把有限样本中的零失败率写成数学上的零风险。

总虚拟时间严格拆分为

$$T = T_{\text{move}} + T_{\text{measure}} + T_{\text{switch}} + T_{\text{fail}} + T_{\text{clear}} = \frac{L}{5} + 5M + S + 3F + 5K. \quad (33)$$

这一区分能够解释改进的作用：批量扫描主要减少 L ，共享停靠可能增加 M 但减少 L ，定向失信号恢复用少量移动和检测换取 F 下降，光学兜底增加失败清除次数但不依赖错误的负观测解释。只报告总时间而不拆分这些项，不能说明机制的真实贡献。

因此我们采用字典序决策：先保留证明域内的全清和逐频道证书，再最小化 T ，最后才考虑 M 和 L 。对没有完整证明的候选策略，只报告相对于安全基线的配对经验差

$$\widehat{\text{Reg}}(\hat{\pi}; \pi_0) = \frac{1}{n} \sum_{j=1}^n \left(\frac{T_j(\hat{\pi})}{N_j} - \frac{T_j(\pi_0)}{N_j} \right), \quad (34)$$

其中 n 是配对世界数， $\hat{\pi}$ 为候选策略， π_0 为安全对照。还须报告未清除数、证书状态与分项成本，不能把负的经验差解释为全局最短路。

8.3 覆盖半径下界和构型最小性边界

对区域 Ω 和有限检测点集 Q 定义覆盖半径

$$\rho_{\text{cov}}(Q; \Omega) = \max_{x \in \Omega} \min_{q \in Q} \|x - q\|. \quad (35)$$

完整覆盖的充要条件是 $\rho_{\text{cov}}(Q; \Omega) \leq r$ 。若 n 个半径 r 圆盘覆盖面积为 $|\Omega|$ 的区域，则有面积下界

$$r \geq \sqrt{\frac{|\Omega|}{n\pi}}. \quad (36)$$

该下界忽略重叠和边界效应，通常不紧；因此本文不把 31 点或七点直接称为任意点集中的全局最少点方案。

但是，对问题三采用的保守覆盖半径 $900 = 1800/2$ ，单个检测点位于距圆心 d 处时，其对目标圆周的最大覆盖半角不超过

$$\alpha_{\max} = \arcsin(900/1800) = 30^\circ.$$

六个半径 900 米圆盘要覆盖整条圆周，必须六个点均达到最大 60 度覆盖弧并恰好分割圆周；这迫使它们位于半径 $900\sqrt{3}$ 的规则环上，但此时原点距任一环点为 1558.846 米，原点未被覆盖。因此，在该保守半径和圆心加环的证明框架内，七点是精确下界；这不等价于物理半径固定为 1000 米时任意六点构型都不可能。

对于问题四，31 点是半径 2790 米截断下的完整三角格集合：

$$i^2 + ij + j^2 \leq (2790/990)^2 = 961/121 < 8.$$

其壳层 0, 1, 3, 4, 7 的点数为 1, 6, 6, 6, 12。对当前三角格的方向无关证书，每个点都是不可随意删除的：在其对边中点向顶点方向略微移动源位置，顶点是唯一同时满足正投影和 1000 米距离的格点。该结论只证明“当前格点证书的不可删性”，不宣称所有可能的 31 点外部布局都是全局最优。

8.4 算法流程、复杂度与可复现性

主算法可概括为：

1. 初始化目标圆域外切多边形、骨架义务 V 、频道状态 $Z_c = U$ 及当前位置；
2. 在当前骨架点按频道顺序逐一检测。返回 `no_signal` 只记录义务；返回 `direction` 则以真实读数裁剪 P_c ；返回 `near` 立即调用清除；
3. 在所有骨架义务完成前，依据当前位置、未清除频道数和已有 P_c 的 MEC 中心选择“继续覆盖”或“定位已有目标”；
4. 对已发现频道最多进行 6 次局部测向。问题四丢失信号时执行有限退回反侧策略；任何时候只用实际正示向更新 P_c ；
5. 若 MEC 不能保证一次清除，使用覆盖整个外包矩形的 19.9 米光学圆盘蛇形路径；清除成功即将该频道置为 C ；
6. 全部频道均为 C 或 N ，或已确认 16 次成功清除时停止；遇到真实截止则输出终端原因，不伪造全清。

本文最终增强策略的有限虚拟时间上界见第 5.4 节。该证明按最近正测点周围的反射半缩递推，分别支付任务入口、最多五条测点连线、小区域试清、光学接入和蛇形覆盖；没有将关于测点的反射误当成关于真实源的等距变换。增强只收缩 Q3 骨架并滚动重排 Q4 骨架，不改变发现证书；区间自适应测点候选不纳入默认上界。

8.5 有对照的改进筛选

表 2: 未采用候选的配对对照；“节省”定义为对照时间减候选时间，负值表示候选增时

问题	候选	对照/s	候选/s	节省/s	更快/对	全清数 对照/候选
Q3	静态 2-opt	316.38	339.46	-23.07	33/200	200/200
Q3	自适应探测	311.80	337.36	-25.56	0/200	200/200
Q4	静态 2-opt	822.26	876.66	-54.40	40/200	200/200
Q4	自适应探测	805.77	810.69	-4.92	70/200	200/200

静态 2-opt 在出发前优化的路线不能预见局部清除后的实际位置；自适应测点评分则只优化正示向后几何不确定性的上界，并不是完整任务的价值函数。它可以减少 Q4 失败清除次数，却付出额外测向、移动和计算。表中结果不否定主动测向理论，只否定本次有限候选及权重配置的默认准入。

这两个负结果同样重要：模型升级的准入标准不是“公式更复杂”或“某个子指标更好”，而是相同证书下完整 T/N 具有稳定收益。最终策略仍采用简单固定偏移、问题四有限恢复和完整光学兜底，所有更复杂候选均不能绕过覆盖义务。

8.6 敏感性、压力测试与可证范围

对参数 θ ，在固定种子和停止规则下报告

$$S_Y(\theta) = \frac{[Y(\theta) - Y(\theta_0)]/Y(\theta_0)}{[\theta - \theta_0]/\theta_0},$$

其中 Y 可取 T/N 、检测次数、失败清除次数或回退比例；若基准为零则使用绝对差。已有实验改变横向偏移尺度 0.12, 0.22, 0.40 以及三角格距 900, 950, 990 米，并在最小接收半径、朝外边界源、聚集源和三种固定误差场下检查。结果只支持声明的合成分布，不外推为官方世界的概率结论。

最终报告同时保存均值、中位数、最近秩 p_{90} 、最坏值、距离、测量次数、切换次数、失败清除次数和全清计数。对配对世界使用式(34)而非合并所有源的总时间比值；这样源数不同造成的尺度差异不会被误认为策略收益。

9 有证据的搜索增强：环收缩与滚动路径

9.1 优化几何，而非减少证明义务

原七点骨架的 900 米覆盖保证留有 100 米接收余量，但这一余量不必全部保留。机器狗的移动速度仅为 5 米/秒，约 1559 米的骨架环会形成较长的空频道排查路径；可在仍小于 1000 米的覆盖半径内，把检测环向圆心收缩。该改变没有减少七个检测位置，也没有推测某频道不存在，而是为每个新位置重新给出完整覆盖证书。

取 $R = 1800$ ，原点加六个角间隔 60° 的环点，环半径为 d 。源的极径为 r ，选择角差不超过 30° 的环点，则在该方向最坏角度处，到最近检测点的距离为

$$f_d(r) = \min \left\{ r, \sqrt{r^2 + d^2 - \sqrt{3}dr} \right\}. \quad (37)$$

两项相等时 $r = d/\sqrt{3}$ ；交点之前原点更近，之后环点更近。在 $[d/\sqrt{3}, R]$ 上，根号内是凸二次函数，所以最大值在端点取得。只要 $d/\sqrt{3} \leq R$ ，整个圆域的精确覆盖半径为

$$\rho_{\text{ring}}(d) = \max \left\{ \frac{d}{\sqrt{3}}, \sqrt{R^2 + d^2 - \sqrt{3}Rd} \right\}. \quad (38)$$

这不仅给出上界：相邻环点的角平分线在 $r = d/\sqrt{3}$ 及 $r = R$ 处分别达到两项，因此是精确值，而不是网格采样估计。

要求 $\rho_{\text{ring}}(d) \leq q$ ，其中 q 是选用的安全接收半径，得到

$$d \leq \sqrt{3}q, \quad \frac{\sqrt{3}R - \sqrt{4q^2 - R^2}}{2} \leq d \leq \frac{\sqrt{3}R + \sqrt{4q^2 - R^2}}{2}. \quad (39)$$

当 $q = 900$ 时只有原构型的 $d = 900\sqrt{3}$ ；当允许 q 向物理下界 1000 靠近时，可选更小的 d 。本文不取恰好触及 1000 米的极限值，而固定 $d = 1200$ 米，计算得到

$$\rho_{\text{ring}}(1200) = 968.901572 \text{ m}, \quad 1000 - \rho_{\text{ring}}(1200) = 31.098428 \text{ m}.$$

该余量远大于双精度和坐标舍入误差，也允许进行明确的覆盖灵敏度分析。若检测中心位置扰动不超过 η 米，三角不等式使覆盖半径至多增加 η ；故在 $\eta < 31.09$ 米时仍可保持物理距离覆盖。这不是题目要求的定位误差容许范围，而是布点构型的稳定性结论。

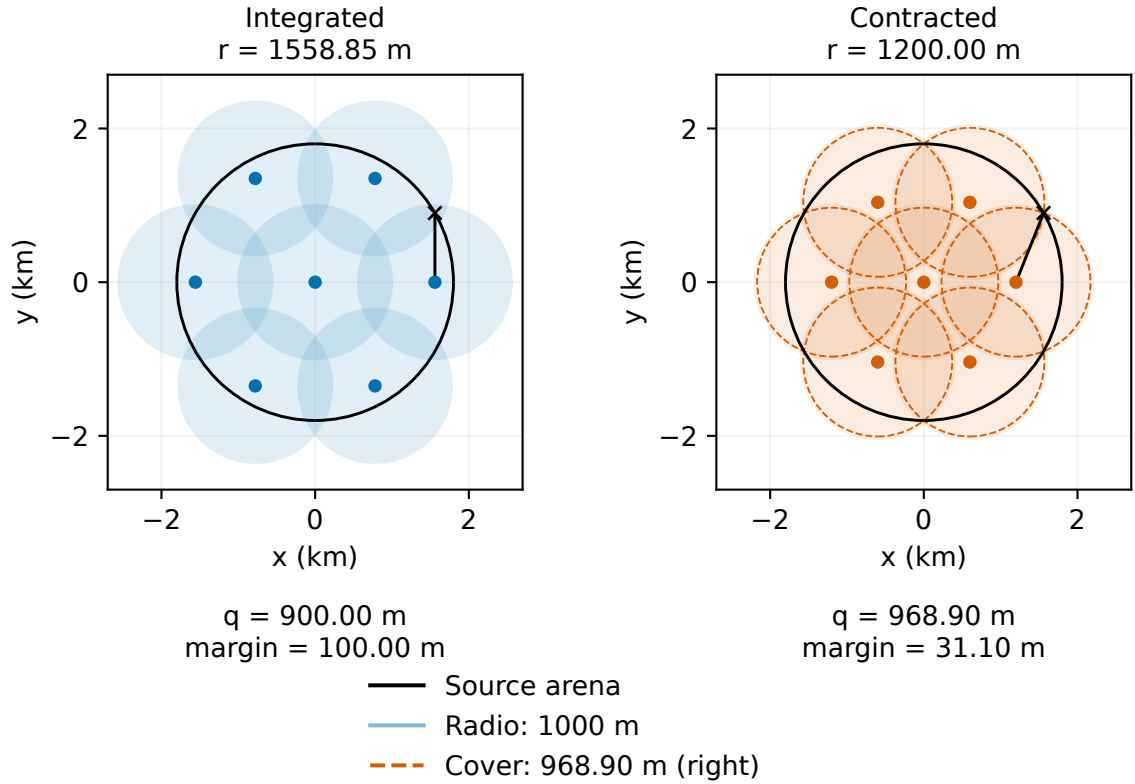


图 4: 七点覆盖的余量-行程权衡: 1200 米收缩环保持完整接收覆盖, 不删去任何频道的检测义务。

9.2 环半径的行程下界与候选筛选

覆盖半径最小并不等价于任务时间最小。对原点与六个正六边形环点, 若只考虑从原点出发、访问全部六个环点且不要求返回的骨架路径, 则其精确最短长度为

$$L_{\text{skeleton}}^{\min}(d) = 6d. \quad (40)$$

证明很直接: 起点到第一个环点的距离为 d , 其后五条环点间连线的长度均不小于相邻点间距 d , 故总长至少 $6d$; 沿圆周依次访问恰好达到该下界。这个结论不含局部定位支线, 也不适用于提前确认 16 源后省略剩余点的轨迹, 但清楚解释了环收缩为何能减少骨架层的移动成本。

在本文支持的区间 $1200 \leq d \leq 900\sqrt{3}$ 上, 式(38)由边界项控制, 且

$$\rho'_{\text{ring}}(d) = \frac{d - \sqrt{3}R/2}{\sqrt{R^2 + d^2 - \sqrt{3}Rd}} \leq 0, \quad (L_{\text{skeleton}}^{\min})'(d) = 6.$$

因此扩大检测环会改善最坏接收余量, 却同时提高完整骨架行程下界; 两者构成可解释的权衡, 而非某一个指标越小越好。相较原半径, $d = 1200$ 把纯骨架下界由 9353.07 米降到 7200 米, 但完整任务仍可能因发现顺序与定位支线改变而增时, 不能把这一下界差额直接当作实际节省。

为避免只比较两个半径就把 1200 米称为最优，本轮在既有开发种子 1000000 至 1000149 上比较 8 个候选半径，保持其余策略不变。表3给出 150 个世界的开发均值；1200 米胜出，1250 米为次优。早期扩展比较曾复用 1100000 组数据，本表改用未参与此前筛选的 2300000 至 2300499 进行最终确认，冻结候选后不再调参。保存的 2200 次执行包括 8×150 次开发运行与 2×500 次新世界配对运行，不是 2200 个独立世界。

表 3: Q3 安全环半径筛选；时间单位为秒/源，破折号表示未进入样本外复核

环半径/m	解析覆盖/m	开发 T/N	新样本 T/N	相对对照节省/s	选择
1200	968.90	272.97	272.71	2.38	保留
1250	951.52	274.88	275.09	—	— 次优
1300	936.48	277.58	—	—	—
1350	923.91	281.27	—	—	—
1400	913.91	286.18	—	—	—
1450	906.56	293.27	—	—	—
1500	901.92	298.08	—	—	—
1550	900.04	305.55	—	—	—

1200 米与 1250 米在 500 个新检验世界均全清；前者平均为 272.71 秒/源，后者为 275.09 秒/源。配对节省均值为 2.377 秒/源，其近似 95% 置信区间为 [1.608, 3.145]，353/500 个世界更快。故保留 1200 米作为当前默认，不引入新策略分支。此结论仅针对声明的候选集与合成分布，不声称连续半径、所有源分布或全部在线策略中的全局最优。

9.3 从静态 2-opt 改为剩余任务的滚动重排

固定骨架路线的静态 2-opt 只能减少骨架间纯移动，不能预知局部定位支线结束后机器狗在哪里。因此本文改为每次准备选择骨架任务时，从已接受动作确定的当前位置出发，在未完成骨架点集合上重新建立最近邻开放路径，再做 2-opt 交换；只执行新的第一个骨架点，下一决策时重新规划。

设当前路径包含边 (a, b) 和 (c, d) ，交换区间方向可使其变为 (a, c) 和 (b, d) ，路径变化为

$$\Delta L = \|a - c\| + \|b - d\| - \|a - b\| - \|c - d\|. \quad (41)$$

仅当 $\Delta L < 0$ 且超过数值容差才接受。若区间到达开放路径终点，不计不存在的终端边。交换只反转索引子序列，故点集和逐频道义务不变；算法不允许因路程较远而删除圆外点。一次扫描 $O(n^2)$ 个交换，严格下降在有限排列集合内终止。本文只给出每次 2-opt 扫描的解析复杂度；实验结果未单独记录其 CPU 时间或总扫描次数。

这一滚动方案的安全性与时间最优性必须分开：安全性仍由完整骨架和光学后备证明，2-opt 仅是当次余程的局部搜索。下一次测向发现新源后，未来真实路线可能不同；因此每次的短路不保证每个世界总时间都缩短。Q3 收缩环后仅采用最近点调度，因为开发样本中叠加滚动优化的收益很小；Q4 采用滚动余点排序，因为开发比较显示其能减少实际长距离骨架路程。

9.4 覆盖点数的下界与格点不可删性

Q3 的 900 米覆盖构造和边界弧下界见式(26)及第8.3节；它们共同给出该保守半径下的精确七点结论，不是 1000 米物理接收下界下任意布局的全局最优性。

Q4 的 31 点结论同样只针对当前 990 米三角格。逐点见证脚本在每个格点附近构造唯一可接收的定向源，因此删除当前集合中的任一点会破坏该证书；这不是对任意替代布局的全局点数下界。再考察圆周上的径向外发射源。探测点范数为 d 、极角差为 ϕ 时，前向条件为 $d \cos \phi \geq R$ ；接收还要求 $d^2 + R^2 - 2dR \cos \phi \leq q^2$ 。 $d < R$ 或 $d > R + q$ 均不可能接收，故单点最大覆盖角半宽为

$$\alpha_4 = \max_{R \leq d \leq R+q} \arccos \left(\max \left\{ \frac{R}{d}, \frac{d^2 + R^2 - q^2}{2dR} \right\} \right) = \arctan \frac{q}{R}. \quad (42)$$

第一项随 d 递减，第二项在 $d \geq R$ 递增，交点 $d = \sqrt{R^2 + q^2}$ 给出等号。 $q = 1000, R = 1800$ 时， $\alpha_4 = 0.50710$ 弧度，故至少需要 $\lceil \pi/\alpha_4 \rceil = 7$ 个圆外探测位置。该特殊边界下界与当前 31 点不可删性不同，不能推出 31 点全局最少。

表 4: 覆盖下界及当前固定集合的不可删性证据

Statement	Value Interpretation
Q3 q=900 lower bound	7 six boundary arcs at equality; origin uncovered
Q3 fixed seven-set at q=1000	7 five ring arcs leave a boundary gap
Q4 outer-boundary lower bound	7 arbitrary placements; outer detectors only
Q4 current lattice witnesses	31 target d=856.365–856.365 m
Q4 nearest other lattice point	495.001 m every witness remains outside near threshold

9.5 隔离开发和检验的数据分工

环收缩和滚动路线的开发只使用 1000000 至 1000149 的 150 个种子；该组比较原环/收缩环、最近点/滚动策略，不作为最终效率宣称的检验集。配置冻结后，1100000 至 1100499 用于每问 500 个随机世界的配对比较；1110000 起的 30 个种子用于每问三种压力几何与三种误差场。所有比较中，基础集成和增强使用同一世界、同一固定地点误差及同一停止证书。主增强比较为 3080 次运行，环半径筛选另有 2200 次运行；两者均单独保存在对应 JSON 中，不与原 4430 次基础对照混为同一分母。

表 5: 冻结配置后的样本外比较；每问 500 个同种子世界，均全清

问题	策略	均值/s	中位数/s	P_{90} /s	全清/世界
Q3	原集成	314.98	311.03	377.59	500/500
Q3	增强	275.68	272.95	326.75	500/500
Q4	原集成	811.73	830.40	1029.89	500/500
Q4	增强	797.76	804.75	1015.10	500/500

Q3 增强平均为 275.68 秒/源，比同世界基础集成方案降低 12.48%；Q4 增强平均为 797.76 秒/源，降低 1.72%。对应逐世界节省的**配对均值近似 95% 置信区间**分别为 [37.04, 41.55] 秒/源和 [9.69, 18.25] 秒/源；这不是逐世界差值的取值区间。两者下端均大于零。Q3 在 478/500 个世界更快，Q4 在 328/500 个世界更快，故不宣称增强对所有实例单调占优。

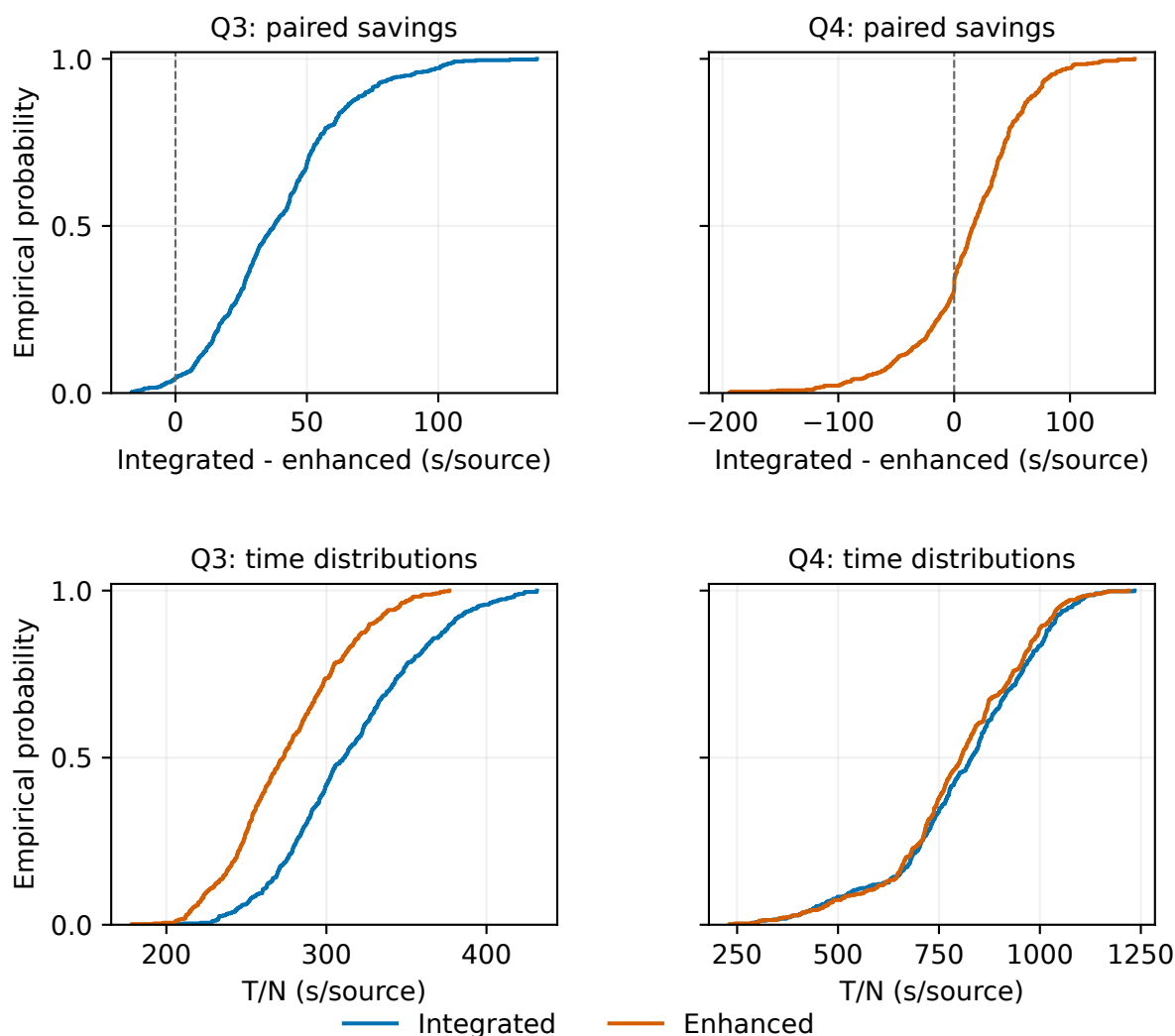


图 5: 样本外配对差异。正节省代表增强更快，零左侧案例必须保留而不能筛掉。

9.6 收益来自哪里：按每世界每源分解

表6先把每个世界的各项成本除以该世界源数，再对世界求平均。未舍入数据满足计费恒等式；本表逐项保留两位小数后，分项之和与总计可相差 0.01 秒/源，另有逐动作微秒舍入误差。不能改用所有世界总时间除以总源数，那会增加大源数世界的权重。

表 6: 增强前后分项成本, 单位为秒/源

问题	策略	$L/(5N)$	$5M/N$	S/N	$3F/N$	$5K/N$	合计/s
Q3	原集成	253.14	47.77	8.88	0.18	5.00	314.98
Q3	增强	212.66	48.79	9.05	0.17	5.00	275.68
Q4	原集成	640.65	139.71	24.83	1.54	5.00	811.73
Q4	增强	627.01	139.42	24.76	1.56	5.00	797.76

Q3 收缩环主要减少纯移动, 允许稍多的检测来换取更短的路程; 这说明最少测向次数不是本题的正确目标。Q4 滚动路线保留同样的 31 点集合, 收益来自定位后实际出发位置对后续路线的反馈。只有完整计费式支持这些解释; 不以失败清除次数下降单独证明总效率提升。

9.7 压力分布下的效果边界

表 7: 新增压力检验: 每组 30 对世界, 表中节省为基础减增强

问题	场景	误差	原集成/s	增强/s	节省/s	更快/对	双全清/对
Q3	最小半径	平滑	315.47	283.89	31.58	29/30	30/30
Q3	最小半径	极端	321.55	290.82	30.74	28/30	30/30
Q3	最小半径	位置独立	316.46	284.65	31.81	30/30	30/30
Q3	边界朝外	平滑	304.93	329.07	-24.14	5/30	30/30
Q3	边界朝外	极端	311.96	330.86	-18.90	6/30	30/30
Q3	边界朝外	位置独立	307.60	332.36	-24.76	2/30	30/30
Q3	聚集	平滑	122.56	105.31	17.26	30/30	30/30
Q3	聚集	极端	122.77	105.41	17.36	29/30	30/30
Q3	聚集	位置独立	122.61	105.66	16.95	29/30	30/30
Q4	最小半径	平滑	831.73	824.61	7.11	21/30	30/30
Q4	最小半径	极端	832.06	819.95	12.11	17/30	30/30
Q4	最小半径	位置独立	823.53	823.04	0.48	18/30	30/30
Q4	边界朝外	平滑	879.59	875.87	3.73	17/30	30/30
Q4	边界朝外	极端	868.11	866.20	1.91	18/30	30/30
Q4	边界朝外	位置独立	881.06	875.84	5.21	19/30	30/30
Q4	聚集	平滑	274.33	307.89	-33.57	1/30	30/30
Q4	聚集	极端	277.62	306.33	-28.72	1/30	30/30
Q4	聚集	位置独立	280.36	316.77	-36.40	1/30	30/30

各组的全清首先检验覆盖和终止不变量; 正负时间差则反映效率对几何、固定误差场和方向的依赖。不能把普通随机世界的平均改善直接推广为所有压力分布的改善, 也不能因某组增时而取消必要的圆外检测。压力场景用于揭示边界, 不用于反复调参直至所有行都变为正数。

9.8 改进结论和实施选择

本文把“能证明不漏检”与“在样本中平均更快”视为两个独立闸门。候选只有同时保留原有证书、在所有列示运行中完成证书，并在与对照相同的完整计费式下表现稳定，才进入默认策略；否则保留为可复核的负对照，而不是删除其结果。

表 8: 候选机制的准入矩阵

机制	不变量或安全条件	证据与代价	决策
Q3 收缩环	式(38)给出全圆域覆盖， $\rho_{\text{ring}}(1200) < 1000$	150 个开发世界筛选，500 个样本外配对世界；额外保留 31.10 米余量	采用
Q4 滚动路径	完整 31 点集合和逐频道检测义务不变	500 个样本外世界中滚动重排只改变访问顺序；不减少圆外点	采用
静态 2-opt	点集不变，故安全但不能预见定位支线终点	200 个同种子世界的完整 T/N 比较中均值增时	不采用
区间自适应测点	正示向评分有保守上界，失败时仍回到光学兜底	Q4 失败次数下降但新增测向和移动，完整时间未稳定改善	不采用
定向负观测裁剪	需维护 (g, R, n) 联合可行集，不能只删位置圆盘	当前实现不做未经证明的高维判空	不启用

该矩阵规定的是策略选择条件，而不是强制先扫完骨架再定位。在线程序仍在未完成骨架和已知源之间选择下一任务，保留所有欠缺的频道义务；任何局部评分都不能绕过完整覆盖或把失败清除改写成成功。“优化损失时间”与“证书失效”必须分别处理。

复现时先运行几何验证和终端回归，再运行基础实验与增强实验，随后由资产脚本从 JSON 重建表格、图形和轨迹，最后编译论文并检查正文页数。该顺序避免人工复制数字，也使每个图表都能追溯到固定代码、输入哈希和种子范围。由于官方测试尚未执行，本节所有“采用”只表示研究程序中的默认实现，不表示已通过正式竞赛测试。

10 实现与可信度验证

10.1 本地环境与决策程序隔离

合成环境严格执行：不同频道互不影响；有效半径及半平面共同决定可检测性；同一地点同一频道误差固定；距离不超过 5 米返回近源；清除与发射方向无关；移动按欧氏距离计时；清除不切换测向机频道。

策略只消费与正式接口同形的动作响应，不读取源真值。环境在退出后才提供总数等统计给评价器。实验脚本逐动作用式(1)重新累计时间，验证成功清除频道不重复、结果数量一致以及每个接受响应的虚拟时刻一致。模拟器自己报告的总时间不是唯一核验依据。

10.2 几何交叉验证与回归

表 9: 已实际执行的数值与接口检查, 采样检查不替代解析证明

Numerical / local check	Cases / points	Observed result
MEC reference	504	1.00e-09 m
Bearing containment	2000	4 readings/case
Optical cover samples	4000	Contained
Discovery samples	2000	Both geometries
Triple-lens samples	10000	Reception checked
Local HTTP fixture	1	Four checks passed

独立审查发现并修正了零误差精确示向度的背向射线、同方差信息矩阵的概率解释、极小格距可能导致不可运行的枚举规模以及终端动作越过虚拟期限的问题。前向射线、覆盖见证、收缩环和终端状态均保留了先失败后通过的回归; 程序支持格距限定为 [900, 999] 米, 不接受数值上几乎没有距离余量或会产生天文级点数的实参。

10.3 HTTP 层的可靠性与现实截止

客户端每个新动作使用唯一请求编号, 只在同一逻辑动作传输失败时复用同一编号及相同请求字节。收到完整 HTTP 响应后才发送下一动作, 同时检查 HTTP 状态与 `accepted`; 拒绝响应里的零虚拟时刻不得覆盖当前状态。

本地 HTTP 故障夹具让首次检测在服务端已执行后只发出半个响应体。客户端恢复时重发同 ID, 计时只增加一次; 附件示例动作序列最终仍为 199 秒。另一个夹具每 0.3 秒发一段响应, 故意使空闲 socket 超时不起作用; 客户端按 0.5 秒绝对动作期限退出, 实际约 0.51 秒, 且不推进未确认状态、不发送另一动作。

实现采用一个仅负责接收的守护线程与绝对单调时钟等待。超时后的线程即使晚些收到同一动作响应, 也不能修改策略状态或发送新动作; 客户端保留结果不明状态, 禁止推进。它不是并发控制机器狗, 只是防止持续慢速响应吞掉全部现实测试窗口。以上两项是本地接口健壮性验证, 不代表已与官方模拟器联调。

11 合成实验、消融和敏感性分析

11.1 分布说明及预先隔离的随机种子

常规世界中, 源数在 10 至 16 间离散均匀取值, 频道无放回抽取, 位置按面积均匀分布于圆盘, 接收半径在 1000 至 1500 米间均匀取值。问题三全为全向源; 问题四随机选择至少一个但非全部为定向源, 方向均匀分布。这些是合成环境的明示设计, 不是对官方测试数据分布的猜测。

误差包含三种固定地点场: 有界平滑正弦叠加; 随位置哈希取 $\pm 1^\circ$ 的极端场; 随位置哈希取区间均匀值的场。第三种也不意味着重复同一位置的读数独立。压力场景包

括所有半径降至 1000 米、16 个圆周源中 15 个向外发射，以及 16 个源聚集在圆边缘附近的 35 米小圆盘内。

丢信号恢复的开发比较使用种子 500000 至 500099。最终随机配对比较使用 600000 至 600499；恢复消融使用 610000 至 610099；压力场景使用 700000 起的 20 个种子；参数扰动使用 800000 和 900000 起的 30 个种子。参数冻结后不根据最终结果反复改动它们。

最终结果文件记录 4430 次完整运行，包括三种策略在相同世界中的重复运行、压力试验和消融；不能把它称为 4430 个独立随机世界。主对照的统计单位是每问 500 个世界中的每个 T/N ，不是把所有世界总时间相加后除以总源数。

11.2 随机世界的效率比较

表 10: 每问 500 个同种子世界的三策略比较，时间单位为秒/源；非官方成绩

Policy	Mean T/N (s)	Median (s)	p_{90} (s)	Complete
Q3 baseline	515.57	513.83	583.17	500/500
Q3 batched	334.31	330.38	396.93	500/500
Q3 integrated	310.67	305.79	375.98	500/500
Q4 baseline	1046.74	1059.30	1282.14	500/500
Q4 batched	810.57	824.63	1034.11	500/500
Q4 integrated	803.28	807.94	1032.51	500/500

基础集成策略在本组世界的平均单源总耗时为 310.67 与 803.28 秒，相对逐源往返基线分别降低 39.74% 与 23.26%。这组数据用于基础机制消融；本轮增强的独立样本外结果见第9节，两个不同种子组不直接混比均值。

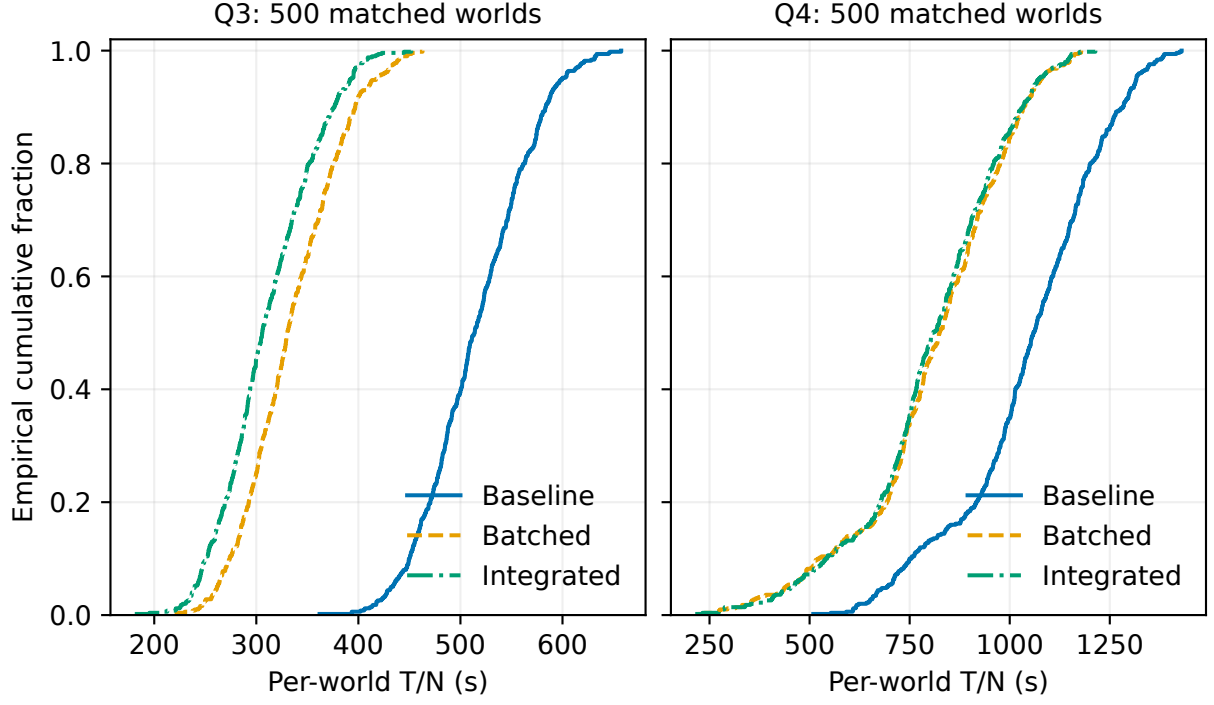


图 6: 配对随机世界的单源任务时间分布。不同方案采用相同世界、覆盖与局部定位条件，变化主要来自调度及观测共享。

统计程序对基线 A 与基础集成策略 C 的逐世界差值 $d_j = T_j^A/N_j - T_j^C/N_j$ 给出均值和近似 95% 区间 $\bar{d} \pm 1.96s_d/\sqrt{n}$ 。这是蒙特卡洛比较的不确定性，不是未知官方成绩的置信区间。B 到 C 的配对共享消融也单独记录，避免将全部路程收益归因于一个机制。对任意保持清除证书不变的候选策略，两个同世界运行的时间差必须按完整成本恒等式解释：

$$\Delta T = \frac{\Delta L}{5} + 5\Delta M + \Delta S + 3\Delta F + 5\Delta K. \quad (43)$$

定义 Δ 为候选减对照。真正更快当且仅当 $\Delta T < 0$ ；两者均全清时 $\Delta K = 0$ ，其他分项的所有有符号变化都应按系数计入。移动增多也可能被检测、切换或失败清除减少抵消。该判据适用于同一世界内比较，不把局部子指标直接等同于完整任务时间。

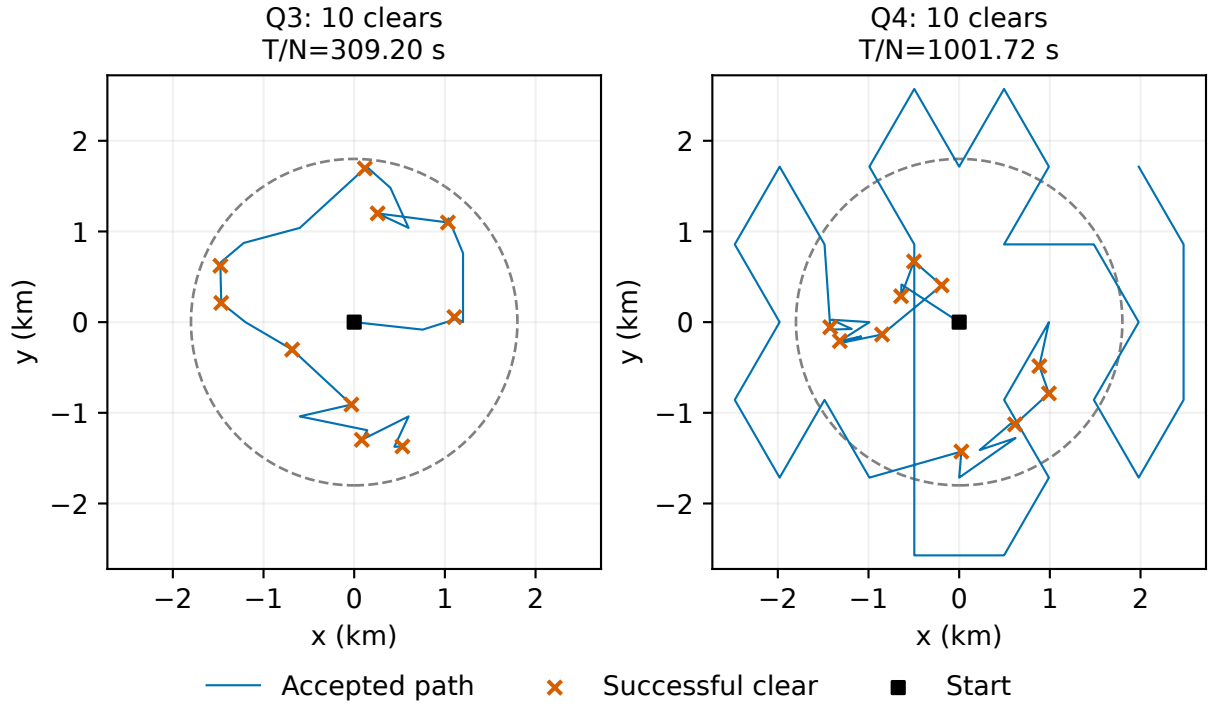


图 7: 固定示例世界中最终增强策略的完整实际轨迹。点标识的是观察到的成功清除位置，不是程序预先读取的源真值；问题四必须保留圆外搜索。

11.3 极端位置与误差场

表 11: 基础集成策略压力测试：每个问题、场景及误差模式各 20 次

Geometry	Error field	Q3 mean (s)	Q4 mean (s)	n/Q
Min. radius	Smooth	336.40	877.21	20
Min. radius	$\pm 1^\circ$ hash	342.17	885.06	20
Min. radius	Uniform hash	335.95	882.62	20
Outward boundary	Smooth	308.78	870.57	20
Outward boundary	$\pm 1^\circ$ hash	303.50	880.52	20
Outward boundary	Uniform hash	298.23	879.97	20
Clustered	Smooth	107.67	352.68	20
Clustered	$\pm 1^\circ$ hash	107.74	361.78	20
Clustered	Uniform hash	107.89	347.05	20

所有列示压力运行均完成全清。朝外边界场景尤其检验“圆内无信号不等于无源”和圆外格点是否完整；极端误差场检验是否错误地把多次读数当成可平均消失的噪声；聚集源检验共享停靠与多频道处理。全清结果与解析覆盖结论一致，但不能仅从有限次试验推断所有可能世界都会成功，后者仍依赖前文的集合包含与覆盖证明。

11.4 丢信号恢复消融

表 12: 问题四同种子恢复消融：每场景 100 个世界，只有失信号处理开关不同

Scenario	T/N off	T/N on	Fails off	Fails on	Pairs
Random	807.03	803.90	68.49	8.11	100
Min. radius	839.95	813.87	96.19	8.34	100
Outward boundary	1147.19	879.96	676.69	34.38	100
Clustered	357.32	312.94	104.63	5.13	100

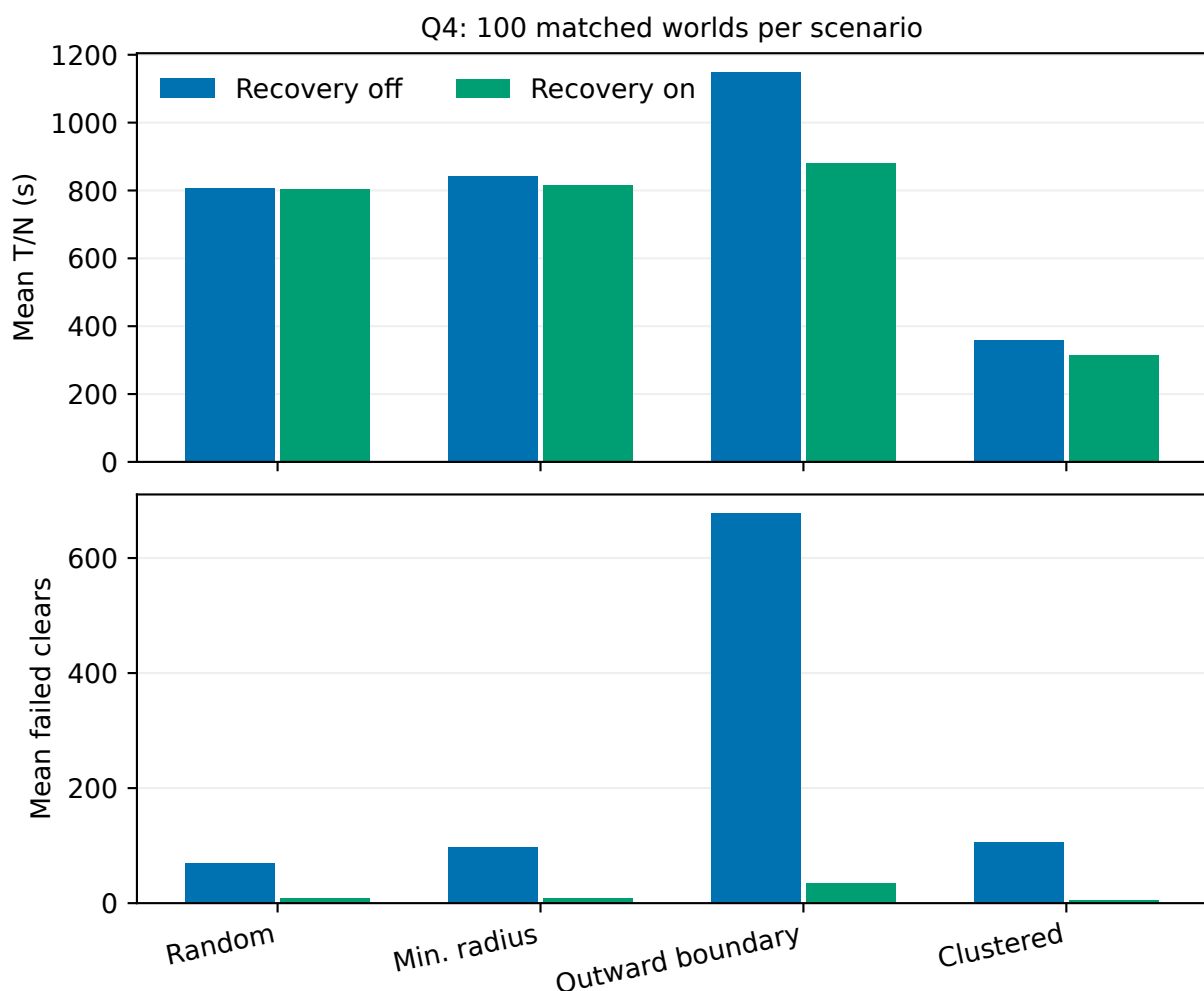


图 8: 有界恢复与直接光学兜底的对照。恢复的主要目的在于降低不利方向场景中大范围光学逐点尝试的开销，而非保证每个普通世界更快。

开发阶段发现，单纯尝试中心两侧后立即覆盖整个窄长区域，在朝外边界场景会产生大量失败光学尝试。有限退回和反侧测向针对这一实际问题而增加；恢复失败时仍使用同一全区域后备覆盖，因此增强没有牺牲清除保证。恢复减少失败清除次数不必然按比例降低总时间，因为新的恢复移动与检测同样计费，应以式(1)的完整结果决定取舍。

11.5 参数敏感性与局限

表 13: 横向测向尺度和三角格距的独立敏感性试验, 各组 30 次

Problem	Setting	Mean (s)	p_{90} (s)	n
Q3	$\lambda = 0.12$	328.57	382.88	30
Q3	$\lambda = 0.22$	329.04	385.26	30
Q3	$\lambda = 0.4$	329.89	385.18	30
Q4	$\lambda = 0.12$	883.89	1067.16	30
Q4	$\lambda = 0.22$	887.07	1060.41	30
Q4	$\lambda = 0.4$	888.82	1066.63	30
Q4	$s = 900$ m	809.26	1140.59	30
Q4	$s = 950$ m	736.95	1006.62	30
Q4	$s = 990$ m	765.66	1005.72	30

横向测向尺度围绕默认 0.22 扰动为 0.12、0.40, 格距比较 900、950、990 米。格距越小并不自动越快: 它减少到可接收点的最坏距离, 却增加骨架节点、检测和移动。离散点集的格层变化和贪心路径会使耗时随参数不完全单调。默认 990 的选择基于简洁的 31 点结构与距离余量, 不把一次敏感性样本中的最小均值重新命名为普适最优格距。

当前主要局限包括: 没有对官方世界分布或误差场进行标定; 对负观测的联合位置-方向信息利用不充分; 任务评分为滚动启发式而非全局最短路求解; 光学覆盖可能包含多边形之外的冗余单元; 本地运行的毫秒量级不含官方 HTTP 往返和界面日志开销。上述局限限制效率结论的外推范围, 但不应通过未经证明的概率停止、错误负观测裁剪或隐藏测试真值来消除。

12 论文结论、交付和后续正式测试

12.1 四问的直接结论

1. 有界交会多边形的直径由最远顶点对给出; 空集、无界集必须另行识别。直径为区域直径的圆不一定能覆盖定位区域, 实际一次保证清除应检验最小包围圆半径。
2. 第二点应兼顾距离和交会角。三圆交给出了无需目标距离的全向安全接收候选区域; 其内部横向点配合后验可行域评分形成可执行选点算法, 但单次读数不足以唯一确定时间最优点。
3. 全向源使用七点覆盖骨架、逐频道完成证书、保守测向定位与有限光学兜底。批量扫描和复用停靠点可以减少重复路程。
4. 混合源使用含 18 个圆外点的 31 点三角格, 以凸包半平面证书保障任意发射方向的发现; 无信号不直接排除附近源, 清除则利用方向无关的光学条件。

12.2 怎样将研究稿升级为可提交成果

按照官方格式规范, 电子版从摘要页开始且不含身份页; 摘要原则上不超过一页, 摘要之后正文不超过 30 页, 附录另计, 附录应包含全部完整可运行代码 [3]。本稿不设

置目录，附录代码与支撑材料相对应，配套复现说明列出环境和命令。

官方模拟器已经明确搁置，因此本稿**不填写虚构的三次正式测试表**。恢复可用后，应先完成对应问题的不限次官方演练，核对成功比例、全部动作计时和现实耗时；固定程序后分别进行问题三、四的三次正式测试。从界面抄录实际案例编码、实际清除数、平均总耗时和程序时间，从模拟器导出六份原名加密日志，并核对上传状态。这些日志不能由本地 JSONL 替代。

官方协议请求日志含参赛队号，只是私有工作材料，不能未经脱敏放入匿名论文支撑包。官方规定要求 AI 输出逐项人工审查、参赛队主导核心建模 [4]；多代理审查或自动测试都不是人工核验。完成上述环节之前，交付状态必须保持研究复核版，而非“保证获奖”或“已经满足提交要求”。

AI 工具使用声明

本研究过程中使用了 AI 工具，参与核心模型推导、代码实现、实验设计、结果整理、独立自动审查及论文写作，详细过程见支撑材料中的《AI 工具使用详情》。本稿尚无真实参赛队完成逐项人工核验的记录，不将自动审查写作人工审查；该声明用于如实披露研究过程，不能替代正式提交前应完成的参赛队责任。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题：无线电干扰源的快速自动定位与清除，以及附件 1、附件 2[Z]. 2026. https://www.mcm.edu.cn/html_cn/node/27b6e148f8113f09b0269f64a02629fb.html.
- [2] Welzl E. Smallest enclosing disks (balls and ellipsoids)[C]//New Results and New Trends in Computer Science. Lecture Notes in Computer Science, 555. Springer, 1991: 359–370. DOI: 10.1007/BFb0038202.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）[EB/OL]. https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd866398f6df0caa3.html.
- [4] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定（2026 年试行）[EB/OL]. https://www.mcm.edu.cn/html_cn/node/fef94648f2836ab6cc81586f4c38512b.html.

A 匿名复现附录

本附录只列出匿名包中实际存在、可供复现使用的源文件。代码路径均相对于本文件所在的 report/目录；结果路径均相对于匿名包项目根目录；不含个人绝对路径、队伍身份或私有凭据。主文件应在导言区加载 listings 并设置合适的 Python 样式。

A.1 运行与验证源代码

Listing 1: 规划器与恢复策略

```
1 """Coverage-certified search; consumes observations only, never simulator truth."""
2
3 from dataclasses import dataclass, field
4 from math import dist, hypot
5 from time import monotonic
6
7 from active_policy_candidate import choose_probe
8 from client import AmbiguousActionError, DeadlineExceeded
9 from coverage import directional_waypoints, omni_waypoints
10 from geometry import clip_bearing, enclosing_circle, initial_polygon, optical_cover
11 from route_policy_candidate import optimize_waypoints
12
13
14 class SessionTerminated(RuntimeError):
15     """The simulator closed after an accepted action reached its limit."""
16
17
18 class InconsistentObservations(RuntimeError):
19     """The specified bounded-error model no longer contains a possible source."""
20
21
22 @dataclass
23 class Track:
24     polygon: list = field(default_factory=initial_polygon)
25     readings: list = field(default_factory=list)
26     observations: list = field(default_factory=list)
27     failures: list = field(default_factory=list)
28
29
30 class Planner:
31     """Baseline clears detections immediately; integrated batches and shares stops.
32
33     Sampling and route heuristics affect time only. Completeness comes from the
34     waypoint certificate and the conservative, finite optical-cover fallback.
35     """
36
37     def __init__(
38         self,
39         environment,
40         problem=3,
41         strategy="enhanced",
42         spacing=990.0,
43         probe_scale=0.22,
44         max_probes=6,
45         loss_recovery=True,
46         anchor_policy=None,
47         probe_policy="fixed",
48         ring_radius=None,
49     ):
50         if problem not in (3, 4) or strategy not in (
51             "baseline",
52             "batched",
53             "integrated",
```

```

54         "enhanced",
55     ):
56         raise ValueError("Unsupported problem or strategy")
57     if anchor_policy is None:
58         anchor_policy = "rolling" if strategy == "enhanced" and problem == 4 else "nearest"
59     if ring_radius is None and strategy == "enhanced" and problem == 3:
60         ring_radius = 1200.0
61     if anchor_policy not in ("nearest", "two_opt", "rolling"):
62         raise ValueError("Unsupported anchor policy")
63     if probe_policy not in ("fixed", "adaptive"):
64         raise ValueError("Unsupported probe policy")
65     if not 900 <= spacing <= 999:
66         raise ValueError("Directional lattice spacing must be in [900, 999] m")
67     if not 0 < probe_scale <= 1 or max_probes < 1:
68         raise ValueError("Invalid localization policy parameters")
69     self.environment = environment
70     self.problem = problem
71     self.strategy = strategy
72     self.waypoints = (
73         omni_waypoints(ring_radius) if problem == 3 else directional_waypoints(spacing)
74     )
75     self.anchor_policy = anchor_policy
76     self.anchor_order = optimize_waypoints(self.waypoints) if anchor_policy == "two_opt" else []
77     self.anchor_rank = {index: rank for rank, index in enumerate(self.anchor_order)}
78     self.pending = set(range(len(self.waypoints)))
79     self.visited = []
80     self.tracks = {}
81     self.cleared = set()
82     self.scan_records = {ch: set() for ch in range(1, 21)}
83     self.position = (0.0, 0.0)
84     self.radio_channel = 1
85     self.virtual_time = 0.0
86     self.deadline = float("inf")
87     self.max_virtual_duration = float("inf")
88     self.terminal_reason = None
89     self.observation_points = {ch: [] for ch in range(1, 21)}
90     self.channel_state = {ch: "unresolved" for ch in range(1, 21)}
91     self.probe_scale = probe_scale
92     self.max_probes = max_probes
93     self.loss_recovery = bool(loss_recovery) and problem == 4
94     self.fallbacks = 0
95     self.probe_policy = probe_policy
96     self.adaptive_probes = 0
97     self.termination = None
98
99     def action(self, path, point=None, channel=None):
100         if self.terminal_reason is not None:
101             raise SessionTerminated(self.terminal_reason)
102         if monotonic() >= self.deadline - 2.0 and path != "/exit":
103             raise TimeoutError("Insufficient real time: completeness not certified")
104         response = self.environment.act(path, position=point, channel=channel)
105         if response.get("accepted") is not True:
106             raise RuntimeError("Rejected action; cached state was not advanced")
107         self.virtual_time = float(response["virtual_time_s"])
108         if path == "/enter":
109             self.max_virtual_duration = float(response.get("max_virtual_duration_s", float("inf")))
110         if point is not None:
111             self.position = tuple(point)
112         if path == "/measure":
113             self.radio_channel = channel
114         if path in ("/measure", "/clear") and self.virtual_time >= self.max_virtual_duration:
115             self.terminal_reason = "virtual_timeout"
116         return response
117
118     def clear(self, channel, point, guaranteed=False):
119         result = self.action("/clear", point, channel)["clear_result"]
120         if result == "success":

```

```

121         self.cleared.add(channel)
122         self.channel_state[channel] = "cleared"
123         self.tracks.pop(channel, None)
124         return True
125     if result != "no_target_in_range":
126         raise RuntimeError(f"Unknown clear result: {result}")
127     if guaranteed:
128         raise InconsistentObservations("Certified radius-20 clearing failed")
129     if channel in self.tracks:
130         self.tracks[channel].failures.append(tuple(point))
131     return False
132
133     def measure(self, channel, point):
134         result = self.action("/measure", point, channel)
135         status = result["measure_result"]
136         self.observation_points[channel].append(tuple(point))
137         track = self.tracks.get(channel)
138         if track is None and status == "direction":
139             track = Track(observations=self.observation_points[channel])
140             self.tracks[channel] = track
141         if track is not None:
142             track.observations = self.observation_points[channel]
143         if status == "near":
144             self.channel_state[channel] = "detected"
145             if self.terminal_reason is None:
146                 self.clear(channel, point, guaranteed=True)
147         elif status == "direction":
148             self.channel_state[channel] = "detected"
149             track.polygon = clip_bearing(track.polygon, point, result["svd_deg"])
150             if not track.polygon:
151                 raise InconsistentObservations(
152                     f"Empty feasible region on channel {channel}"
153                 )
154             track.readings.append((tuple(point), float(result["svd_deg"])))
155         elif status != "no_signal":
156             raise RuntimeError(f"Unknown measure result: {status}")
157         # No-signal observations are not convex halfplanes. In Q4 in particular,
158         # deleting a 1000-m disk would silently exclude real sources.
159         return status
160
161     def scan_anchor(self, index):
162         point = self.waypoints[index]
163         channels = [ch for ch in range(1, 21) if ch not in self.cleared]
164         channels.sort(key=lambda ch: (ch != self.radio_channel, ch))
165         for channel in channels:
166             if channel in self.cleared:
167                 continue
168             self.measure(channel, point)
169             self.scan_records[channel].add(index)
170             if len(self.cleared) == 16:
171                 self.termination = "known_count_upper_bound_16"
172                 break
173             if self.terminal_reason is not None:
174                 break
175             if self.strategy == "baseline" and channel in self.tracks:
176                 self.localize(channel)
177             if len(self.cleared) == 16:
178                 self.termination = "known_count_upper_bound_16"
179                 break
180             if self.terminal_reason is not None:
181                 break
182         self.pending.discard(index)
183         if index not in self.visited:
184             self.visited.append(index)
185
186     def optical_finish(self, channel):
187         self.fallbacks += 1

```

```

188     cover = optical_cover(self.tracks[channel].polygon, radius=19.9)
189     if not cover:
190         raise InconsistentObservations("An empty region cannot certify clearing")
191     start = min(range(len(cover)), key=lambda k: dist(self.position, cover[k]))
192     # Rotate the certified snake to the nearest cell. At most one long
193     # wraparound jump; the remaining edges are adjacent rectangle cells.
194     order = cover[start:] + cover[:start]
195     for point in order:
196         if self.clear(channel, point):
197             return
198         if self.terminal_reason is not None:
199             return
200     raise InconsistentObservations("Exhausted a certified optical cover without clearing")
201
202 def localize(self, channel):
203     losses = 0
204     failed_probe = None
205     for probe_index in range(self.max_probes):
206         if channel in self.cleared or self.terminal_reason is not None:
207             return
208         track = self.tracks[channel]
209         center, radius = enclosing_circle(track.polygon)
210         if max(dist(self.position, p) for p in track.polygon) <= 19.9:
211             self.clear(channel, self.position, guaranteed=True)
212             return
213         if radius <= 19.9:
214             self.clear(channel, center, guaranteed=True)
215             return
216         # When the remaining uncertainty is small, a 3-s failed optical
217         # attempt can cost less than travelling to take another bearing.
218         if radius <= 38 and center not in track.failures:
219             if self.clear(channel, center):
220                 return
221             if self.terminal_reason is not None:
222                 return
223             self.optical_finish(channel)
224             return
225         previous = track.readings[-1][0]
226         dx, dy = center[0] - previous[0], center[1] - previous[1]
227         length = hypot(dx, dy)
228         if length < 1e-8:
229             self.optical_finish(channel)
230             return
231         unit, normal = (dx / length, dy / length), (-dy / length, dx / length)
232         if failed_probe is not None and self.loss_recovery:
233             vector = (failed_probe[0] - previous[0], failed_probe[1] - previous[1])
234             axial = vector[0] * unit[0] + vector[1] * unit[1]
235             lateral = vector[0] * normal[0] + vector[1] * normal[1]
236             point = (
237                 previous[0] + 0.5 * (axial * unit[0] - lateral * normal[0]),
238                 previous[1] + 0.5 * (axial * unit[1] - lateral * normal[1]),
239             )
240             if any(point == p for p in track.observations) or point in track.failures:
241                 self.optical_finish(channel)
242                 return
243         else:
244             point = None
245             if self.probe_policy == "adaptive":
246                 point = choose_probe(
247                     track.polygon,
248                     track.readings,
249                     self.position,
250                     self.problem,
251                     probes_remaining=self.max_probes - probe_index,
252                     rejected_points=track.failures + track.observations,
253                     probe_scale=self.probe_scale,
254                 )

```

```

255         if point is not None:
256             self.adaptive_probes += 1
257         if point is None:
258             offset = min(120.0, max(25.0, self.probe_scale * radius))
259             candidates = [
260                 (
261                     center[0] + sign * offset * normal[0],
262                     center[1] + sign * offset * normal[1],
263                 )
264                 for sign in (-1, 1)
265             ]
266             fresh = [
267                 p
268                 for p in candidates
269                 if p not in track.observations
270                 and all(p != s for s in track.failures)
271             ]
272             if not fresh:
273                 self.optical_finish(channel)
274                 return
275             point = min(fresh, key=lambda p: dist(self.position, p))
276             status = self.measure(channel, point)
277             if channel in self.cleared or self.terminal_reason is not None:
278                 return
279             if status == "no_signal":
280                 losses += 1
281                 if self.clear(channel, point):
282                     return
283                 if self.terminal_reason is not None:
284                     return
285                 failed_probe = point
286                 if losses >= 2 and not self.loss_recovery:
287                     self.optical_finish(channel)
288                     return
289             else:
290                 failed_probe = None
291             if channel not in self.cleared and self.terminal_reason is None:
292                 self.optical_finish(channel)
293
294     def share_current_stop(self):
295         """Fuse an extra bearing for known sources at an already reached stop."""
296         if self.strategy not in ("integrated", "enhanced") or self.terminal_reason is not None or len(self.
297         cleared) == 16:
298             return
299         for channel in list(self.tracks):
300             if channel in self.cleared:
301                 continue
302             track = self.tracks[channel]
303             center, radius = enclosing_circle(track.polygon)
304             if radius <= 19.9:
305                 continue
306             if any(self.position == p for p in track.observations):
307                 continue
308             if dist(self.position, center) - radius > 1500:
309                 continue
310             self.measure(channel, self.position)
311             if self.terminal_reason is not None:
312                 return
313
314     def next_anchor(self):
315         if self.anchor_policy == "two_opt":
316             return min(self.pending, key=lambda i: self.anchor_rank[i])
317         if self.anchor_policy == "rolling":
318             indices = sorted(self.pending)
319             order = optimize_waypoints([self.waypoints[i] for i in indices], self.position)
320             return indices[order[0]]
321         return min(self.pending, key=lambda i: (dist(self.position, self.waypoints[i]), i))

```

```

321 def _result(self):
322     complete = len(self.cleared) == 16
323     if not complete:
324         for channel in range(1, 21):
325             if channel in self.cleared:
326                 self.channel_state[channel] = "cleared"
327             elif (self.channel_state[channel] != "detected"
328                   and len(self.scan_records[channel]) == len(self.waypoints)):
329                 self.channel_state[channel] = "absent-certified"
330             else:
331                 complete = False
332                 break
333     else:
334         complete = True
335     if self.terminal_reason is not None:
336         self.termination = ("complete_after_" if complete else "incomplete_") + self.terminal_reason
337     return {
338         "problem": self.problem,
339         "strategy": self.strategy,
340         "cleared_channels": sorted(self.cleared),
341         "n_cleared": len(self.cleared),
342         "virtual_time_s": self.virtual_time,
343         "average_time_s": self.virtual_time / len(self.cleared)
344         if self.cleared else None,
345         "probe_policy": self.probe_policy,
346         "anchor_policy": self.anchor_policy,
347         "ring_radius_m": hypot(*self.waypoints[1]) if self.problem == 3 else None,
348         "loss_recovery": self.loss_recovery,
349         "adaptive_probes": self.adaptive_probes,
350         "channel_certificates": dict(self.channel_state),
351         "certificate_complete": complete,
352         "terminal_reason": self.terminal_reason,
353         "visited_anchors": len(self.visited),
354         "total_anchors": len(self.waypoints),
355         "optical_fallbacks": self.fallbacks,
356         "termination": self.termination,
357     }
358
359 def run(self):
360     try:
361         return self._run_body()
362     except (TimeoutError, ConnectionError, AmbiguousActionError,
363            DeadlineExceeded, SessionTerminated) as exc:
364         self.terminal_reason = self.terminal_reason or type(exc).__name__
365         self.termination = self.termination or "incomplete_" + type(exc).__name__
366         return self._result()
367
368 def _run_body(self):
369     entry_started = monotonic()
370     entered = self.action("/enter")
371     self.deadline = entry_started + entered["remaining_real_duration_s"]
372     self.scan_anchor(0)
373     while (self.pending or self.tracks) and self.terminal_reason is None:
374         if len(self.cleared) == 16:
375             self.termination = "known_count_upper_bound_16"
376             break
377         if self.strategy == "baseline":
378             if self.tracks:
379                 self.localize(next(iter(self.tracks)))
380             elif self.pending:
381                 self.scan_anchor(self.next_anchor())
382             continue
383         targets = [
384             (dist(self.position, enclosing_circle(t.polygon)[0]), ch)
385             for ch, t in self.tracks.items()
386         ]
387         anchors = [

```

```

388         (
389             dist(self.position, self.waypoints[i])
390             + 30.0 * (20 - len(self.cleared)),
391             i,
392         )
393     for i in ([self.next_anchor()] if self.pending and self.anchor_policy == "rolling" else self.
pending)
394 ]
395     # Convert a full scan to equivalent travel metres (6 s * 5 m/s).
396     # This is a myopic time heuristic, not a global-optimality claim.
397     if targets and (not anchors or min(targets)[0] <= min(anchors)[0]):
398         self.localize(min(targets)[1])
399         self.share_current_stop()
400     else:
401         self.scan_anchor(self.next_anchor())
402     if self.termination is None:
403         if self.terminal_reason is None:
404             for channel in range(1, 21):
405                 if channel in self.cleared:
406                     self.channel_state[channel] = "cleared"
407                 elif (self.channel_state[channel] != "detected"
408                     and len(self.scan_records[channel]) == len(self.waypoints)):
409                     self.channel_state[channel] = "absent-certified"
410                 else:
411                     raise RuntimeError("Incomplete per-channel discovery coverage")
412             self.termination = "all_channels_cleared_or_coverage_certified_absent"
413         else:
414             current = self._result()
415             if current["certificate_complete"]:
416                 self.termination = "complete_after_" + self.terminal_reason
417             else:
418                 self.termination = "incomplete_" + self.terminal_reason
419         if self.terminal_reason is None:
420             self.action("/exit")
421     return self._result()

```

Listing 2: 实验主程序

```

1  """Reproducible synthetic comparison; these are NOT official simulator scores."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  import math
8  import statistics
9  from pathlib import Path
10
11  from environment import LocalSimulator
12  from planner import Planner
13
14
15  def run_case(
16      seed,
17      problem,
18      strategy,
19      scenario="random",
20      error_mode="smooth",
21      probe_scale=0.22,
22      spacing=990.0,
23      loss_recovery=True,
24      anchor_policy=None,
25      ring_radius=None,
26      probe_policy="fixed",
27  ):
28     env = LocalSimulator(seed, problem, error_mode, scenario)
29     planner = Planner(

```

```

30     env,
31     problem,
32     strategy,
33     spacing=spacing,
34     probe_scale=probe_scale,
35     loss_recovery=loss_recovery,
36     anchor_policy=anchor_policy,
37     ring_radius=ring_radius,
38     probe_policy=probe_policy,
39 )
40 p = planner.run()
41 assert p["certificate_complete"] is True
42 s = env.summary()
43 # Independent action-by-action accounting, without reading hidden world data.
44 position, radio, total = (0.0, 0.0), 1, 0.0
45 successes = set()
46 for event in env.log:
47     request, response = event["request"], event["response"]
48     if not response["accepted"]:
49         continue
50     path = request["path"]
51     if path in ("/measure", "/clear"):
52         q = (request["position"]["x"], request["position"]["y"])
53         total += round(math.dist(position, q) / 5, 6)
54         position = q
55         if path == "/measure":
56             total += 5 + int(radio != request["channel"])
57             radio = request["channel"]
58         else:
59             success = response["clear_result"] == "success"
60             total += 5 if success else 3
61             if success:
62                 assert request["channel"] not in successes
63                 successes.add(request["channel"])
64             assert abs(total - response["virtual_time_s"]) < 2e-6
65 assert len(successes) == s["n_cleared"] == p["n_cleared"]
66 assert s["n_sources"] == s["n_cleared"], (seed, problem, scenario, error_mode, s)
67 return {
68     **s,
69     "strategy": strategy,
70     "visited_anchors": p["visited_anchors"],
71     "optical_fallbacks": p["optical_fallbacks"],
72     "probe_scale": probe_scale,
73     "loss_recovery_requested": loss_recovery,
74     "loss_recovery": p["loss_recovery"],
75     "anchor_policy": p["anchor_policy"],
76     "ring_radius_m": p["ring_radius_m"],
77     "spacing_m": spacing,
78     "certificate_complete": p["certificate_complete"],
79     "timing_independently_verified": True,
80 }
81
82
83 def aggregate(rows):
84     values = sorted(row["average_time_s"] for row in rows)
85     return {
86         "cases": len(rows),
87         "all_cleared_cases": sum(r["fraction_cleared"] == 1 for r in rows),
88         "mean_average_time_s": statistics.mean(values),
89         "median_average_time_s": statistics.median(values),
90         "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1],
91         "worst_average_time_s": values[-1],
92         "mean_total_time_s": statistics.mean(r["virtual_time_s"] for r in rows),
93         "mean_distance_m": statistics.mean(r["distance_m"] for r in rows),
94         "mean_measures": statistics.mean(r["measures"] for r in rows),
95         "mean_failed_clears": statistics.mean(r["failed_clears"] for r in rows),
96         "max_real_duration_s": max(r["real_duration_s"] for r in rows),

```

```

97     }
98
99
100 def main():
101     parser = argparse.ArgumentParser()
102     parser.add_argument("--cases", type=int, default=200)
103     parser.add_argument("--output", type=Path, default=Path("results/experiments.json"))
104     args = parser.parse_args()
105     if args.cases < 2:
106         parser.error("At least two paired cases required")
107     rows, grouped, paired, stress, sensitivity = [], {}, {}, {}, {}
108     for problem in (3, 4):
109         strategy_rows = {}
110         for strategy in ("baseline", "batched", "integrated"):
111             current = [
112                 run_case(600000 + k, problem, strategy) for k in range(args.cases)
113             ]
114             rows.extend(current)
115             strategy_rows[strategy] = current
116             grouped[f"Q{problem}_{strategy}"] = aggregate(current)
117         differences = [
118             a["average_time_s"] - b["average_time_s"]
119             for a, b in zip(strategy_rows["baseline"], strategy_rows["integrated"])
120         ]
121         delta = statistics.mean(differences)
122         standard_error = statistics.stdev(differences) / math.sqrt(len(differences))
123         paired[f"Q{problem}"] = {
124             "mean_saved_s_per_source": delta,
125             "approx_95pct_ci_s": [
126                 delta - 1.96 * standard_error,
127                 delta + 1.96 * standard_error,
128             ],
129             "mean_time_reduction_pct": 100
130             * delta
131             / grouped[f"Q{problem}_baseline"]["mean_average_time_s"],
132             "faster_cases": sum(d > 0 for d in differences),
133             "paired_cases": len(differences),
134         }
135         share_differences = [
136             a["average_time_s"] - b["average_time_s"]
137             for a, b in zip(strategy_rows["batched"], strategy_rows["integrated"])
138         ]
139         paired[f"Q{problem}_sharing_ablation"] = {
140             "mean_saved_s_per_source": statistics.mean(share_differences),
141             "faster_cases": sum(d > 0 for d in share_differences),
142             "paired_cases": len(share_differences),
143         }
144     for scenario in ("minimum_radius", "outward_boundary", "clustered"):
145         for mode in ("smooth", "extreme", "iid_location"):
146             current = [
147                 run_case(700000 + k, problem, "integrated", scenario, mode)
148                 for k in range(20)
149             ]
150             rows.extend(current)
151             stress[f"Q{problem}_{scenario}_{mode}"] = aggregate(current)
152     # Sensitivity uses different seeds, no held-out retuning afterwards.
153     for scale in (0.12, 0.22, 0.40):
154         current = [
155             run_case(800000 + k, problem, "integrated", probe_scale=scale)
156             for k in range(30)
157         ]
158         rows.extend(current)
159         sensitivity[f"Q{problem}_probe_{scale}"] = aggregate(current)
160     for spacing in (900.0, 950.0, 990.0):
161         current = [
162             run_case(900000 + k, 4, "integrated", spacing=spacing) for k in range(30)
163         ]

```

```

164         rows.extend(current)
165         sensitivity[f"Q4_spacing_{spacing}"] = aggregate(current)
166     recovery_ablation = {}
167     for scenario in ("random", "minimum_radius", "outward_boundary", "clustered"):
168         for enabled in (False, True):
169             current = [
170                 run_case(610000 + k, 4, "integrated", scenario, loss_recovery=enabled)
171                 for k in range(100)
172             ]
173             rows.extend(current)
174             recovery_ablation[f"{scenario}_recovery_{enabled}"] = aggregate(current)
175     result = {
176         "evidence": "local_synthetic_not_official",
177         "assumptions": "See environment.py module docstring",
178         "paired_random_cases_per_problem": args.cases,
179         "total_runs": len(rows),
180         "summary": grouped,
181         "paired_comparison": paired,
182         "stress": stress,
183         "sensitivity": sensitivity,
184         "recovery_ablation": recovery_ablation,
185         "runs": rows,
186     }
187     args.output.parent.mkdir(parents=True, exist_ok=True)
188     args.output.write_text(
189         json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
190     )
191     print(
192         json.dumps(
193             {k: v for k, v in result.items() if k != "runs"},
194             ensure_ascii=False,
195             indent=2,
196         )
197     )
198
199
200 if __name__ == "__main__":
201     main()

```

Listing 3: 统一运行入口

```

1  """Run local synthetic practice; official GUI sessions require client.HTTPClient separately."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  from pathlib import Path
8
9  from environment import LocalSimulator
10 from planner import Planner
11
12
13 def run_local(
14     seed: int,
15     problem: int,
16     scenario: str = "random",
17     error_mode: str = "smooth",
18     strategy: str = "enhanced",
19 ) -> dict:
20     env = LocalSimulator(
21         seed, problem=problem, scenario=scenario, error_mode=error_mode
22     )
23     planner = Planner(env, problem=problem, strategy=strategy)
24     planner_result = planner.run()
25     result = {"planner": planner_result, "simulator": env.summary()}
26     return result

```

```

27
28
29 def main() -> None:
30     parser = argparse.ArgumentParser()
31     parser.add_argument("--seed", type=int, default=20260910)
32     parser.add_argument("--problem", type=int, choices=(3, 4), default=3)
33     parser.add_argument("--scenario", default="random")
34     parser.add_argument("--error-mode", default="smooth")
35     parser.add_argument(
36         "--strategy",
37         choices=("baseline", "batched", "integrated", "enhanced"),
38         default="enhanced",
39     )
40     parser.add_argument("--output", type=Path)
41     args = parser.parse_args()
42     result = run_local(
43         args.seed, args.problem, args.scenario, args.error_mode, args.strategy
44     )
45     text = json.dumps(result, ensure_ascii=False, indent=2)
46     if args.output:
47         args.output.parent.mkdir(parents=True, exist_ok=True)
48         args.output.write_text(text + "\n", encoding="utf-8")
49     print(text)
50
51
52 if __name__ == "__main__":
53     main()

```

Listing 4: 几何计算

```

1  """Conservative bearing geometry and whole-region optical clearing certificates.
2
3  Polygons are ordered convex vertex lists (either winding); [] means infeasible,
4  never unbounded. All distances are metres and bearings are counterclockwise degrees.
5  """
6
7  from __future__ import annotations
8
9  from decimal import Decimal, localcontext
10 from math import ceil, cos, hypot, isfinite, nextafter, pi, radians, sin, sqrt, ulp
11 from random import Random
12
13 Point = tuple[float, float]
14 Circle = tuple[Point, float]
15 ARENA_RADIUS = 1800.0
16 MAX_RADIO_RADIUS = 1500.0
17 BEARING_ERROR_DEG = 1.005
18
19
20 def _check_points(points: list[Point]) -> None:
21     if any(not (isfinite(x) and isfinite(y)) for x, y in points):
22         raise ValueError("Geometry requires finite coordinates")
23
24
25 def _padding(scale: float) -> float:
26     return max(1e-9, 64.0 * ulp(max(1.0, scale)))
27
28
29 def initial_polygon(sides: int = 180) -> list[Point]:
30     """Return a CCW circumscribed polygon, never an inscribed approximation."""
31     if isinstance(sides, bool) or not isinstance(sides, int) or sides < 3:
32         raise ValueError("sides must be an integer at least three")
33     radius = (ARENA_RADIUS + _padding(ARENA_RADIUS)) / cos(pi / sides)
34     return [
35         (radius * cos(2 * pi * i / sides), radius * sin(2 * pi * i / sides))
36         for i in range(sides)
37     ]

```

```

38
39
40 def _clip_halfplane(
41     poly: list[Point], origin: Point, normal: Point, bound: float, padding: float
42 ) -> list[Point]:
43     """Clip n.(x-origin) <= bound+padding using the same shifted boundary."""
44     if not poly:
45         return []
46     nx, ny = normal
47     ox, oy = origin
48     limit = bound + padding
49     result: list[Point] = []
50     previous = poly[-1]
51     dp = nx * (previous[0] - ox) + ny * (previous[1] - oy) - limit
52     for current in poly:
53         dc = nx * (current[0] - ox) + ny * (current[1] - oy) - limit
54         if (dp <= 0.0) != (dc <= 0.0):
55             # Opposite signs prevent cancellation in dp-dc. Interpolate from
56             # the nearer endpoint to reduce cancellation near a polygon vertex.
57             if abs(dp) <= abs(dc):
58                 t = dp / (dp - dc)
59                 crossing = (
60                     previous[0] + t * (current[0] - previous[0]),
61                     previous[1] + t * (current[1] - previous[1]),
62                 )
63             else:
64                 t = dc / (dc - dp)
65                 crossing = (
66                     current[0] + t * (previous[0] - current[0]),
67                     current[1] + t * (previous[1] - current[1]),
68                 )
69             if not result or crossing != result[-1]:
70                 result.append(crossing)
71             if dc <= 0.0 and (not result or current != result[-1]):
72                 result.append(current)
73             previous, dp = current, dc
74         if len(result) > 1 and result[0] == result[-1]:
75             result.pop()
76     return result
77
78
79 def clip_bearing(
80     poly: list[Point],
81     point: Point,
82     bearing_deg: float,
83     error_deg: float = BEARING_ERROR_DEG,
84 ) -> list[Point]:
85     """Intersect a bounded region with a forward wedge and a safe range cap.
86
87     The default includes the 0.005-degree two-decimal API rounding allowance.
88     The cap is a relaxation of distance <= 1500, not an assumed known range.
89     Contradictions remain []; no fallback region or constraint reset is applied.
90     """
91     _check_points(poly)
92     _check_points([point])
93     if not isfinite(bearing_deg) or not isfinite(error_deg) or not 0 <= error_deg < 90:
94         raise ValueError("Use finite bearing and 0 <= error_deg < 90")
95     if not poly:
96         return []
97     theta = radians(bearing_deg % 360.0)
98     error = radians(error_deg)
99     low, high = theta - error, theta + error
100     scale = max(
101         MAX_RADIO_RADIUS,
102         abs(point[0]),
103         abs(point[1]),
104         *(max(abs(x), abs(y)) for x, y in poly),

```

```

105     )
106     padding = _padding(scale)
107     # cross(u_low, x-point)>=0 and cross(u_high, x-point)<=0.
108     result = _clip_halfplane(poly, point, (sin(low), -cos(low)), 0.0, padding)
109     result = _clip_halfplane(result, point, (-sin(high), cos(high)), 0.0, padding)
110     result = _clip_halfplane(result, point, (-cos(theta), -sin(theta)), 0.0, padding)
111     return _clip_halfplane(
112         result, point, (cos(theta), sin(theta)), MAX_RADIO_RADIUS, padding
113     )
114
115
116 def _farthest_pair(poly: list[Point]) -> tuple[Point, Point, float]:
117     first = second = poly[0]
118     distance = 0.0
119     for i, a in enumerate(poly):
120         for j in range(i):
121             b = poly[j]
122             candidate = hypot(a[0] - b[0], a[1] - b[1])
123             if candidate > distance:
124                 first, second, distance = a, b, candidate
125     return first, second, distance
126
127
128 def diameter(poly: list[Point]) -> float:
129     """Exact vertex-pair characterization,  $O(n^2)$ ; empty input is an error."""
130     _check_points(poly)
131     if not poly:
132         raise ValueError("An infeasible region has no localization diameter")
133     return _farthest_pair(poly)[2]
134
135
136 def _pair_circle(a: Point, b: Point) -> Circle:
137     center = (a[0] + (b[0] - a[0]) / 2, a[1] + (b[1] - a[1]) / 2)
138     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b))
139
140
141 def _cross(a: Point, b: Point, c: Point) -> float:
142     bx, by = b[0] - a[0], b[1] - a[1]
143     cx, cy = c[0] - a[0], c[1] - a[1]
144     determinant = bx * cy - by * cx
145     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
146     if abs(determinant) > 64 * ulp(max(1.0, scale * scale)):
147         return determinant
148     with localcontext() as context:
149         context.prec = 80
150         ax, ay = Decimal(a[0]), Decimal(a[1])
151         return float(
152             (Decimal(b[0]) - ax) * (Decimal(c[1]) - ay)
153             - (Decimal(b[1]) - ay) * (Decimal(c[0]) - ax)
154         )
155
156
157 def _circumcircle(a: Point, b: Point, c: Point) -> Circle | None:
158     bx, by = b[0] - a[0], b[1] - a[1]
159     cx, cy = c[0] - a[0], c[1] - a[1]
160     determinant = bx * cy - by * cx
161     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
162     if abs(determinant) <= 64 * ulp(max(1.0, scale * scale)):
163         # Decimal fallback resolves almost-collinear triples from the actual
164         # binary input values, without pretending a small determinant is zero.
165         with localcontext() as context:
166             context.prec = 80
167             axd, ayd = Decimal(a[0]), Decimal(a[1])
168             bxd, byd = Decimal(b[0]) - axd, Decimal(b[1]) - ayd
169             cxd, cyd = Decimal(c[0]) - axd, Decimal(c[1]) - ayd
170             det = 2 * (bxd * cyd - byd * cxd)
171             if not det:

```

```

172         return None
173         bb, cc = bxd * bxd + byd * byd, cxd * cxd + cyd * cyd
174         center = (
175             float(axd + (cyd * bb - byd * cc) / det),
176             float(ayd + (bxd * cc - cxd * bb) / det),
177         )
178     else:
179         bb, cc = bx * bx + by * by, cx * cx + cy * cy
180         center = (
181             a[0] + (cy * bb - by * cc) / (2 * determinant),
182             a[1] + (bx * cc - cx * bb) / (2 * determinant),
183         )
184     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b, c))
185
186
187 def _contains(circle: Circle, point: Point) -> bool:
188     center, radius = circle
189     return hypot(point[0] - center[0], point[1] - center[1]) <= nextafter(
190         radius, float("inf")
191     )
192
193
194 def _two_boundary_circle(points: list[Point], end: int, a: Point, b: Point) -> Circle:
195     base = _pair_circle(a, b)
196     left: Circle | None = None
197     right: Circle | None = None
198     left_cross = right_cross = 0.0
199     for i in range(end):
200         p = points[i]
201         if _contains(base, p):
202             continue
203         side = _cross(a, b, p)
204         candidate = _circumcircle(a, b, p)
205         if candidate is None:
206             continue
207         center_side = _cross(a, b, candidate[0])
208         if side > 0 and (left is None or center_side > left_cross):
209             left, left_cross = candidate, center_side
210         elif side < 0 and (right is None or center_side < right_cross):
211             right, right_cross = candidate, center_side
212     if left is None:
213         return base if right is None else right
214     if right is None:
215         return left
216     return left if left[1] <= right[1] else right
217
218
219 def enclosing_circle(poly: list[Point]) -> Circle:
220     """Minimum enclosing circle, with a tiny outward floating-point guard.
221
222     Randomized incremental 1/2/3-support construction, not a diameter-disk or
223     approximate iterative fit. The local fixed shuffle never alters global RNG.
224     Empty input raises instead of manufacturing a spurious clearing location.
225     """
226     _check_points(poly)
227     if not poly:
228         raise ValueError("An infeasible region has no enclosing-circle center")
229     points = list(dict.fromkeys(poly))
230     Random(0).shuffle(points)
231     circle: Circle | None = None
232     for i, p in enumerate(points):
233         if circle is not None and _contains(circle, p):
234             continue
235         circle = (p, 0.0)
236         for j in range(i):
237             q = points[j]
238             if _contains(circle, q):

```

```

239         continue
240     if circle[1] == 0.0:
241         circle = _pair_circle(p, q)
242     else:
243         circle = _two_boundary_circle(points, j + 1, p, q)
244     assert circle is not None
245     center = circle[0]
246     radius = max(hypot(p[0] - center[0], p[1] - center[1]) for p in points)
247     if radius == 0.0:
248         return center, 0.0
249     scale = max(radius, abs(center[0]), abs(center[1]))
250     return center, nextafter(radius + _padding(scale), float("inf"))
251
252
253 def optical_cover(poly: list[Point], radius: float = 19.9) -> list[Point]:
254     """Cover the ENTIRE convex polygon, not merely its vertices or boundary.
255
256     Use its MEC when possible; otherwise tile an enclosing rectangle aligned
257     with a diameter. Every grid cell fits in its own radius-radius disk. Some
258     centers may lie outside the polygon or arena, which the API permits.
259     [] stays []: callers must treat it as infeasibility, not successful clearing.
260     """
261     _check_points(poly)
262     if not isfinite(radius) or radius <= 0:
263         raise ValueError("Optical covering radius must be finite and positive")
264     if not poly:
265         return []
266     center, enclosing_radius = enclosing_circle(poly)
267     if enclosing_radius <= radius:
268         return [center]
269     a, b, distance = _farthest_pair(poly)
270     if distance == 0.0:
271         return [a]
272     ux, uy = (b[0] - a[0]) / distance, (b[1] - a[1]) / distance
273     # Re-normalize before forming the orthogonal frame.
274     norm = hypot(ux, uy)
275     ux, uy = ux / norm, uy / norm
276     projected = [
277         ((x - a[0]) * ux + (y - a[1]) * uy, -(x - a[0]) * uy + (y - a[1]) * ux)
278         for x, y in poly
279     ]
280     scale = max(distance, *(max(abs(x), abs(y)) for x, y in poly))
281     guard = _padding(scale)
282     safe_radius = radius - 8 * guard
283     if safe_radius <= 0:
284         raise ValueError(
285             "Radius is below floating-point resolution at these coordinates"
286         )
287     xmin, xmax = (
288         min(p[0] for p in projected) - guard,
289         max(p[0] for p in projected) + guard,
290     )
291     ymin, ymax = (
292         min(p[1] for p in projected) - guard,
293         max(p[1] for p in projected) + guard,
294     )
295     side = sqrt(2.0) * safe_radius
296     nx = max(1, ceil((xmax - xmin) / side))
297     ny = max(1, ceil((ymax - ymin) / side))
298     dx, dy = (xmax - xmin) / nx, (ymax - ymin) / ny
299     centers: list[Point] = []
300     for j in range(ny):
301         y = ymin + (j + 0.5) * dy
302         indices = range(nx) if j % 2 == 0 else range(nx - 1, -1, -1)
303         for i in indices:
304             x = xmin + (i + 0.5) * dx
305             centers.append((a[0] + x * ux - y * uy, a[1] + x * uy + y * ux))

```

Listing 5: 几何验证

```

1  """Independent numerical geometry checks; no official simulator calls."""
2
3  from __future__ import annotations
4
5  import itertools
6  import json
7  import math
8  import random
9  from pathlib import Path
10
11 import numpy as np
12 from scipy.optimize import linprog
13
14 from coverage import coverage_certificate, directional_waypoints, omni_waypoints
15 from geometry import (
16     clip_bearing,
17     diameter,
18     enclosing_circle,
19     initial_polygon,
20     optical_cover,
21 )
22
23
24 def contains(poly, p, tolerance=1e-6):
25     if not poly:
26         return False
27     area = sum(a[0] * b[1] - a[1] * b[0] for a, b in zip(poly, poly[1:] + poly[:1]))
28     sign = 1 if area >= 0 else -1
29     return all(
30         sign * ((b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0]))
31         >= -tolerance
32         for a, b in zip(poly, poly[1:] + poly[:1])
33     )
34
35
36 def reference_circle(points):
37     """Exhaust every 1/2/3-point support, independently of incremental MEC."""
38     candidates = [(p, 0.0) for p in points]
39     for a, b in itertools.combinations(points, 2):
40         c = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
41         candidates.append((c, math.dist(a, b) / 2))
42     for a, b, c in itertools.combinations(points, 3):
43         ab, ac = np.asarray(b) - a, np.asarray(c) - a
44         matrix = 2 * np.array([ab, ac])
45         if abs(np.linalg.det(matrix)) < 1e-12:
46             continue
47         offset = np.linalg.solve(matrix, np.array([ab @ ab, ac @ ac]))
48         center = np.asarray(a) + offset
49         candidates.append((tuple(center), math.dist(center, a)))
50     feasible = [
51         (c, r)
52         for c, r in candidates
53         if all(math.dist(c, p) <= r + 1e-7 for p in points)
54     ]
55     return min(feasible, key=lambda item: item[1])
56
57
58 def main():
59     exact_ray = clip_bearing(
60         [(-100.0, 0.0), (100.0, 0.0)], (0.0, 0.0), 0.0, error_deg=0.0
61     )
62     assert min(p[0] for p in exact_ray) >= -1e-6, (
63         "Zero-error bearing retained its backward ray"
64     )

```

```

64 )
65 try:
66     directional_waypoints(1e-300)
67 except ValueError:
68     pass
69 else:
70     raise AssertionError("Extreme lattice refinement was not rejected")
71 rng = random.Random(831762)
72 max_mec_error = 0.0
73 sets = [
74     [(0.0, 0.0)],
75     [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))],
76     [(0.0, 0.0), (1.0, 1e-12), (2.0, 0.0)],
77     [(7.0, 5.0), (7.0, 5.0)],
78 ]
79 sets += [
80     [
81         (rng.uniform(-2000, 2000), rng.uniform(-2000, 2000))
82         for _ in range(rng.randint(2, 9))
83     ]
84     for _ in range(500)
85 ]
86 for points in sets:
87     actual = enclosing_circle(points)
88     expected = reference_circle(points)
89     max_mec_error = max(max_mec_error, abs(actual[1] - expected[1]))
90     assert abs(actual[1] - expected[1]) < 1e-6, (points, actual, expected)
91     assert all(math.dist(actual[0], p) <= actual[1] for p in points)
92 triangle = [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))]
93 polygon = initial_polygon()
94 length = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
95 stations = []
96 for i, (a, b) in enumerate(zip(triangle, triangle[1:] + triangle[:1])):
97     unit = ((b[0] - a[0]) / 36, (b[1] - a[1]) / 36)
98     s = (a[0] - length * unit[0], a[1] - length * unit[1])
99     stations.append(s)
100    polygon = clip_bearing(polygon, s, 1 + 120 * i, error_deg=1.0)
101 counterexample = {
102     "stations": stations,
103     "bearings_deg": [1, 121, 241],
104     "diameter_m": diameter(polygon),
105     "mec_radius_m": enclosing_circle(polygon)[1],
106 }
107 assert abs(counterexample["diameter_m"] - 36) < 1e-6
108 assert counterexample["mec_radius_m"] > 20
109 # True-source containment including boundary source, wraparound and +/-1 noise.
110 containment_trials = 2000
111 optical_trials = 0
112 for index in range(containment_trials):
113     angle = rng.uniform(0, math.tau)
114     radius = 1800 if index % 5 == 0 else 1800 * math.sqrt(rng.random())
115     source = (radius * math.cos(angle), radius * math.sin(angle))
116     poly = initial_polygon()
117     for j in range(4):
118         r, phi = rng.uniform(6, 1500), rng.uniform(0, math.tau)
119         station = (source[0] + r * math.cos(phi), source[1] + r * math.sin(phi))
120         error = rng.choice([-1.0, 1.0]) if index % 2 else rng.uniform(-1, 1)
121         bearing = round((math.degrees(phi) + 180 + error) % 360, 2) % 360
122         poly = clip_bearing(poly, station, bearing)
123         assert contains(poly, source), (index, source, poly)
124     if index < 200:
125         centers = optical_cover(poly)
126         for _ in range(20):
127             weights = [rng.random() for _ in poly]
128             total = sum(weights)
129             p = tuple(
130                 sum(w * vertex[k] for w, vertex in zip(weights, poly)) / total

```

```

131         for k in (0, 1)
132     )
133     assert min(math.dist(p, c) for c in centers) <= 19.9 + 1e-7
134     optical_trials += 1
135 omni, directional = omni_waypoints(), directional_waypoints()
136 max_omni_distance, min_directional_slack = 0.0, float("inf")
137 for i in range(2000):
138     angle = rng.uniform(0, math.tau)
139     radius = 1800 if i % 2 else 1800 * math.sqrt(rng.random())
140     p = (radius * math.cos(angle), radius * math.sin(angle))
141     max_omni_distance = max(max_omni_distance, min(math.dist(p, s) for s in omni))
142     nearby = [s for s in directional if math.dist(p, s) <= 990 + 1e-7]
143     # Independent convex-hull feasibility, all orientations at once for p.
144     points = np.asarray(nearby)
145     solved = linprog(
146         np.zeros(len(points)),
147         A_eq=np.vstack([points.T, np.ones(len(points))]),
148         b_eq=np.array([*p, 1.0]),
149         bounds=(0, None),
150         method="highs",
151     )
152     assert solved.success, p
153     for _ in range(5):
154         phi = rng.uniform(0, math.tau)
155         seen = [
156             math.dist(p, s)
157             for s in directional
158             if (s[0] - p[0]) * math.cos(phi) + (s[1] - p[1]) * math.sin(phi) >= 0
159         ]
160         min_directional_slack = min(min_directional_slack, 1000 - min(seen))
161     assert max_omni_distance <= 900 + 1e-7 and min_directional_slack >= 0
162     # Exact triple-lens safe region sampled as a separate numerical sanity check.
163     alpha = math.radians(1.005)
164     max_reception_excess = -float("inf")
165     for _ in range(10000):
166         t = rng.uniform(1, 999)
167         bmax = min(
168             math.sqrt(1e6 - t * t),
169             math.sqrt(1e6 - (t - 1000 * math.cos(alpha)) ** 2) - 1000 * math.sin(alpha),
170         )
171         b = rng.uniform(-1, 1) * max(0.0, bmax)
172         r, phi = rng.uniform(0, 1500), rng.uniform(-alpha, alpha)
173         excess = math.dist((t, b), (r * math.cos(phi), r * math.sin(phi))) - max(
174             1000, r
175         )
176         max_reception_excess = max(max_reception_excess, excess)
177     assert excess <= 1e-6
178     result = {
179         "evidence": "independent_numerical_checks_not_a_substitute_for_analytic_proofs",
180         "exact_forward_ray_regression": True,
181         "unbounded_refinement_rejected": True,
182         "mec_reference_cases": len(sets),
183         "maximum_mec_radius_error_m": max_mec_error,
184         "realizable_counterexample": counterexample,
185         "containment_cases": containment_trials,
186         "bearings_per_case": 4,
187         "optical_interior_samples": optical_trials,
188         "coverage_source_samples": 2000,
189         "maximum_sampled_omni_distance_m": max_omni_distance,
190         "minimum_sampled_directional_radio_slack_m": min_directional_slack,
191         "triple_lens_samples": 10000,
192         "maximum_sampled_reception_excess_m": max_reception_excess,
193         "coverage_certificate": coverage_certificate(),
194     }
195     Path("results").mkdir(exist_ok=True)
196     Path("results/geometry_verification.json").write_text(
197         json.dumps(result, ensure_ascii=False, indent=2) + "\n"

```

```

198     )
199     # The default enhanced policy uses the 1200m Q3 ring and rolling Q4
200     # skeleton. The fixed-probe/local-recovery bound is deliberately separate
201     # from the optional adaptive candidate. Reflection is about the last
202     # positive station and contracts its station distance by one half.
203     station_probe_bound = math.hypot(1501, 120)
204     source_probe_bound = 1500 + station_probe_bound
205     assert station_probe_bound < 1506 and source_probe_bound < 3006
206     max_move = 31 * 5240 + 16 * (7500 + 5 * 6020 + 4520 + 4520 + 7570)
207     max_time = max_move / 5 + 1212 * 6 + 3584 * 5
208     assert max_time < 100 * 3600 <= 360000
209     bound = {
210         "evidence": "arithmetic_of_analytic_bound_not_simulation",
211         "scope": "enhanced policy: Q3 ring1200, Q4 rolling skeleton, fixed probes, six probes",
212         "skeleton_edge_upper_m": 5240,
213         "skeleton_move_upper_m": 31 * 5240,
214         "positive_source_distance_upper_m": 1500,
215         "strip_axial_upper_m": 1501,
216         "strip_width_upper_m": 53,
217         "station_probe_bound_m": station_probe_bound,
218         "source_probe_bound_m": source_probe_bound,
219         "local_transition_upper_m": 6020,
220         "local_task_entry_upper_m": 7500,
221         "optional_center_clear_access_upper_m": 4520,
222         "optical_access_upper_m": 4520,
223         "optical_cover_centers_upper": 216,
224         "optical_snake_length_upper_m": 7570,
225         "distance_upper_m": max_move,
226         "measure_upper": 1212,
227         "clear_attempt_upper": 3584,
228         "virtual_time_upper_s": max_time,
229         "virtual_time_upper_h": max_time / 3600,
230         "proof": "report/bound_proof.tex; no guarantee for real network deadline",
231     }
232     Path("results/operation_bound.json").write_text(json.dumps(bound, indent=2) + "\n")
233     print(json.dumps(result, ensure_ascii=False, indent=2))
234
235
236 if __name__ == "__main__":
237     main()

```

Listing 6: 问题一原始交会区域分类与直径

```

1  """Q1: actual bearing-halfplane intersection, without an artificial bounding box.
2
3  Feasibility and four coordinate LPs distinguish empty/unbounded/bounded sets.
4  For a bounded set all feasible pairwise boundary intersections are enumerated.
5  This optional Q1 analysis uses SciPy; the online search policy uses stdlib only.
6  """
7
8  from __future__ import annotations
9
10 import itertools
11 import json
12 import math
13 from pathlib import Path
14
15 import numpy as np
16 from scipy.optimize import linprog
17
18 from geometry import diameter, enclosing_circle
19
20
21 def solve_intersection(stations, bearings_deg, error_deg=1.0):
22     if len(stations) != len(bearings_deg) or not 0 <= error_deg < 90:
23         raise ValueError("Match stations/readings and use an error in [0,90) degrees")
24     if not stations:

```

```

25     return {"status": "unbounded", "vertices": [], "diameter_m": None}
26 rows, rhs = [], []
27 alpha = math.radians(error_deg)
28 for (x, y), angle in zip(stations, bearings_deg):
29     if not all(math.isfinite(value) for value in (x, y, angle)):
30         raise ValueError("Finite coordinates and bearings are required")
31     theta = math.radians(angle % 360)
32     for normal in (
33         (math.sin(theta - alpha), -math.cos(theta - alpha)),
34         (-math.sin(theta + alpha), math.cos(theta + alpha)),
35         (-math.cos(theta), -math.sin(theta)),
36     ):
37         rows.append(normal)
38         rhs.append(normal[0] * x + normal[1] * y)
39 a, b = np.asarray(rows), np.asarray(rhs)
40 feasible = linprog(
41     np.zeros(2), A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
42 )
43 if feasible.status == 2:
44     return {"status": "infeasible", "vertices": [], "diameter_m": None}
45 if not feasible.success:
46     raise RuntimeError(f"Halfplane feasibility LP failed: {feasible.message}")
47 for objective in ((1.0, 0.0), (-1.0, 0.0), (0.0, 1.0), (0.0, -1.0)):
48     bound = linprog(
49         objective, A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
50     )
51     if bound.status == 3:
52         return {"status": "unbounded", "vertices": [], "diameter_m": None}
53     if not bound.success:
54         raise RuntimeError(f"Halfplane boundedness LP failed: {bound.message}")
55 vertices = []
56 for i, j in itertools.combinations(range(len(a)), 2):
57     determinant = a[i, 0] * a[j, 1] - a[i, 1] * a[j, 0]
58     if determinant == 0:
59         continue
60     point = (
61         (b[i] * a[j, 1] - a[i, 1] * b[j]) / determinant,
62         (a[i, 0] * b[j] - b[i] * a[j, 0]) / determinant,
63     )
64     if np.max(a @ point - b) <= 1e-7 and all(
65         math.dist(point, old) > 1e-7 for old in vertices
66     ):
67         vertices.append(point)
68 if not vertices:
69     raise ArithmeticError(
70         "Bounded nonempty intersection had no numerical vertex; inspect conditioning"
71     )
72 center = tuple(np.mean(vertices, axis=0))
73 vertices.sort(
74     key=lambda point: math.atan2(point[1] - center[1], point[0] - center[0])
75 )
76 circle = enclosing_circle(vertices)
77 return {
78     "status": "bounded",
79     "vertices": vertices,
80     "diameter_m": diameter(vertices),
81     "mec_center": circle[0],
82     "mec_radius_m": circle[1],
83     "max_halfplane_residual_m": max(float(np.max(a @ p - b)) for p in vertices),
84 }
85
86
87 def main():
88     triangle = [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))]
89     length = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
90     stations = []
91     for first, second in zip(triangle, triangle[1:] + triangle[:1]):

```

```

92     stations.append(
93         (
94             first[0] - length * (second[0] - first[0]) / 36,
95             first[1] - length * (second[1] - first[1]) / 36,
96         )
97     )
98     bounded = solve_intersection(stations, [1, 121, 241])
99     unbounded = solve_intersection([(0.0, 0.0)], [0.0])
100     infeasible = solve_intersection([(0.0, 0.0), (-10.0, 0.0)], [0.0, 180.0])
101     assert bounded["status"] == "bounded" and abs(bounded["diameter_m"] - 36) < 1e-6
102     assert unbounded["status"] == "unbounded" and infeasible["status"] == "infeasible"
103     result = {
104         "evidence": "Q1_direct_halfplane_algorithm_examples",
105         "bounded": bounded,
106         "unbounded": unbounded,
107         "infeasible": infeasible,
108     }
109     Path("results").mkdir(exist_ok=True)
110     Path("results/intersection_verification.json").write_text(
111         json.dumps(result, indent=2) + "\n"
112     )
113     print(json.dumps(result, indent=2))
114
115
116 if __name__ == "__main__":
117     main()

```

Listing 7: 覆盖与采样工具

```

1  """Finite, analytically justified discovery waypoint sets for CUMCM B Q3/Q4.
2
3  These functions prescribe measurement locations, not a route or a stopping rule.
4  Every unresolved channel needs its own completed coverage record. In particular,
5  Q4 waypoints outside the source arena must not be projected back into the arena.
6  """
7
8  from __future__ import annotations
9
10 import math
11
12 Point = tuple[float, float]
13
14 ARENA_RADIUS = 1800.0
15 MIN_RADIO_RADIUS = 1000.0
16 OMNI_COVER_RADIUS = 900.0
17 DEFAULT_SPACING = 990.0
18
19
20 def omni_waypoints(ring_radius: float | None = None) -> list[Point]:
21     """Return origin plus six ring points; the legacy radius certifies 900 m.
22
23     The 1200-m contracted ring instead has exact cover radius given by
24     ring_cover_radius, still below the guaranteed 1000-m reception distance.
25     No unspecified source-position prior is used by this geometric guarantee.
26     """
27     ring_radius = OMNI_COVER_RADIUS * math.sqrt(3.0) if ring_radius is None else float(ring_radius)
28     if not math.isfinite(ring_radius) or not 1200.0 <= ring_radius <= OMNI_COVER_RADIUS * math.sqrt(3.0):
29         raise ValueError("Supported ring radius is [1200, 900*sqrt(3)] m")
30     return [(0.0, 0.0)] + [
31         (
32             ring_radius * math.cos(k * math.pi / 3.0),
33             ring_radius * math.sin(k * math.pi / 3.0),
34         )
35         for k in range(6)
36     ]
37
38

```

```

39 def ring_cover_radius(ring_radius: float) -> float:
40     """Exact covering radius of origin plus six equally spaced ring points."""
41     omni_waypoints(ring_radius)
42     return max(ring_radius / math.sqrt(3.0),
43               math.sqrt(ARENA_RADIUS**2 + ring_radius**2
44                       - math.sqrt(3.0) * ARENA_RADIUS * ring_radius))
45
46
47 def directional_waypoints(spacing: float = DEFAULT_SPACING) -> list[Point]:
48     """Return a finite triangular lattice with a full source-domain guard.
49
50     For a source g inside the arena, the elementary triangle containing g has
51     vertices within spacing of g and norm <= ARENA_RADIUS + spacing.
52     If g is a lattice vertex, all six nearest neighbours satisfy that same
53     norm bound, so every emission direction has a nonzero forward neighbour.
54     No such six-neighbour claim is made for vertices outside the source arena.
55     Support 900 <= spacing <= 999; ordering is origin then row and column.
56     """
57     spacing = float(spacing)
58     if not math.isfinite(spacing) or not 900.0 <= spacing <= MIN_RADIO_RADIUS - 1.0:
59         raise ValueError("spacing must be finite and in [900, 999] m")
60
61     # With a=(s,0), b=(s/2,s*sqrt(3)/2), the squared norm of i*a+j*b
62     # is s*s*(i*i+i*j+j*j). Compare the radial cutoff using integers, so
63     # floating-point rounding cannot accidentally remove a boundary vertex.
64     numerator, denominator = spacing.as_integer_ratio()
65     guard_numerator = 1800 * denominator + numerator
66     numerator_squared = numerator * numerator
67     guard_squared = guard_numerator * guard_numerator
68     # q=i*i+j*j >= 3*i*i/4 and >= 3*j*j/4. The simpler bound
69     # |i|,|j| <= ceil(2*(1800+s)/s) safely contains the radial selection.
70     bound = (2 * guard_numerator + numerator - 1) // numerator
71     row_height = spacing * math.sqrt(3.0) / 2.0
72     points: list[Point] = [(0.0, 0.0)]
73     for j in range(-bound, bound + 1):
74         for i in range(-bound, bound + 1):
75             if i == 0 and j == 0:
76                 continue
77             norm_squared_units = i * i + i * j + j * j
78             if norm_squared_units * numerator_squared <= guard_squared:
79                 points.append((spacing * (i + j / 2.0), row_height * j))
80     return points
81
82
83 def coverage_certificate() -> dict:
84     """Return analytic constants/counts, not a sampled or experimental result.
85
86     Counts describe the default 990 m lattice. Norm shells are exact integer
87     values of i*i+j*j; their multiplicities can be checked independently.
88     No simulator actions, numerical coverage sampling, or routing occur here.
89     """
90     return {
91         "certificate_kind": "analytic_geometric_cover",
92         "arena_radius_m": ARENA_RADIUS,
93         "minimum_radio_radius_m": MIN_RADIO_RADIUS,
94         "channels": 20,
95         "omni_legacy": {
96             "waypoint_count": 7,
97             "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
98             "cover_radius_m": OMNI_COVER_RADIUS,
99             "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
100            "proof": (
101                "r<=900 is covered by the origin. For 900<=r<=1800, "
102                "choose a ring center with angular gap <=pi/6. Then "
103                "distance^2<=r^2-2700*r+2430000<=810000 by convexity "
104                "and equality of the two endpoint values."
105            ),

```

```

106     },
107     "omni_enhanced": {
108         "waypoint_count": 7,
109         "ring_radius_m": 1200.0,
110         "cover_radius_m": ring_cover_radius(1200.0),
111         "radio_slack_m": MIN_RADIO_RADIUS - ring_cover_radius(1200.0),
112         "proof": "Exact envelope formula ring_cover_radius(1200); no point or channel obligation is removed
113     },
114     "omni": {
115         "waypoint_count": 7,
116         "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
117         "cover_radius_m": OMNI_COVER_RADIUS,
118         "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
119         "proof": "Legacy 900m certificate; enhanced 1200m certificate is in omni_enhanced.",
120     },
121     "directional": {
122         "spacing_m": DEFAULT_SPACING,
123         "basis_m": [
124             [DEFAULT_SPACING, 0.0],
125             [DEFAULT_SPACING / 2.0, DEFAULT_SPACING * math.sqrt(3.0) / 2.0],
126         ],
127         "selection_radius_m": ARENA_RADIUS + DEFAULT_SPACING,
128         "maximum_source_vertex_distance_m": DEFAULT_SPACING,
129         "radio_slack_m": MIN_RADIO_RADIUS - DEFAULT_SPACING,
130         "waypoint_count": 31,
131         "inside_or_on_arena_count": 13,
132         "outside_arena_count": 18,
133         "norm_squared_units_counts": {0: 1, 1: 6, 3: 6, 4: 6, 7: 12},
134         "maximum_waypoint_radius_m": DEFAULT_SPACING * math.sqrt(7.0),
135         "closed_halfplane_required": True,
136         "boundary_guard_required": True,
137         "proof": (
138             "An elementary triangular-lattice triangle containing g has "
139             "three vertices within spacing of g. Their norms are at most "
140             "1800+spacing, hence all are selected. Write "
141             "g=sum(lambda_i*v_i), lambda_i>=0, sum(lambda_i)=1. "
142             "For any emission normal n, sum(lambda_i*n dot (v_i-g))=0, "
143             "so at least one vertex is in the closed emission halfplane. "
144             "If g is a lattice vertex inside the source arena, all six "
145             "nearest neighbors have norm <=1800+spacing and are selected; "
146             "one makes angle <=pi/6 with any emission normal."
147         ),
148         "count_proof": (
149             "At spacing 990 the cutoff is i^2+i*j+j^2<=961/121. "
150             "The possible shells are 0,1,3,4,7 with multiplicities "
151             "1,6,6,6,12. The arena cutoff is 400/121, leaving shells "
152             "0,1,3 inside and shells 4,7 outside."
153         ),
154     },
155     "termination": (
156         "For each of 20 channels, retain either a confirmed successful "
157         "clear or a complete all-no_signal covering scan for that channel. "
158         "A detected but uncleared channel remains unresolved. With at "
159         "most one source per channel, all 20 certificates imply no "
160         "remaining source. Independently, 16 distinct confirmed clears "
161         "reach the stated global upper bound. Fewer clears alone do not."
162     ),
163     "no_signal": {
164         "q3": "Excludes every source at distance <=1000 from that detector.",
165         "q4": (
166             "Does not exclude that disk: a source may emit away from the "
167             "detector. Only the completed per-channel halfplane cover "
168             "excludes every source position and orientation."
169         ),
170     },
171     "assumptions": [

```

```

172         "Sources are stationary, persist until cleared, and use distinct channels.",
173         "Directional emission is a closed 180-degree halfplane with fixed orientation.",
174         "Reception reaches at least 1000 m throughout the emission halfplane.",
175         "All designated measurements actually complete on the indicated channel.",
176         "Near-source responses count as detections, not as no_signal.",
177         "Robot coordinates outside the radius-1800 source arena are permitted.",
178         "Geometric proofs use exact real coordinates; default float coordinates have 10 m range slack.",
179     ],
180 }

```

Listing 8: 环境与模拟器接口

```

1  """Synthetic, not official, CUMCM 2026 B environment (standard library only).
2
3  Generation assumptions are deliberately explicit, not claims about official data:
4  * random: N is discrete uniform on 10..16; N distinct channels sampled uniformly
5    without replacement; positions iid uniform in the radius-1800 disk (sqrt(U)
6    radial law); radio radii iid uniform [1000,1500]; orientations uniform [0,2*pi).
7  Q3 is all omni. Q4 chooses a uniform count in 1..N-1 and a uniform subset
8    of directional sources, guaranteeing both types.
9  * minimum_radius: random geometry, but every radio radius is exactly 1000.
10 * outward_boundary: 16 equally spaced sources on the arena boundary, a seeded
11   common rotation, all radii 1000. Q4 has 15 outward-facing directional sources
12   and one omni, so inward-only search is deliberately inadequate.
13 * clustered: 16 sources uniform in a 35m disk around a seeded center at radius
14   1650; all radio radii 1000, with the random mixed-type rule for Q4.
15
16 Errors are deterministic functions of (seed, channel, exact coordinate): smooth
17 is a bounded two-sinusoid field; extreme is spatial-hashed +/-1 degree;
18 iid_location is spatial-hashed uniform [-1,1], NOT independent repeat readings.
19 The latter is a synthetic hash field, not an assertion of official IID noise.
20 Sources and their parameters are private. Only post-exit summary reveals counts.
21 There are no virtual-duration sleeps; movement time is rounded to microseconds.
22 """
23
24 from __future__ import annotations
25
26 import hashlib
27 import math
28 import random
29 import struct
30 import threading
31 import time
32 from dataclasses import dataclass
33
34 Point = tuple[float, float]
35 SCENARIOS = ("random", "minimum_radius", "outward_boundary", "clustered")
36 ERROR_MODES = ("smooth", "extreme", "iid_location")
37
38
39 def _action_args(path: str, position: Point | None, channel: int | None):
40     """Validate before any state mutation; match protocol coordinate limits."""
41     if path not in ("/enter", "/exit", "/measure", "/clear"):
42         raise ValueError("Unknown action path (paths must be exact)")
43     if path in ("/enter", "/exit"):
44         if position is not None or channel is not None:
45             raise ValueError("enter/exit do not accept position or channel")
46         return None, None
47     if not isinstance(position, (tuple, list)) or len(position) != 2:
48         raise ValueError("position must contain exactly two finite coordinates")
49     if any(isinstance(v, bool) or not isinstance(v, (int, float)) for v in position):
50         raise ValueError("Coordinates must be JSON numbers")
51     try:
52         point = (float(position[0]), float(position[1]))
53     except OverflowError as exc:
54         raise ValueError("Coordinate outside protocol range") from exc
55     if any(not math.isfinite(v) or abs(v) > 2_000_000 for v in point):

```

```

56         raise ValueError("Coordinates must be finite and within +/-2000000m")
57     if (
58         isinstance(channel, bool)
59         or not isinstance(channel, (int, float))
60         or not 1 <= channel <= 20
61         or int(channel) != channel
62     ):
63         raise ValueError("channel must be an integer in 1..20")
64     return point, int(channel)
65
66
67 @dataclass(slots=True)
68 class _Source:
69     position: Point
70     radius: float
71     normal: Point | None
72     phase: float
73     cleared: bool = False
74
75
76 class LocalSimulator:
77     """One local test session; /enter initializes state, duplicate entry rejects.
78
79     log contains only observable request/response/timing data. After termination,
80     construct a new instance for another session, just as the official interface
81     requires a new GUI test. Automatic deadline termination closes this session;
82     no fabricated /exit response is generated in that case.
83     """
84
85     def __init__(
86         self,
87         seed: int,
88         problem: int = 3,
89         error_mode: str = "smooth",
90         scenario: str = "random",
91     ):
92         if isinstance(seed, bool) or not isinstance(seed, int):
93             raise TypeError("seed must be an integer")
94         if problem not in (3, 4):
95             raise ValueError("problem must be 3 or 4")
96         if error_mode not in ERROR_MODES or scenario not in SCENARIOS:
97             raise ValueError(f"Use scenarios {SCENARIOS} and modes {ERROR_MODES}")
98         self._seed, self._problem = seed, problem
99         self._error_mode, self._scenario = error_mode, scenario
100         self._lock = threading.Lock()
101         self._status = "ready"
102         self._sources: dict[int, _Source] = {}
103         self._position: Point = (0.0, 0.0)
104         self._radio_channel = 1
105         self._virtual_us = 0
106         self._distance = 0.0
107         self._measures = self._switches = self._clear_attempts = self._failed_clears = 0
108         self._started: float | None = None
109         self._finished: float | None = None
110         self._exit_reason: str | None = None
111         self.log: list[dict] = []
112
113     def _generate(self):
114         rng = random.Random(self._seed)
115         n = (
116             16
117             if self._scenario in ("outward_boundary", "clustered")
118             else rng.randint(10, 16)
119         )
120         channels = rng.sample(range(1, 21), n)
121         if self._problem == 3:
122             directional = set()

```

```

123     elif self._scenario == "outward_boundary":
124         directional = set(channels[:-1])
125     else:
126         directional = set(rng.sample(channels, rng.randint(1, n - 1)))
127     rotation = rng.uniform(0, math.tau)
128     center = (1650 * math.cos(rotation), 1650 * math.sin(rotation))
129     sources = {}
130     for i, channel in enumerate(channels):
131         theta = rng.uniform(0, math.tau)
132         if self._scenario == "outward_boundary":
133             theta = rotation + math.tau * i / n
134             position = (1800 * math.cos(theta), 1800 * math.sin(theta))
135         elif self._scenario == "clustered":
136             r = 35 * math.sqrt(rng.random())
137             position = (
138                 center[0] + r * math.cos(theta),
139                 center[1] + r * math.sin(theta),
140             )
141         else:
142             r = 1800 * math.sqrt(rng.random())
143             position = (r * math.cos(theta), r * math.sin(theta))
144         radius = rng.uniform(1000, 1500) if self._scenario == "random" else 1000.0
145         orientation = (
146             theta
147             if self._scenario == "outward_boundary"
148             else rng.uniform(0, math.tau)
149         )
150         normal = (
151             (math.cos(orientation), math.sin(orientation))
152             if channel in directional
153             else None
154         )
155         sources[channel] = _Source(
156             position, radius, normal, rng.uniform(0, math.tau)
157         )
158     self._sources = sources
159
160 def _bearing_error(self, point: Point, channel: int, source: _Source) -> float:
161     x, y = point
162     if self._error_mode == "smooth":
163         return 0.6 * math.sin(x / 173 + y / 251 + source.phase) + 0.4 * math.sin(
164             x / 419 - y / 137 + 2 * source.phase
165         )
166     # Canonicalize signed zero: it is the same physical coordinate.
167     packed = struct.pack("!ddB", x or 0.0, y or 0.0, channel)
168     salt = str(self._seed).encode("ascii") + b":"
169     value = int.from_bytes(
170         hashlib.blake2b(salt + packed, digest_size=8).digest(), "big"
171     )
172     if self._error_mode == "extreme":
173         return 1.0 if value & 1 else -1.0
174     return 2 * (value / ((1 << 64) - 1)) - 1
175
176 def _visible(self, source: _Source, point: Point) -> bool:
177     dx, dy = point[0] - source.position[0], point[1] - source.position[1]
178     if math.hypot(dx, dy) > source.radius:
179         return False
180     if source.normal is None:
181         return True
182     a, b = dx * source.normal[0], dy * source.normal[1]
183     # Include the closed halfplane boundary despite floating-point dot error;
184     # tolerance is only a handful of machine ulps, not an angular margin.
185     return a + b >= -8 * math.ulp(max(abs(a), abs(b), 1.0))
186
187 def _response(self, accepted: bool = True, **fields) -> dict:
188     return {
189         "accepted": accepted,

```

```

190         "real_timestamp_ms": int(time.time() * 1000),
191         "virtual_time_s": self._virtual_us / 1_000_000 if accepted else 0,
192         **fields,
193     }
194
195     def _finish(self, reason: str):
196         self._status = "exited"
197         self._finished = time.monotonic()
198         self._exit_reason = reason
199
200     def act(
201         self, path: str, position: Point | None = None, channel: int | None = None
202     ) -> dict:
203         position, channel = _action_args(path, position, channel)
204         with self._lock:
205             if self._status == "exited":
206                 raise ConnectionError(
207                     "Local session interface closed after exit/timeout"
208                 )
209             if self._started is not None and time.monotonic() >= self._started + 1200:
210                 self._finish("real_timeout")
211                 raise ConnectionError("Local session reached its real deadline")
212             request = {"path": path}
213             if position is not None:
214                 request.update(
215                     position={"x": position[0], "y": position[1]}, channel=channel
216                 )
217             before = self._virtual_us
218             radio_before = self._radio_channel
219             distance = 0.0
220             switched = False
221             action_seconds = 0
222             if path == "/enter":
223                 if self._status != "ready":
224                     response = self._response(False)
225                 else:
226                     self._generate()
227                     self._position, self._radio_channel = (0.0, 0.0), 1
228                     self._virtual_us = 0
229                     self._started = time.monotonic()
230                     self._status = "active"
231                     response = self._response(
232                         max_virtual_duration_s=360000,
233                         max_real_duration_s=1200,
234                         remaining_real_duration_s=1200,
235                     )
236             elif self._status != "active":
237                 response = self._response(False)
238             elif path == "/exit":
239                 self._finish("user_exit")
240                 response = self._response(exit_reason="user_exit")
241             else:
242                 distance = math.dist(self._position, position)
243                 source = self._sources.get(channel)
244                 live = source is not None and not source.cleared
245                 if path == "/measure":
246                     switched = channel != self._radio_channel
247                     self._radio_channel = channel
248                     self._switches += int(switched)
249                     self._measures += 1
250                     action_seconds = 5
251                     fields = {"measure_result": "no_signal"}
252                     if live and self._visible(source, position):
253                         if math.dist(position, source.position) <= 5:
254                             fields = {"measure_result": "near"}
255                         else:
256                             angle = math.degrees(

```

```

257         math.atan2(
258             source.position[1] - position[1],
259             source.position[0] - position[0],
260         )
261     )
262     bearing = (
263         round(
264             (
265                 angle
266                 + self._bearing_error(position, channel, source)
267             )
268             % 360,
269             2,
270         )
271         % 360
272     )
273     fields = {"measure_result": "direction", "svd_deg": bearing}
274 else:
275     self._clear_attempts += 1
276     success = live and math.dist(position, source.position) <= 20
277     if success:
278         source.cleared = True
279     else:
280         self._failed_clears += 1
281         action_seconds = 5 if success else 3
282         fields = {
283             "clear_result": "success" if success else "no_target_in_range"
284         }
285     self._position = position
286     self._distance += distance
287     self._virtual_us += (
288         round(distance / 5 * 1_000_000)
289         + (action_seconds + int(switched)) * 1_000_000
290     )
291     response = self._response(**fields)
292     if self._virtual_us >= 360000 * 1_000_000:
293         # An action registered before the deadline is allowed to finish.
294         self._finish("virtual_timeout")
295     self.log.append(
296         {
297             "request": request,
298             "response": dict(response),
299             "distance_m": distance,
300             "move_time_s": round(distance / 5, 6),
301             "switch_time_s": int(switched),
302             "action_time_s": action_seconds,
303             "radio_channel_before": radio_before,
304             "radio_channel_after": self._radio_channel,
305             "action_virtual_time_s": (self._virtual_us - before) / 1_000_000,
306         }
307     )
308     return response
309
310 def summary(self) -> dict:
311     """Evaluation only: inaccessible during a run; never consumed by planner."""
312     with self._lock:
313         if self._status != "exited":
314             raise RuntimeError("summary is available only after session exit")
315         n = len(self._sources)
316         cleared = sum(source.cleared for source in self._sources.values())
317         virtual_time = self._virtual_us / 1_000_000
318         directional = sum(
319             source.normal is not None for source in self._sources.values()
320         )
321     return {
322         "evidence": "local_synthetic_not_official",
323         "seed": self._seed,

```

```

324         "problem": self._problem,
325         "scenario": self._scenario,
326         "error_mode": self._error_mode,
327         "n_sources": n,
328         "n_cleared": cleared,
329         "n_omnidirectional": n - directional,
330         "n_directional": directional,
331         "fraction_cleared": cleared / n,
332         "virtual_time_s": virtual_time,
333         "average_time_s": virtual_time / cleared if cleared else None,
334         "distance_m": self._distance,
335         "measures": self._measures,
336         "switches": self._switches,
337         "clear_attempts": self._clear_attempts,
338         "failed_clears": self._failed_clears,
339         "real_duration_s": self._finished - self._started,
340         "exit_reason": self._exit_reason,
341     }

```

Listing 9: 第二交点计算

```

1  """Q2 robust reception lens and explicitly sampled min-max geometry scoring."""
2
3  from __future__ import annotations
4
5  import json
6  import math
7  from pathlib import Path
8
9  from geometry import clip_bearing, enclosing_circle, initial_polygon
10
11
12  def safe_candidates(station=(0.0, 0.0), bearing_deg=0.0, alpha_deg=1.005):
13      alpha, angle = math.radians(alpha_deg), math.radians(bearing_deg)
14      u, n = (math.cos(angle), math.sin(angle)), (-math.sin(angle), math.cos(angle))
15      choices = []
16      for t in (125.0, 250.0, 375.0, 500.0, 625.0, 750.0, 875.0):
17          bound = min(
18              math.sqrt(1000**2 - t * t),
19              math.sqrt(1000**2 - (t - 1000 * math.cos(alpha)) ** 2)
20              - 1000 * math.sin(alpha),
21          )
22          for proportion in (0.25, 0.5, 0.75):
23              for sign in (-1, 1):
24                  b = sign * proportion * bound
25                  point = (
26                      station[0] + t * u[0] + b * n[0],
27                      station[1] + t * u[1] + b * n[1],
28                  )
29                  choices.append({"point": point, "t_m": t, "b_m": b})
30      return choices
31
32
33  def example():
34      """Normalized illustrative station/readout, not a supplied official case.
35
36      Safe reception follows the analytic lens theorem; sampled ranking does not
37      certify continuous worst-case optimality. No prior distribution is assumed.
38      """
39      first = clip_bearing(initial_polygon(), (0.0, 0.0), 0.0)
40      source_samples = [
41          (r * math.cos(math.radians(phi)), r * math.sin(math.radians(phi)))
42          for r in range(25, 1501, 25)
43          for phi in (-1.0, 0.0, 1.0)
44      ]
45      output = []
46      choices = safe_candidates() + [{"point": (750.0, 0.0), "t_m": 750.0, "b_m": 0.0}]

```

```

47     for candidate in choices:
48         q = candidate["point"]
49         worst_radius = 0.0
50         for source in source_samples:
51             assert math.dist(q, source) <= max(1000, math.dist((0, 0), source)) + 1e-7
52             if math.dist(q, source) <= 5:
53                 radius = 5.0
54             else:
55                 truth = math.degrees(math.atan2(source[1] - q[1], source[0] - q[0]))
56                 radius = 0.0
57                 for error in (-1.0, 0.0, 1.0):
58                     reading = round((truth + error) % 360, 2) % 360
59                     poly = clip_bearing(first, q, reading)
60                     radius = max(radius, enclosing_circle(poly)[1])
61                 worst_radius = max(worst_radius, radius)
62         output.append(
63             {
64                 **candidate,
65                 "sampled_worst_mec_radius_m": worst_radius,
66                 "first_move_and_measure_s": math.hypot(*q) / 5 + 5,
67             }
68         )
69     best = min(
70         output,
71         key=lambda row: (
72             row["sampled_worst_mec_radius_m"],
73             row["first_move_and_measure_s"],
74         ),
75     )
76     result = {
77         "evidence": "illustrative_Q2_grid_score_not_global_optimum",
78         "first_station": [0, 0],
79         "first_reading_deg": 0,
80         "source_samples": len(source_samples),
81         "second_error_values_deg": [-1, 0, 1],
82         "best_sampled_geometry": best,
83         "collinear_control": output[-1],
84         "candidates": output,
85     }
86     Path("results").mkdir(exist_ok=True)
87     Path("results/second_point_example.json").write_text(
88         json.dumps(result, indent=2) + "\n"
89     )
90     print(json.dumps({k: v for k, v in result.items() if k != "candidates"}, indent=2))
91
92
93 if __name__ == "__main__":
94     example()

```

Listing 10: 恢复策略比较

```

1 import json
2 import statistics
3 from pathlib import Path
4
5 from environment import LocalSimulator
6 from planner import Planner
7
8 rows, summary = [], {}
9 for scenario in ("random", "minimum_radius", "outward_boundary", "clustered"):
10     for enabled in (False, True):
11         name = "recovery" if enabled else "two_flanks"
12         group = []
13         for seed in range(500000, 500100):
14             env = LocalSimulator(seed, 4, "smooth", scenario)
15             result = Planner(env, 4, strategy="integrated", loss_recovery=enabled).run()
16             actual = env.summary()

```

```

17         assert actual["fraction_cleared"] == 1
18         group.append(actual)
19         rows.append(**actual, "policy": name)
20         summary[f"{scenario}_{name}"] = {
21             "runs": len(group),
22             "mean_average_time_s": statistics.mean(r["average_time_s"] for r in group),
23             "mean_failed_clears": statistics.mean(r["failed_clears"] for r in group),
24             "worst_virtual_time_s": max(r["virtual_time_s"] for r in group),
25         }
26     result = {
27         "evidence": "synthetic_development_comparison_not_final_holdout",
28         "summary": summary,
29         "runs": rows,
30     }
31     Path("results/recovery_development.json").write_text(
32         json.dumps(result, indent=2) + "\n"
33     )
34     print(json.dumps(summary, indent=2))

```

Listing 11: HTTP 客户端

```

1  """Serial, append-only-logged HTTP adapter for an already-open simulator test.
2
3  This module does NOT authenticate, launch a GUI, or start a formal test. The user
4  must first start the intended test in the official GUI and wait for readiness.
5  Only explicit act calls cause simulator actions; close never sends /exit.
6
7  A new logical action receives a UUID. Transient retries preserve exactly the
8  same bytes, path, and request ID. Exhausted ambiguous actions fail closed: no
9  new action can be submitted on this client because the previous action may have
10 executed. Logs are ordinary plaintext observations, NOT official encrypted logs.
11 Real sleeps are used solely for short transport-retry backoff, never virtual time.
12 """
13
14 from __future__ import annotations
15
16 import http.client
17 import json
18 import math
19 import queue
20 import threading
21 import time
22 import unicodedata
23 import uuid
24 from pathlib import Path
25 from urllib.error import HTTPError, URLError
26 from urllib.parse import urlsplit
27 from urllib.request import HTTPRedirectHandler, ProxyHandler, Request, build_opener
28
29 from environment import Point, _action_args
30
31
32 class ClientError(RuntimeError):
33     """An action did not produce a verified accepted response."""
34
35
36 class RejectedActionError(ClientError):
37     def __init__(self, status: int, response: dict):
38         self.status, self.response = status, response
39         super().__init__(
40             f"Simulator rejected action: HTTP {status}, accepted={response.get('accepted')}!r}"
41         )
42
43
44 class AmbiguousActionError(ClientError):
45     """The action may have executed; consult the log and simulator GUI."""
46

```

```

47
48 class ClientProtocolError(ClientError):
49     pass
50
51
52 class DeadlineExceeded(ClientError):
53     pass
54
55
56 class _NoRedirect(HTTPRedirectHandler):
57     def redirect_request(self, req, fp, code, msg, headers, newurl):
58         # Never let urllib turn a POST into a GET or send identifiers elsewhere.
59         return None
60
61
62 def _number(value) -> bool:
63     return (
64         not isinstance(value, bool)
65         and isinstance(value, (int, float))
66         and math.isfinite(value)
67     )
68
69
70 def _unique_object(pairs):
71     result = {}
72     for key, value in pairs:
73         if key in result:
74             raise ValueError(f"Duplicate response key: {key}")
75         result[key] = value
76     return result
77
78
79 class HTTPClient:
80     """Official wire schema, no truth API, one in-flight logical action at a time.
81
82     Cached public position/radio_channel/virtual_time_s describe only confirmed
83     accepted actions. The real deadline uses /enter.remaining_real_duration_s,
84     conservatively subtracting all elapsed time since its first send, including
85     transport retries. Before entry, a bounded 20-second connection budget applies
86     because the actual GUI window deadline is not available through the API.
87     """
88
89     _MAX_ATTEMPTS = 4
90     _REQUEST_TIMEOUT_S = 5.0
91     _ACTION_RETRY_BUDGET_S = 20.0
92     _MAX_RESPONSE_BYTES = 65536
93     _TRANSIENT_HTTP = frozenset((429, 500, 502, 503, 504))
94
95     def __init__(self, base_url: str, robot_id: str, log_path: str):
96         parsed = urlsplit(base_url)
97         if (
98             parsed.scheme not in ("http", "https")
99             or not parsed.hostname
100             or parsed.path not in ("", "/")
101             or parsed.query
102             or parsed.fragment
103             or parsed.username is not None
104             or parsed.password is not None
105         ):
106             raise ValueError(
107                 "base_url must be an HTTP(S) origin, without credentials/path/query"
108             )
109         # Force port validation before sending anything.
110         _ = parsed.port
111         if (
112             not isinstance(robot_id, str)
113             or not 1 <= len(robot_id.encode("utf-8")) <= 64

```

```

114         ):
115             raise ValueError("robot_id must contain 1..64 UTF-8 bytes")
116         if any(unicodedata.category(c) in ("Cc", "Cf") for c in robot_id):
117             raise ValueError("robot_id cannot contain control or format characters")
118         self._base_url, self._robot_id = base_url.rstrip("/"), robot_id
119         self._lock = threading.Lock()
120         self._opener = build_opener(ProxyHandler({}), _NoRedirect())
121         self._closed = False
122         self._uncertain = False
123         self._entered = self._exited = False
124         self._deadline: float | None = None
125         self._max_virtual_duration_s: float | None = None
126         self.position: Point = (0.0, 0.0)
127         self.radio_channel = 1
128         self.virtual_time_s = 0.0
129         self._log = Path(log_path).open("a", encoding="utf-8", buffering=1) # noqa: SIM115 - lifecycle closed
130         by close()
131
132     @property
133     def remaining_real_duration_s(self) -> float | None:
134         """Locally conservative seconds left; unknown until accepted /enter."""
135         return (
136             None
137             if self._deadline is None
138             else max(0.0, self._deadline - time.monotonic())
139         )
140
141     def _record(self, event: dict):
142         self._log.write(
143             json.dumps(
144                 {"logged_at_ms": int(time.time() * 1000), **event},
145                 ensure_ascii=False,
146                 allow_nan=False,
147                 separators=(",", ":"),
148             )
149             + "\n"
150         )
151         self._log.flush()
152
153     def _decode(self, raw: bytes) -> dict:
154         if len(raw) > self._MAX_RESPONSE_BYTES:
155             raise ClientProtocolError("Response exceeds bounded JSON body size")
156         try:
157             response = json.loads(raw.decode("utf-8"), object_pairs_hook=_unique_object)
158         except (UnicodeDecodeError, ValueError) as exc:
159             raise ClientProtocolError(
160                 "Response is not valid unambiguous UTF-8 JSON"
161             ) from exc
162         if not isinstance(response, dict) or type(response.get("accepted")) is not bool:
163             raise ClientProtocolError("Response lacks a boolean accepted field")
164         if (
165             not _number(response.get("real_timestamp_ms"))
166             or not _number(response.get("virtual_time_s"))
167             or response["virtual_time_s"] < 0
168         ):
169             raise ClientProtocolError("Response lacks valid numeric timestamps")
170         return response
171
172     def _validate_accepted(self, path: str, response: dict):
173         virtual_time = response["virtual_time_s"]
174         if path == "/enter":
175             remaining = response.get("remaining_real_duration_s")
176             if (
177                 not _number(remaining)
178                 or not 0 <= remaining <= 1200
179                 or int(remaining) != remaining
180             ):

```

```

180         raise ClientProtocolError("Invalid actual remaining real duration")
181     if (
182         not _number(response.get("max_virtual_duration_s"))
183         or response["max_virtual_duration_s"] <= 0
184         or not _number(response.get("max_real_duration_s"))
185         or response["max_real_duration_s"] <= 0
186         or virtual_time != 0
187     ):
188         raise ClientProtocolError(
189             "Invalid accepted entry limits or initial time"
190         )
191     elif virtual_time < self.virtual_time_s:
192         raise ClientProtocolError("Accepted action moved virtual time backwards")
193     if path == "/measure":
194         result = response.get("measure_result")
195         if result not in ("near", "no_signal", "direction"):
196             raise ClientProtocolError("Unknown measure result")
197         if result == "direction":
198             angle = response.get("svd_deg")
199             if not _number(angle) or not 0 <= angle < 360:
200                 raise ClientProtocolError("Invalid direction angle")
201         elif "svd_deg" in response:
202             raise ClientProtocolError("Bearing supplied without direction result")
203     elif path == "/clear" and response.get("clear_result") not in (
204         "success",
205         "no_target_in_range",
206     ):
207         raise ClientProtocolError("Unknown clear result")
208     elif path == "/exit" and response.get("exit_reason") != "user_exit":
209         raise ClientProtocolError("Unexpected successful exit reason")
210
211     def _commit(
212         self,
213         path: str,
214         position: Point | None,
215         channel: int | None,
216         response: dict,
217         first_send: float,
218     ):
219         if path == "/enter":
220             self._entered = True
221             self.position, self.radio_channel = (0.0, 0.0), 1
222             self._deadline = first_send + response["remaining_real_duration_s"]
223             self._max_virtual_duration_s = response["max_virtual_duration_s"]
224         elif path == "/exit":
225             self._exited = True
226         else:
227             self.position = position
228             if path == "/measure":
229                 self.radio_channel = channel
230             self.virtual_time_s = response["virtual_time_s"]
231             self._uncertain = False
232
233     def _exchange(self, request: Request, deadline: float) -> tuple[int, bytes]:
234         """Bound connection, headers and body together, not just idle sockets.
235
236         On expiry the daemon may finish the same in-flight request, but cannot
237         mutate client state or send another action. act remains fail-closed.
238         """
239         completed = queue.Queue(maxsize=1)
240
241     def receive():
242         try:
243             available = deadline - time.monotonic()
244             if available <= 0:
245                 raise TimeoutError("Absolute HTTP deadline elapsed")
246         try:

```

```

247         stream = self._opener.open(
248             request, timeout=min(self._REQUEST_TIMEOUT_S, available)
249         )
250     except HTTPError as exc:
251         stream = exc
252     with stream:
253         status = stream.code
254         expected_length = stream.headers.get("Content-Length")
255         if expected_length is not None:
256             try:
257                 expected_length = int(expected_length)
258             except ValueError as exc:
259                 raise ClientProtocolError("Invalid Content-Length") from exc
260             if not 0 <= expected_length <= self._MAX_RESPONSE_BYTES:
261                 raise ClientProtocolError(
262                     "Response exceeds bounded JSON body size"
263                 )
264         raw = stream.read(self._MAX_RESPONSE_BYTES + 1)
265         if expected_length is not None and len(raw) < expected_length:
266             raise http.client.IncompleteRead(
267                 raw, expected_length - len(raw)
268             )
269         completed.put((True, (status, raw)))
270     except (
271         ClientError,
272         OSError,
273         ValueError,
274         TypeError,
275         TimeoutError,
276         http.client.HTTPException,
277     ) as exc:
278         completed.put((False, exc))
279
280     threading.Thread(target=receive, daemon=True, name="simulator-response").start()
281     try:
282         succeeded, value = completed.get(
283             timeout=max(0.0, deadline - time.monotonic())
284         )
285     except queue.Empty as exc:
286         raise TimeoutError(
287             "Absolute HTTP deadline elapsed; outcome remains unknown"
288         ) from exc
289     if not succeeded:
290         raise value
291     return value
292
293     def act(
294         self, path: str, position: Point | None = None, channel: int | None = None
295     ) -> dict:
296         position, channel = _action_args(path, position, channel)
297         with self._lock:
298             if self._closed:
299                 raise ClientError("Client is closed")
300             if self._uncertain:
301                 raise AmbiguousActionError(
302                     "Previous outcome is unresolved; no different action may be sent"
303                 )
304             if self._exited:
305                 raise ClientError("Session already exited; do not query /exit again")
306             if self._deadline is not None and time.monotonic() >= self._deadline:
307                 raise DeadlineExceeded(
308                     "Actual remaining real duration has elapsed; interface may be closed"
309                 )
310             if (
311                 self._max_virtual_duration_s is not None
312                 and self.virtual_time_s >= self._max_virtual_duration_s
313             ):

```

```

314         raise DeadlineExceeded(
315             "Virtual deadline reached; no post-timeout /exit query is legal"
316         )
317     if path == "/enter" and self._entered:
318         raise ClientError("Cannot enter the same test twice")
319     if path != "/enter" and not self._entered:
320         raise ClientError(
321             "Explicit successful /enter is required before other actions"
322         )
323     request_id = uuid.uuid4().hex
324     payload = {
325         "arena_id": "default",
326         "robot_id": self._robot_id,
327         "request_id": request_id,
328     }
329     if position is not None:
330         payload.update(
331             position={"x": position[0], "y": position[1]}, channel=channel
332         )
333     body = json.dumps(
334         payload, ensure_ascii=False, allow_nan=False, separators=(",", ":"),
335     ).encode("utf-8")
336     first_send = time.monotonic()
337     stop = first_send + self._ACTION_RETRY_BUDGET_S
338     if self._deadline is not None:
339         stop = min(stop, self._deadline)
340     last_error = "No response"
341     for attempt in range(1, self._MAX_ATTEMPTS + 1):
342         available = stop - time.monotonic()
343         if available <= 0:
344             break
345         self._record(
346             {
347                 "event": "request",
348                 "path": path,
349                 "request_id": request_id,
350                 "attempt": attempt,
351                 "body": payload,
352             }
353         )
354         request = Request(
355             self._base_url + path,
356             data=body,
357             headers={"Content-Type": "application/json"},
358             method="POST",
359         )
360         # From this point until a valid outcome, execution is uncertain.
361         self._uncertain = True
362         try:
363             status, raw = self._exchange(request, stop)
364             self._record(
365                 {
366                     "event": "response",
367                     "path": path,
368                     "request_id": request_id,
369                     "attempt": attempt,
370                     "http_status": status,
371                     "body_utf8": raw.decode("utf-8", errors="replace"),
372                 }
373             )
374         except (URLError, OSError, http.client.HTTPException) as exc:
375             # Includes connection refusal, broken connections, timeouts,
376             # truncated HTTP bodies. No state is inferred from any of these.
377             last_error = f"{type(exc).__name__}: {exc}"
378             self._record(
379                 {
380                     "event": "transport_error",

```

```

381         "path": path,
382         "request_id": request_id,
383         "attempt": attempt,
384         "error": last_error,
385     }
386 )
387 else:
388     if status in self._TRANSIENT_HTTP:
389         last_error = f"Transient HTTP {status}"
390     else:
391         response = self._decode(raw)
392         if status != 200 or response["accepted"] is not True:
393             if response["accepted"] is True:
394                 raise ClientProtocolError(
395                     "accepted=true with non-200 HTTP status"
396                 )
397             self._uncertain = False
398             raise RejectedActionError(status, response)
399         self._validate_accepted(path, response)
400         radio_before = self.radio_channel
401         distance = (
402             math.dist(self.position, position)
403             if position is not None
404             else 0.0
405         )
406         switch = int(path == "/measure" and channel != radio_before)
407         self._record(
408             {
409                 "event": "accepted",
410                 "path": path,
411                 "request_id": request_id,
412                 "distance_m": distance,
413                 "radio_channel_before": radio_before,
414                 "radio_channel_after": channel
415                 if path == "/measure"
416                 else radio_before,
417                 "switch_time_s": switch,
418                 "action_virtual_time_s": response["virtual_time_s"]
419                 - self.virtual_time_s,
420             }
421         )
422         self._commit(path, position, channel, response, first_send)
423         return response
424     if attempt < self._MAX_ATTEMPTS:
425         delay = min(
426             0.2 * 2 ** (attempt - 1), max(0.0, stop - time.monotonic())
427         )
428         if delay > 0:
429             time.sleep(delay)
430     raise AmbiguousActionError(
431         f"Action {request_id} has no confirmed outcome after bounded retries: {last_error}"
432     )
433
434 def close(self):
435     """Close only this adapter's log. Never start/end a simulator session."""
436     with self._lock:
437         if not self._closed:
438             self._log.close()
439             self._closed = True

```

Listing 12: HTTP 验证

```

1  """HTTP fault-injection regression. Local test server, never the official simulator.
2
3  Start --server with hub; run without --server as a finite client verification.
4  The server executes the first measure, truncates its HTTP body, then answers its
5  same-ID retry from cache. Passing requires exactly-once virtual-time effects.

```

```

6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import json
12 import tempfile
13 import time
14 from http.server import BaseHTTPRequestHandler, HTTPServer
15 from pathlib import Path
16
17 from client import AmbiguousActionError, HTTPClient
18 from environment import LocalSimulator
19
20
21 def serve(port, case):
22     environment = LocalSimulator(20260910)
23     cache = {}
24     truncated = False
25
26     class Handler(BaseHTTPRequestHandler):
27         def log_message(self, format, *args):
28             return
29
30         def do_POST(self):
31             nonlocal truncated
32             raw = self.rfile.read(int(self.headers["Content-Length"]))
33             body = json.loads(raw)
34             identity = body["request_id"]
35             key = (self.path, raw)
36             if identity in cache:
37                 old_key, response = cache[identity]
38                 if key != old_key:
39                     self.send_error(409)
40                     return
41             else:
42                 point = body.get("position")
43                 point = None if point is None else (point["x"], point["y"])
44                 response = environment.act(self.path, point, body.get("channel"))
45                 cache[identity] = (key, response)
46             encoded = json.dumps(response).encode()
47             self.send_response(200)
48             self.send_header("Content-Type", "application/json")
49             self.send_header("Content-Length", str(len(encoded)))
50             self.end_headers()
51             if self.path == "/measure" and not truncated:
52                 truncated = True
53                 if case == "deadline":
54                     size = max(1, len(encoded) // 8)
55                     for start in range(0, len(encoded), size):
56                         self.wfile.write(encoded[start : start + size])
57                         self.wfile.flush()
58                         time.sleep(0.3)
59                 else:
60                     self.wfile.write(encoded[: len(encoded) // 2])
61                     self.wfile.flush()
62                     self.close_connection = True
63             else:
64                 self.wfile.write(encoded)
65
66     server = HTTPServer(("127.0.0.1", port), Handler)
67     print(f"HTTP fault fixture ready on {port}", flush=True)
68     server.serve_forever()
69
70
71 def verify(port, case):
72     Path("results").mkdir(exist_ok=True)

```

```

73 with tempfile.TemporaryDirectory(prefix="b-http-check-") as directory:
74     path = Path(directory) / "requests.jsonl"
75     client = HTTPClient(f"http://127.0.0.1:{port}", "local-regression", str(path))
76     try:
77         client.act("/enter")
78         if case == "deadline":
79             client._ACTION_RETRY_BUDGET_S = 0.5
80             started = time.monotonic()
81             try:
82                 client.act("/measure", (300.0, 400.0), 1)
83             except AmbiguousActionError:
84                 elapsed = time.monotonic() - started
85             else:
86                 raise AssertionError("Slow response bypassed the absolute deadline")
87             assert elapsed < 1.5, elapsed
88             assert client.virtual_time_s == 0
89             try:
90                 client.act("/measure", (0.0, 0.0), 2)
91             except AmbiguousActionError:
92                 pass
93             else:
94                 raise AssertionError(
95                     "Different action sent after an ambiguous timeout"
96                 )
97             result = {
98                 "evidence": "local_HTTP_fault_fixture_not_official",
99                 "absolute_deadline_fail_closed": True,
100                 "deadline_s": 0.5,
101                 "observed_elapsed_s": elapsed,
102                 "no_unconfirmed_state_advance": True,
103             }
104             output = Path("results/http_deadline_verification.json")
105         else:
106             assert (
107                 client.act("/measure", (300.0, 400.0), 1)["virtual_time_s"] == 105
108             )
109             assert (
110                 client.act("/measure", (300.0, 400.0), 2)["virtual_time_s"] == 111
111             )
112             clear = client.act("/clear", (300.0, 0.0), 3)
113             expected = 196 if clear["clear_result"] == "success" else 194
114             assert clear["virtual_time_s"] == expected
115             assert client.radio_channel == 2
116             assert (
117                 client.act("/measure", (300.0, 0.0), 2)["virtual_time_s"]
118                 == expected + 5
119             )
120             assert client.act("/exit")["virtual_time_s"] == expected + 5
121             result = {
122                 "evidence": "local_HTTP_fault_fixture_not_official",
123                 "truncated_response_recovered": True,
124                 "retry_reused_identical_request": True,
125                 "virtual_time_exactly_once": True,
126                 "clear_kept_radio_channel": True,
127                 "final_virtual_time_s": expected + 5,
128             }
129             output = Path("results/http_verification.json")
130         finally:
131             client.close()
132         events = [json.loads(line) for line in path.read_text().splitlines()]
133         requests = [
134             row
135             for row in events
136             if row["event"] == "request" and row["path"] == "/measure"
137         ]
138         if case == "deadline":
139             assert len(requests) == 1

```

```

140         else:
141             assert requests[0]["request_id"] == requests[1]["request_id"]
142             assert requests[0]["body"] == requests[1]["body"]
143             output.write_text(json.dumps(result, indent=2) + "\n")
144             output.with_suffix(".jsonl").write_text(path.read_text())
145         print(json.dumps(result, indent=2))
146
147
148 if __name__ == "__main__":
149     parser = argparse.ArgumentParser()
150     parser.add_argument("--server", action="store_true")
151     parser.add_argument("--port", type=int, default=12026)
152     parser.add_argument(
153         "--case", choices=("truncated", "deadline"), default="truncated"
154     )
155     arguments = parser.parse_args()
156     serve(arguments.port, arguments.case) if arguments.server else verify(
157         arguments.port, arguments.case
158     )

```

Listing 13: 官方运行入口（仅获授权后使用）

```

1  """Connect the policy to an already started OFFICIAL simulator session.
2
3  Never starts a formal test or supplies team credentials automatically. Use the
4  real GUI to choose the intended practice/formal module, then invoke this CLI.
5  Plaintext request logs include robot_id and are private working files: do not
6  put them unredacted in anonymous submission. Official encrypted logs are exported
7  separately from the simulator with unchanged filenames.
8  """
9
10 from __future__ import annotations
11
12 import argparse
13 import json
14 from pathlib import Path
15 from time import monotonic
16
17 from client import HTTPClient
18 from planner import Planner
19
20
21 def main():
22     parser = argparse.ArgumentParser()
23     parser.add_argument("--robot-id", required=True)
24     parser.add_argument("--problem", required=True, type=int, choices=(3, 4))
25     parser.add_argument("--base-url", default="http://127.0.0.1:2026")
26     parser.add_argument("--output-dir", type=Path, required=True)
27     args = parser.parse_args()
28     if args.output_dir.exists():
29         parser.error("Output directory must be new to keep separate session evidence")
30     args.output_dir.mkdir(parents=True)
31     client = HTTPClient(
32         args.base_url, args.robot_id, str(args.output_dir / "private_requests.jsonl")
33     )
34     started = monotonic()
35     try:
36         result = Planner(client, problem=args.problem).run()
37         result.update(
38             evidence="official_HTTP_observations_not_encrypted_export",
39             local_wall_duration_s=monotonic() - started,
40             case_code="Obtain actual case code from simulator GUI; not returned by HTTP",
41             total_source_count="Not disclosed by official formal interface",
42         )
43         (args.output_dir / "observed_result.json").write_text(
44             json.dumps(result, ensure_ascii=False, indent=2) + "\n"
45         )

```

```

46     print(json.dumps(result, ensure_ascii=False, indent=2))
47 finally:
48     client.close()
49
50
51 if __name__ == "__main__":
52     main()

```

Listing 14: 数据驱动图表与轨迹生成

```

1  """Deterministic local-evidence assets; run with conda run -n py314 python make_assets.py.
2
3  No official service is accessed. Replay logs contain public observations, not source
4  coordinates. Wall-clock durations in raw replay evidence are inherently variable;
5  figures, table values, and virtual-time paths do not depend on those durations.
6  """
7
8  from __future__ import annotations
9
10 import hashlib
11 import json
12 import math
13 import os
14 import statistics
15 import tempfile
16 from pathlib import Path
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import numpy as np
23 from matplotlib.patches import Circle, Polygon
24
25 from coverage import directional_waypoints, omni_waypoints
26 from environment import LocalSimulator
27 from geometry import BEARING_ERROR_DEG, clip_bearing, enclosing_circle, initial_polygon
28 from planner import Planner
29
30 ROOT = Path(__file__).resolve().parent
31 GENERATED = ROOT / "report" / "generated"
32 FIGURES = ROOT / "report" / "figures"
33 POLICIES = ("baseline", "batched", "integrated")
34 SCENARIOS = ("random", "minimum_radius", "outward_boundary", "clustered")
35 SCENARIO_LABELS = ("Random", "Min. radius", "Outward boundary", "Clustered")
36 COLORS = ("0072B2", "E69F00", "009E73")
37 WIDTH = 16 / 2.54
38
39
40 def require(condition, message):
41     if not condition:
42         raise ValueError(message)
43
44
45 def atomic_bytes(path, content):
46     """Replace only our named outputs; never remove/rewrite source evidence."""
47     require(path.parent in (GENERATED, FIGURES), "Output outside owned directories")
48     path.parent.mkdir(parents=True, exist_ok=True)
49     fd, temporary = tempfile.mkstemp(prefix="." + path.name + ".", dir=path.parent)
50     try:
51         with os.fdopen(fd, "wb") as stream:
52             stream.write(content)
53         os.replace(temporary, path)
54     finally:
55         Path(temporary).unlink(missing_ok=True)
56
57

```

```

58 def write_json(name, value):
59     atomic_bytes(
60         GENERATED / name,
61         (
62             json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n"
63         ).encode("utf-8"),
64     )
65
66
67 def save_figure(fig, name):
68     # Fixed physical width, not bbox_inches=tight (which changes final font scale).
69     from io import BytesIO
70
71     buffer = BytesIO()
72     fig.savefig(
73         buffer,
74         format="pdf",
75         metadata={"CreationDate": None, "ModDate": None, "Creator": "make_assets.py"},
76     )
77     atomic_bytes(FIGURES / name, buffer.getvalue())
78     plt.close(fig)
79
80
81 def tex_escape(value):
82     return str(value).replace("_", r"\_").replace("%", r"\%")
83
84
85 def table(name, headers, rows, alignment=None):
86     alignment = alignment or "l" + "r" * (len(headers) - 1)
87     lines = [
88         r"\begin{tabular}{%s} %s" % (alignment, headers),
89         r"\toprule",
90         " & ".join(headers) + r" \\",
91         r"\midrule",
92     ]
93     for row in rows:
94         require(len(row) == len(headers), "Ragged table")
95         lines.append(" & ".join(str(v) for v in row) + r" \\\n")
96     lines += [r"\bottomrule", r"\end{tabular}"]
97     atomic_bytes(GENERATED / name, ("\\n".join(lines) + "\\n").encode("utf-8"))
98
99
100 def number(value):
101     require(
102         isinstance(value, (int, float)) and math.isfinite(value), "Nonfinite metric"
103     )
104     return f"{value:.2f}"
105
106
107 def metrics(rows):
108     values = sorted(row["average_time_s"] for row in rows)
109     require(bool(values), "Empty evidence group")
110     return {
111         "cases": len(rows),
112         "all_cleared_cases": sum(r["fraction_cleared"] == 1 for r in rows),
113         "mean_average_time_s": statistics.mean(values),
114         "median_average_time_s": statistics.median(values),
115         "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1],
116         "mean_failed_clears": statistics.mean(r["failed_clears"] for r in rows),
117     }
118
119
120 def validate_group(rows, summary, name):
121     actual = metrics(rows)
122     for key, value in actual.items():
123         require(
124             key in summary

```

```

125         and math.isclose(value, summary[key], rel_tol=1e-10, abs_tol=1e-8),
126         f"{name}: summary mismatch for {key}",
127     )
128     return rows
129
130
131 def load_inputs():
132     names = (
133         "final_experiments.json",
134         "geometry_verification.json",
135         "http_verification.json",
136         "second_point_example.json",
137     )
138     data, hashes = {}, {}
139     for name in names:
140         raw = (ROOT / "results" / name).read_bytes()
141         data[name] = json.loads(raw)
142         hashes["results/" + name] = hashlib.sha256(raw).hexdigest()
143     final = data[names[0]]
144     require(
145         final["evidence"] == "local_synthetic_not_official", "Not synthetic evidence"
146     )
147     require(final["total_runs"] == len(final["runs"]), "Run denominator mismatch")
148     for row in final["runs"]:
149         require(
150             row["evidence"] == "local_synthetic_not_official"
151             and row["fraction_cleared"] == 1,
152             "Incomplete or wrongly attributed run",
153         )
154         require(row["n_sources"] == row["n_cleared"] > 0, "Invalid clearing count")
155         require(
156             math.isclose(
157                 row["average_time_s"],
158                 row["virtual_time_s"] / row["n_sources"],
159                 rel_tol=1e-10,
160             ),
161             "Metric is not per-world T/N",
162         )
163         require(row["timing_independently_verified"] is True, "Missing timing evidence")
164         require(row["certificate_complete"] is True, "Missing complete certificate")
165         require(row["loss_recovery"] is (row["problem"] == 4 and row.get("loss_recovery_requested", True)),
166                 "Effective recovery mismatch")
167     main, recovery = {}, {}
168     n = final["paired_random_cases_per_problem"]
169     for problem in (3, 4):
170         for strategy in POLICIES:
171             key = f"Q{problem}_{strategy}"
172             rows = sorted(
173                 (
174                     r
175                     for r in final["runs"]
176                     if r["problem"] == problem
177                     and r["strategy"] == strategy
178                     and 600000 <= r["seed"] < 600000 + n
179                 ),
180                 key=lambda r: r["seed"],
181             )
182             require(
183                 [r["seed"] for r in rows] == list(range(600000, 600000 + n)),
184                 "Missing/duplicate paired seed",
185             )
186             main[key] = validate_group(rows, final["summary"][key], key)
187     paired = final["paired_comparison"][f"Q{problem}"]
188     saved = statistics.mean(
189         a["average_time_s"] - b["average_time_s"]
190         for a, b in zip(
191             main[f"Q{problem}_baseline"], main[f"Q{problem}_integrated"]

```

```

192     )
193 )
194 require(
195     paired["paired_cases"] == n
196     and math.isclose(saved, paired["mean_saved_s_per_source"], rel_tol=1e-10),
197     "Pairing mismatch",
198 )
199 require(
200     math.isclose(
201         paired["mean_time_reduction_pct"],
202         100
203         * saved
204         / final["summary"][f"Q{problem}_baseline"]["mean_average_time_s"],
205         rel_tol=1e-10,
206     ),
207     "Gain mismatch",
208 )
209 for scenario in SCENARIOS[1:]:
210     for mode in ("smooth", "extreme", "iid_location"):
211         key = f"Q{problem}_{scenario}_{mode}"
212         rows = [
213             r
214             for r in final["runs"]
215             if r["problem"] == problem
216             and r["scenario"] == scenario
217             and r["error_mode"] == mode
218             and 700000 <= r["seed"] < 700020
219         ]
220         validate_group(rows, final["stress"][key], key)
221 for scale in (0.12, 0.22, 0.40):
222     key = f"Q{problem}_probe_{scale}"
223     rows = [
224         r
225         for r in final["runs"]
226         if r["problem"] == problem
227         and r["probe_scale"] == scale
228         and 800000 <= r["seed"] < 800030
229     ]
230     validate_group(rows, final["sensitivity"][key], key)
231 for spacing in (900.0, 950.0, 990.0):
232     key = f"Q4_spacing_{spacing}"
233     rows = [
234         r
235         for r in final["runs"]
236         if r["problem"] == 4
237         and r["spacing_m"] == spacing
238         and 900000 <= r["seed"] < 900030
239     ]
240     validate_group(rows, final["sensitivity"][key], key)
241 for scenario in SCENARIOS:
242     for enabled in (False, True):
243         key = f"{scenario}_recovery_{enabled}"
244         rows = sorted(
245             (
246                 r
247                 for r in final["runs"]
248                 if r["problem"] == 4
249                 and r["scenario"] == scenario
250                 and r["loss_recovery"] is enabled
251                 and 610000 <= r["seed"] < 610100
252             ),
253             key=lambda r: r["seed"],
254         )
255         require(
256             [r["seed"] for r in rows] == list(range(610000, 610100)),
257             "Recovery pairing mismatch",
258         )

```

```

259         recovery[key] = validate_group(rows, final["recovery_ablation"][key], key)
260 geometry, http, second = (data[name] for name in names[1:])
261 require(
262     geometry["evidence"]
263     == "independent_numerical_checks_not_a_substitute_for_analytic_proofs",
264     "Geometry evidence mismatch",
265 )
266 require(
267     http["evidence"] == "local_HTTP_fault_fixture_not_official",
268     "HTTP evidence mismatch",
269 )
270 require(
271     second["evidence"] == "illustrative_Q2_grid_score_not_global_optimum",
272     "Q2 evidence mismatch",
273 )
274 for key in (
275     "truncated_response_recovered",
276     "retry_reused_identical_request",
277     "virtual_time_exactly_once",
278     "clear_kept_radio_channel",
279 ):
280     require(http[key] is True, f"Failed fixture: {key}")
281 return final, geometry, http, second, main, recovery, hashes
282
283
284 def generate_tables(final, geometry, http):
285     summary = final["summary"]
286     macros = {"ExperimentRuns": str(final["total_runs"])}
287     for problem, word in ((3, "Three"), (4, "Four")):
288         macros[f"Q{word}Time"] = number(
289             summary[f"Q{problem}_integrated"]["mean_average_time_s"]
290         )
291         macros[f"Q{word}Baseline"] = number(
292             summary[f"Q{problem}_baseline"]["mean_average_time_s"]
293         )
294         # Numeric macro: paper supplies its own percent sign and unit.
295         macros[f"Q{word}Gain"] = number(
296             final["paired_comparison"][f"Q{problem}"]["mean_time_reduction_pct"]
297         )
298     atomic_bytes(
299         GENERATED / "numbers.tex",
300         (
301             "\n".join(
302                 r"\newcommand{" + "\\ " + k + "}" + v + "}" for k, v in macros.items()
303             )
304             + "\n"
305         ).encode(),
306     )
307     rows = []
308     for problem in (3, 4):
309         for policy in POLICIES:
310             s = summary[f"Q{problem}_{policy}"]
311             rows.append(
312                 (
313                     f"Q{problem} {policy}",
314                     number(s["mean_average_time_s"]),
315                     number(s["median_average_time_s"]),
316                     number(s["p90_average_time_s"]),
317                     f"{s['all_cleared_cases']}/{s['cases']}",
318                 )
319             )
320     table(
321         "comparison_table.tex",
322         ("Policy", r"Mean $T/N$ (s)", "Median (s)", "$p_{90}$ (s)", "Complete"),
323         rows,
324     )
325     rows = []

```

```

326 for scenario, label in zip(SCENARIOS[1:], SCENARIO_LABELS[1:]):
327     for mode, mode_label in (
328         ("smooth", "Smooth"),
329         ("extreme", r"$\pm 1^{\circ}$ hash"),
330         ("iid_location", "Uniform hash"),
331     ):
332         s3, s4 = (final["stress"][f"Q{p}_{scenario}_{mode}"] for p in (3, 4))
333         require(s3["cases"] == s4["cases"], "Stress denominator mismatch")
334         rows.append(
335             (
336                 label,
337                 mode_label,
338                 number(s3["mean_average_time_s"]),
339                 number(s4["mean_average_time_s"]),
340                 str(s3["cases"]),
341             )
342         )
343     table(
344         "stress_table.tex",
345         ("Geometry", "Error field", "Q3 mean (s)", "Q4 mean (s)", "$n$/Q"),
346         rows,
347         "llrrr",
348     )
349     rows = []
350     for key, s in final["sensitivity"].items():
351         problem, parameter, value = key.split("_")
352         label = (
353             r"$\lambda$=" + value + "$"
354             if parameter == "probe"
355             else "$s=" + f"{float(value):.0f}" + r"$\mathrm{m}$"
356         )
357         rows.append(
358             (
359                 problem,
360                 label,
361                 number(s["mean_average_time_s"]),
362                 number(s["p90_average_time_s"]),
363                 str(s["cases"]),
364             )
365         )
366     table(
367         "sensitivity_table.tex",
368         ("Problem", "Setting", "Mean (s)", "$p_{90}$ (s)", "$n$"),
369         rows,
370         "llrrr",
371     )
372     rows = []
373     for scenario, label in zip(SCENARIOS, SCENARIO_LABELS):
374         off, on = (
375             final["recovery_ablation"][f"{scenario}_recovery_{enabled}"]
376             for enabled in (False, True)
377         )
378         rows.append(
379             (
380                 label,
381                 number(off["mean_average_time_s"]),
382                 number(on["mean_average_time_s"]),
383                 number(off["mean_failed_clears"]),
384                 number(on["mean_failed_clears"]),
385                 str(on["cases"]),
386             )
387         )
388     table(
389         "recovery_table.tex",
390         ("Scenario", "$T/N$ off", "$T/N$ on", "Fails off", "Fails on", "Pairs"),
391         rows,
392     )

```

```

393 rows = [
394     (
395         "MEC reference",
396         str(geometry["mec_reference_cases"]),
397         f"{geometry['maximum_mec_radius_error_m']:.2e} m",
398     ),
399     (
400         "Bearing containment",
401         str(geometry["containment_cases"]),
402         f"{geometry['bearings_per_case']} readings/case",
403     ),
404     (
405         "Optical cover samples",
406         str(geometry["optical_interior_samples"]),
407         "Contained",
408     ),
409     (
410         "Discovery samples",
411         str(geometry["coverage_source_samples"]),
412         "Both geometries",
413     ),
414     (
415         "Triple-lens samples",
416         str(geometry["triple_lens_samples"]),
417         "Reception checked",
418     ),
419     ("Local HTTP fixture", "1", "Four checks passed"),
420 ]
421 table(
422     "verification_table.tex",
423     ("Numerical / local check", "Cases / points", "Observed result"),
424     rows,
425     "lrl",
426 )
427
428
429 def spatial_axis(ax, limit, unit="km"):
430     ax.set(
431         xlim=(-limit, limit),
432         ylim=(-limit, limit),
433         xlabel=f"x ({unit})",
434         ylabel=f"y ({unit})",
435         aspect="equal",
436     )
437     ax.grid(alpha=0.18)
438
439
440 def coverage_figure(geometry):
441     cert = geometry["coverage_certificate"]
442     omni, lattice = np.array(omni_waypoints()), np.array(directional_waypoints())
443     require(
444         len(omni) == cert["omni"]["waypoint_count"]
445         and len(lattice) == cert["directional"]["waypoint_count"],
446         "Coverage count mismatch",
447     )
448     fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 3.7))
449     fig.subplots_adjust(left=0.08, right=0.98, bottom=0.30, top=0.88, wspace=0.4)
450     for ax in axes:
451         spatial_axis(ax, 3.0)
452         ax.add_patch(
453             Circle(
454                 (0, 0), cert["arena_radius_m"] / 1000, fill=False, color="black", lw=1.3
455             )
456         )
457     for point in omni / 1000:
458         axes[0].add_patch(
459             Circle(

```

```

460         point,
461         cert["omni"]["cover_radius_m"] / 1000,
462         color=COLORS[0],
463         fill=False,
464         alpha=0.6,
465         lw=0.8,
466     )
467 )
468 axes[0].scatter(
469     *(omni / 1000).T, s=20, color=COLORS[0], label=f"{len(omni)} detectors"
470 )
471 inside = np.linalg.norm(lattice, axis=1) <= cert["arena_radius_m"]
472 require(
473     int((~inside).sum()) == cert["directional"]["outside_arena_count"],
474     "Outside-node count mismatch",
475 )
476 # Draw only nearest-neighbour lattice edges, including exterior nodes.
477 for i, p in enumerate(lattice):
478     for q in lattice[:i]:
479         if math.isclose(
480             math.dist(p, q), cert["directional"]["spacing_m"], abs_tol=1e-6
481         ):
482             axes[1].plot(
483                 [p[0] / 1000, q[0] / 1000],
484                 [p[1] / 1000, q[1] / 1000],
485                 color="0.78",
486                 lw=0.6,
487                 zorder=0,
488             )
489 axes[1].scatter(
490     *(lattice[inside] / 1000).T,
491     color=COLORS[0],
492     s=22,
493     label=f"Inside: {int(inside.sum())}",
494 )
495 axes[1].scatter(
496     *(lattice[~inside] / 1000).T,
497     color="#D55E00",
498     marker="^",
499     s=30,
500     label=f"Outside: {int((~inside).sum())}",
501 )
502 axes[0].set_title("Q3: 900 m covering disks")
503 axes[1].set_title(f"Q4: {len(lattice)} lattice nodes")
504 for ax in axes:
505     box = ax.get_position()
506     ax.legend(loc="lower center", bbox_to_anchor=(box.x0 + box.x1) / 2, 0.01),
507     bbox_transform=fig.transFigure, frameon=False)
508 save_figure(fig, "coverage.pdf")
509
510
511 def geometry_figure(geometry):
512     example = geometry["realizable_counterexample"]
513     polygon = initial_polygon()
514     for station, bearing in zip(example["stations"], example["bearings_deg"]):
515         polygon = clip_bearing(polygon, tuple(station), bearing, error_deg=1.0)
516     center, radius = enclosing_circle(polygon)
517     require(
518         math.isclose(radius, example["mec_radius_m"], abs_tol=1e-6),
519         "Triangle reconstruction mismatch",
520     )
521     a, b = max(
522         ((a, b) for a in polygon for b in polygon), key=lambda pair: math.dist(*pair)
523     )
524     midpoint = tuple((x + y) / 2 for x, y in zip(a, b))
525     fig, ax = plt.subplots(figsize=(WIDTH, 3.6), layout="constrained")
526     ax.add_patch(Polygon(polygon, facecolor="#56B4E9", alpha=0.35, edgecolor=COLORS[0]))

```

```

527 ax.plot(
528     *zip(*(polygon + [polygon[0]])),
529     color=COLORS[0],
530     label=f"Feasible region: D={example['diameter_m']:.0f} m",
531 )
532 ax.add_patch(
533     Circle(
534         midpoint,
535         example["diameter_m"] / 2,
536         fill=False,
537         color="#D55E00",
538         ls="--",
539         label=f"Diameter circle: {example['diameter_m'] / 2:.2f} m",
540     )
541 )
542 ax.add_patch(
543     Circle(
544         center,
545         radius,
546         fill=False,
547         color=COLORS[2],
548         lw=1.5,
549         label=f"MEC: {radius:.2f} m > 20 m",
550     )
551 )
552 ax.scatter(*center, color=COLORS[2], marker="+", s=60)
553 ax.set(
554     xlim=(-9, 49), ylim=(-22, 40), xlabel="x (m)", ylabel="y (m)", aspect="equal"
555 )
556 ax.legend(loc="center left", bbox_to_anchor=(1.0, 0.5), frameon=False)
557 ax.grid(alpha=0.2)
558 save_figure(fig, "geometry.pdf")
559
560
561 def second_point_figure(second):
562     require(
563         second["first_station"] == [0, 0] and second["first_reading_deg"] == 0,
564         "Q2 figure expects normalized illustrative case",
565     )
566     alpha = math.radians(BEARING_ERROR_DEG)
567     centers = np.array(
568         [
569             (0, 0),
570             (1000 * math.cos(alpha), 1000 * math.sin(alpha)),
571             (1000 * math.cos(alpha), -1000 * math.sin(alpha)),
572         ]
573     )
574     t = np.linspace(0, 1000, 1001)
575     lower = np.full_like(t, -np.inf)
576     upper = np.full_like(t, np.inf)
577     for x, y in centers:
578         h = np.sqrt(np.maximum(0, 1000**2 - (t - x)**2))
579         lower, upper = np.maximum(lower, y - h), np.minimum(upper, y + h)
580     mask = lower <= upper
581     fig, ax = plt.subplots(figsize=(WIDTH, 3.6), layout="constrained")
582     ax.fill_between(
583         t,
584         lower,
585         upper,
586         where=mask,
587         color="#56B4E9",
588         alpha=0.35,
589         label="Triple-disk reception lens",
590     )
591     ax.plot(t[mask], lower[mask], color=COLORS[0])
592     ax.plot(t[mask], upper[mask], color=COLORS[0])
593     candidates = second["candidates"]

```

```

594     ax.scatter(
595         [c["t_m"] for c in candidates],
596         [c["b_m"] for c in candidates],
597         color="0.55",
598         marker=".",
599         s=12,
600         label="Finite candidate grid",
601     )
602     for key, label, marker, color in (
603         ("best_sampled_geometry", "Sampled choice", "*", COLORS[2]),
604         ("collinear_control", "Collinear control", "s", "#D55E00"),
605     ):
606         c = second[key]
607         require(
608             all(math.dist(c["point"], p) <= 1000 + 1e-6 for p in centers),
609             "Candidate outside lens",
610         )
611         ax.scatter(
612             c["t_m"],
613             c["b_m"],
614             s=90,
615             marker=marker,
616             color=color,
617             label=f"{label}\nSampled max R={c['sampled_worst_mec_radius_m']:.2f} m",
618         )
619     ax.scatter(0, 0, color="black", s=20)
620     ax.axhline(0, color="0.6", lw=0.6, ls="--")
621     ax.set(
622         xlabel="Along first bearing t (m)",
623         ylabel="Transverse b (m)",
624         xlim=(-40, 1040),
625         ylim=(-920, 920),
626     )
627     ax.legend(loc="center left", bbox_to_anchor=(1.0, 0.5), frameon=False)
628     save_figure(fig, "q2_lens.pdf")
629
630
631 def performance_figure(main):
632     fig, axes = plt.subplots(
633         1, 2, figsize=(WIDTH, 3.7), layout="constrained", sharey=True
634     )
635     for problem, ax in zip((3, 4), axes):
636         for policy, color, style in zip(POLICIES, COLORS, ("-", "--", "-.")):
637             rows = main[f"Q{problem}_{policy}"]
638             values = sorted(r["average_time_s"] for r in rows)
639             ax.step(
640                 values,
641                 np.arange(1, len(values) + 1) / len(values),
642                 where="post",
643                 color=color,
644                 ls=style,
645                 label=policy.capitalize(),
646             )
647             ax.set(
648                 title=f"Q{problem}: {len(rows)} matched worlds",
649                 xlabel="Per-world T/N (s)",
650                 ylim=(0, 1.02),
651             )
652             ax.grid(alpha=0.2)
653             ax.legend(loc="lower right", frameon=False)
654             axes[0].set_ylabel("Empirical cumulative fraction")
655             save_figure(fig, "performance.pdf")
656
657
658 def replay_trajectories():
659     traces = []
660     for problem in (3, 4):

```

```

661     env = LocalSimulator(
662         20260910, problem=problem, error_mode="smooth", scenario="random"
663     )
664     planner = Planner(
665         env, problem=problem, strategy="enhanced", loss_recovery=True
666     )
667     result = planner.run()
668     summary = env.summary() # Evaluation after termination; never supplied to Planner.
669     require(summary["fraction_cleared"] == 1, "Trajectory replay incomplete")
670     require(result["certificate_complete"] is True, "Trajectory certificate incomplete")
671     points, clears = [[0.0, 0.0]], []
672     for event in env.log:
673         request, response = event["request"], event["response"]
674         if not response["accepted"] or "position" not in request:
675             continue
676         point = [request["position"]["x"], request["position"]["y"]]
677         if point != points[-1]:
678             points.append(point)
679         if response.get("clear_result") == "success":
680             clears.append(
681                 {
682                     "channel": request["channel"],
683                     "point": point,
684                     "virtual_time_s": response["virtual_time_s"],
685                 }
686             )
687     require(
688         len(clears) == summary["n_cleared"] == result["n_cleared"],
689         "Replay clear count mismatch",
690     )
691     trace = {
692         "evidence": "local_synthetic_not_official",
693         "seed": 20260910,
694         "problem": problem,
695         "strategy": "enhanced",
696         "scenario": "random",
697         "error_mode": "smooth",
698         "requested_loss_recovery": True,
699         "effective_loss_recovery": problem == 4,
700         "point_semantics": "Accepted robot coordinates; successful clear within 20 m, not source
701         coordinates",
702         "path": points,
703         "successful_clears": clears,
704         "planner": result,
705         "post_exit_summary": summary,
706         "observable_log": env.log,
707     }
708     write_json(f"trajectory_q{problem}.json", trace)
709     traces.append(trace)
710     write_json(
711         "trajectory_summary.json",
712         [
713             {k: t[k] for k in ("problem", "seed", "planner", "post_exit_summary")}
714             for t in traces
715         ],
716     )
717     fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 3.8), layout="constrained")
718     limit = max(
719         2.0, max(abs(v) / 1000 for t in traces for p in t["path"] for v in p) + 0.15
720     )
721     for ax, trace in zip(axes, traces):
722         path = np.array(trace["path"]) / 1000
723         clear = np.array([c["point"] for c in trace["successful_clears"]]) / 1000
724         spatial_axis(ax, limit)
725         ax.add_patch(Circle((0, 0), 1.8, color="0.5", fill=False, ls="--"))
726         ax.plot(*path.T, color=COLORS[0], lw=0.8, label="Accepted path")
727         ax.scatter(

```

```

727         *clear.T,
728         color="#D55E00",
729         marker="x",
730         s=26,
731         label="Successful clear",
732         zorder=3,
733     )
734     ax.scatter(0, 0, color="black", marker="s", s=22, label="Start", zorder=4)
735     ax.set_title(
736         f"Q{trace['problem']}: {len(clear)} clears\N/T/N={trace['post_exit_summary']['average_time_s']:.2f}
s"
737     )
738     handles, labels = axes[0].get_legend_handles_labels()
739     fig.legend(handles, labels, loc="outside lower center", ncol=3, frameon=False)
740     save_figure(fig, "trajectories.pdf")
741     return traces
742
743
744 def recovery_figure(final):
745     fig, axes = plt.subplots(
746         2, 1, figsize=(WIDTH, 5.2), layout="constrained", sharex=True
747     )
748     x = np.arange(len(SCENARIOS))
749     for enabled, offset, color, hatch, label in (
750         (False, -0.18, COLORS[0], "/", "Recovery off"),
751         (True, 0.18, COLORS[2], "", "Recovery on"),
752     ):
753         summaries = [
754             final["recovery_ablation"][f"{s}_recovery_{enabled}"] for s in SCENARIOS
755         ]
756         for ax, metric in zip(axes, ("mean_average_time_s", "mean_failed_clears")):
757             ax.bar(
758                 x + offset,
759                 [s[metric] for s in summaries],
760                 width=0.34,
761                 color=color,
762                 hatch=hatch,
763                 label=label,
764             )
765             ax.grid(axis="y", alpha=0.2)
766             ax.set_axisbelow(True)
767         axes[0].set_ylabel("Mean T/N (s)")
768         axes[1].set_ylabel("Mean failed clears")
769         axes[1].set_xticks(x, SCENARIO_LABELS, rotation=12, ha="right")
770         axes[0].legend(frameon=False, ncol=2, loc="upper left")
771         counts = sorted({v["cases"] for v in final["recovery_ablation"].values()})
772         require(len(counts) == 1, "Recovery counts differ")
773         axes[0].set_title(f"Q4: {counts[0]} matched worlds per scenario")
774         save_figure(fig, "recovery.pdf")
775
776
777 def main():
778     plt.rcParams.update(
779         {
780             "font.family": "DejaVu Sans",
781             "font.size": 10,
782             "axes.titlesize": 10,
783             "axes.labelsize": 10,
784             "xtick.labelsize": 10,
785             "ytick.labelsize": 10,
786             "legend.fontsize": 10,
787             "pdf.fonttype": 42,
788             "savefig.transparent": False,
789             "figure.facecolor": "white",
790         }
791     )
792     final, geometry, http, second, main_rows, _recovery_rows, hashes = load_inputs()

```

```

793 generate_tables(final, geometry, http)
794 coverage_figure(geometry)
795 geometry_figure(geometry)
796 second_point_figure(second)
797 performance_figure(main_rows)
798 traces = replay_trajectories()
799 recovery_figure(final)
800 for name in (
801     "make_assets.py",
802     "planner.py",
803     "environment.py",
804     "geometry.py",
805     "coverage.py",
806     "second_point.py",
807 ):
808     hashes[name] = hashlib.sha256((ROOT / name).read_bytes()).hexdigest()
809 enhancement_input = ROOT / "results" / "enhancement_experiments.json"
810 if enhancement_input.exists():
811     hashes["results/enhancement_experiments.json"] = hashlib.sha256(enhancement_input.read_bytes()).
812     hexdigest()
813 ring_sweep_input = ROOT / "results" / "ring_sweep.json"
814 if ring_sweep_input.exists():
815     hashes["results/ring_sweep.json"] = hashlib.sha256(ring_sweep_input.read_bytes()).hexdigest()
816 write_json(
817     "metadata.json",
818     {
819         "evidence": "local_synthetic_not_official",
820         "official_calls": 0,
821         "asset_set": "base_comparison_with_enhanced_trajectory",
822         "input_sha256": hashes,
823         "matplotlib_version": matplotlib.__version__,
824         "figures": [
825             "coverage.pdf", "geometry.pdf", "q2_lens.pdf", "performance.pdf",
826             "trajectories.pdf", "recovery.pdf", "enhancement_geometry.pdf",
827             "enhancement_performance.pdf",
828         ],
829         "tables": [
830             "comparison_table.tex", "stress_table.tex", "sensitivity_table.tex",
831             "recovery_table.tex", "verification_table.tex", "enhancement_table.tex",
832             "enhancement_costs.tex", "enhancement_stress.tex", "candidate_table.tex",
833             "coverage_limits_table.tex",
834         ],
835         "enhancement_metadata": "enhancement_metadata.json",
836         "layout": {
837             "width_cm": 16,
838             "minimum_font_pt": 10,
839             "backend": "Agg",
840             "format": "vector PDF",
841         },
842         "metric": "Mean, median and nearest-rank p90 of per-world T/N; never pooled sum(T)/sum(N). Gains
843         are reduction of group means; ECDFs use the same seeds per policy.",
844         "denominators": {
845             "all_runs_including_paired_repeats": final["total_runs"],
846             "main_worlds_per_problem": final["paired_random_cases_per_problem"],
847             "trajectory_replays_additional_to_experiments": len(traces),
848         },
849         "seed_ranges": {
850             "main": [600000, 600000 + final["paired_random_cases_per_problem"] - 1],
851             "stress": [700000, 700019],
852             "probe_sensitivity": [800000, 800029],
853             "spacing_sensitivity": [900000, 900029],
854             "recovery": [610000, 610099],
855             "trajectory": 20260910,
856         },
857         "parameters": {
858             "strategy": "enhanced",
859             "anchor_policy_Q3": "nearest",

```

```

858         "anchor_policy_Q4": "rolling",
859         "ring_radius_Q3_m": 1200,
860         "spacing_Q4_m": 990,
861         "probe_scale": 0.22,
862         "max_probes": 6,
863         "loss_recovery_Q4": True,
864         "bearing_halfwidth_deg": BEARING_ERROR_DEG,
865         "Q2_source_samples": second["source_samples"],
866         "Q2_second_errors_deg": second["second_error_values_deg"],
867         "Q2_optimality": "Finite candidate/sample comparison only, no global or continuous minimax
certificate",
868     },
869     "distributions": {
870         "random": "N uniform integer 10..16; distinct channels uniformly sampled; positions iid area-
uniform radius-1800 disk; radii uniform [1000,1500]; orientations uniform [0,2pi). Q3 all omni; Q4 uniform
directional count 1..N-1 and uniform subset.",
871         "minimum_radius": "Random geometry, radio radius exactly 1000 m.",
872         "outward_boundary": "16 equally spaced boundary sources with seeded rotation, radius 1000; Q4
15 outward directional and one omni.",
873         "clustered": "16 area-uniform sources in radius-35 disk with center at radius1650; radio
radius1000.",
874     },
875     "error_fields": {
876         "smooth": "Deterministic bounded two-sinusoid field.",
877         "extreme": "Coordinate/seed/channel hash selecting +/-1 degree.",
878         "iid_location": "Coordinate/seed/channel hash uniform [-1,1], not independent repeat
measurements.",
879     },
880     "trajectory_evidence": "trajectory_q3.json and trajectory_q4.json preserve complete observable logs
, accepted paths, successful robot clearing locations and post-exit aggregate summaries. No hidden source
coordinates inspected.",
881     "determinism": "PDF creation timestamps suppressed; no random plotting jitter. Raw real-duration/
timing observations may vary between replays; figures use virtual time only.",
882 },
883 )
884 print(
885     "Generated six local-evidence PDFs, five tabular fragments, numbers.tex, metadata.json and two complete
observable replay logs."
886 )
887
888
889 if __name__ == "__main__":
890     main()

```

Listing 15: PDF 逐页渲染与结构审计

```

1  """Render every PDF page and record structural/layout evidence for visual review."""
2
3  from __future__ import annotations
4
5  import argparse
6  import hashlib
7  import json
8  import re
9  from pathlib import Path
10
11  import pymupdf
12  from PIL import Image, ImageDraw
13
14
15  def audit(pdf_path: Path, prefix: str, dpi: int = 120):
16      directory = Path(__file__).resolve().parent / "report" / "visual"
17      directory.mkdir(parents=True, exist_ok=True)
18      document = pymupdf.open(pdf_path)
19      pages, outside, thin, replacements = [], [], [], 0
20      for number, page in enumerate(document, 1):
21          text = page.get_text()

```

```

22     replacements += text.count("\ufffd")
23     if len(text.strip()) < 15:
24         thin.append(number)
25     spans = [
26         span
27         for block in page.get_text("dict")["blocks"]
28         if "lines" in block
29         for line in block["lines"]
30         for span in line["spans"]
31         if span["text"].strip()
32     ]
33     for span in spans:
34         rect = pymupdf.Rect(span["bbox"])
35         if (
36             rect.x0 < -0.5
37             or rect.y0 < -0.5
38             or rect.x1 > page.rect.width + 0.5
39             or rect.y1 > page.rect.height + 0.5
40         ):
41             outside.append(
42                 {"page": number, "text": span["text"], "bbox": list(rect)}
43             )
44     image = directory / f"{prefix}_{number:03d}.png"
45     page.get_pixmap(dpi=dpi).save(image)
46     pages.append(
47         {
48             "page": number,
49             "characters": len(text),
50             "image": image.name,
51             "smallest_text_pt": min((s["size"] for s in spans), default=None),
52             "largest_text_pt": max((s["size"] for s in spans), default=None),
53         }
54     )
55     sheets = []
56     for start in range(0, len(pages), 6):
57         width, height, gutter = 600, 880, 15
58         sheet = Image.new(
59             "RGB", (3 * width + 4 * gutter, 2 * height + 3 * gutter), "#d4d4d4"
60         )
61         draw = ImageDraw.Draw(sheet)
62         for offset, record in enumerate(pages[start : start + 6]):
63             picture = Image.open(directory / record["image"]).convert("RGB")
64             picture.thumbnail((width, height - 28), Image.Resampling.LANCZOS)
65             x = gutter + (offset % 3) * (width + gutter)
66             y = gutter + (offset // 3) * (height + gutter)
67             draw.text((x + 8, y + 4), f"Page {record['page']}", fill="black")
68             sheet.paste(picture, (x + (width - picture.width) // 2, y + 27))
69             name = f"{prefix}_sheet_{start // 6 + 1:02d}.png"
70             sheet.save(directory / name)
71             sheets.append(name)
72     endbody = None
73     auxiliary = pdf_path.with_suffix(".aux")
74     if auxiliary.exists():
75         match = re.search(
76             r"\newlabel{endbody}\{\{[~]*\}\{(\d+)\}", auxiliary.read_text()
77         )
78         if match:
79             endbody = int(match[1])
80     result = {
81         "pdf": str(pdf_path),
82         "sha256": hashlib.sha256(pdf_path.read_bytes()).hexdigest(),
83         "pages": len(document),
84         "size_mib": pdf_path.stat().st_size / 2**20,
85         "body_end_page": endbody,
86         "body_pages_excluding_abstract": endbody - 1 if endbody else None,
87         "abstract_keywords_on_first_page": "关键词" in document[0].get_text(),
88         "out_of_page_spans": outside,

```

```

89     "replacement_characters": replacements,
90     "near_empty_pages": thin,
91     "rendered_all_pages": True,
92     "dpi": dpi,
93     "contact_sheets": sheets,
94     "page_records": pages,
95     "visual_review_status": "rendered_for_inspection_not_automatically_approved",
96 }
97 output = directory / f"{prefix}_audit.json"
98 output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n")
99 if outside or replacements:
100     raise RuntimeError(f"PDF has structural text issues; inspect {output}")
101 if prefix == "paper" and (
102     endbody is None
103     or endbody - 1 > 30
104     or not result["abstract_keywords_on_first_page"]
105 ):
106     raise RuntimeError(
107         f"Paper abstract/body page requirement failed; inspect {output}"
108     )
109 print(
110     json.dumps(
111         {k: v for k, v in result.items() if k != "page_records"},
112         ensure_ascii=False,
113         indent=2,
114     )
115 )
116 return result
117
118
119 if __name__ == "__main__":
120     parser = argparse.ArgumentParser()
121     parser.add_argument("--pdf", type=Path, default=Path("report/paper.pdf"))
122     parser.add_argument("--prefix", default="paper")
123     parser.add_argument("--dpi", type=int, default=120)
124     args = parser.parse_args()
125     audit(args.pdf, args.prefix, args.dpi)

```

Listing 16: 匿名材料打包与解压复现验证

```

1  """Build an anonymous, reproducible RESEARCH archive; official runs are deferred."""
2
3  from __future__ import annotations
4
5  import argparse
6  import hashlib
7  import importlib.metadata
8  import json
9  import os
10 import re
11 import shutil
12 import subprocess
13 import sys
14 import tempfile
15 import time
16 import zipfile
17 from pathlib import Path, PurePosixPath
18
19 import pymupdf
20
21 ROOT = Path(__file__).resolve().parent
22 SOURCES = (
23     "geometry.py",
24     "coverage.py",
25     "planner.py",
26     "environment.py",
27     "client.py",

```

```

28     "run.py",
29     "official_run.py",
30     "intersection.py",
31     "second_point.py",
32     "experiments.py",
33     "compare_recovery.py",
34     "compare_routes.py",
35     "compare_probes.py",
36     "verify_geometry.py",
37     "verify_coverage_limits.py",
38     "verify_refinements.py",
39     "verify_terminal.py",
40     "verify_http.py",
41     "make_enhancement_assets.py",
42     "make_assets.py",
43     "audit_pdf.py",
44     "package.py",
45     "active_policy_candidate.py",
46     "route_policy_candidate.py",
47     "active_policy_notes.txt",
48     "route_policy_notes.txt",
49     "enhancements.py",
50     "ring_sweep.py",
51     "develop_anchors.py",
52     "theory_additions.txt",
53     "paper_expansion.txt",
54     "coverage_notes.txt",
55     "coverage_limits.tex",
56     "requirements.txt",
57     "reproduce.txt",
58     "official_status.json",
59 )
60 RESULTS = (
61     "final_experiments.json",
62     "geometry_verification.json",
63     "intersection_verification.json",
64     "second_point_example.json",
65     "recovery_development.json",
66     "route_candidate_comparison.json",
67     "adaptive_candidate_comparison.json",
68     "terminal_verification.json",
69     "ring_sweep.json",
70     "enhancement_experiments.json",
71     "anchor_development.json",
72     "enhancement_review.json",
73     "coverage_limits.json",
74     "refinement_verification.json",
75     "http_verification.json",
76     "http_deadline_verification.json",
77     "review_resolutions.json",
78     "operation_bound.json",
79     "paper_review.json",
80     "visual_review.json",
81     "final_review.json",
82     "environment_manifest.json",
83 )
84
85
86 def verify_archive(archive: Path):
87     records = []
88     with tempfile.TemporaryDirectory(prefix="b-reproduce-") as directory:
89         extracted = Path(directory)
90         with zipfile.ZipFile(archive) as bundle:
91             for item in bundle.infolist():
92                 relative = PurePosixPath(item.filename)
93                 if relative.is_absolute() or ".." in relative.parts:
94                     raise ValueError("Unsafe member in generated archive")

```

```

95         bundle.extractall(extracted)
96     manifest = json.loads((extracted / "MANIFEST.json").read_text())
97     for name, information in manifest["files"].items():
98         raw = (extracted / name).read_bytes()
99         assert hashlib.sha256(raw).hexdigest() == information["sha256"], name
100     baseline = json.loads(
101         (extracted / "results/final_experiments.json").read_text()
102     )
103     enhanced_baseline = json.loads((extracted / "results/enhancement_experiments.json").read_text())
104     ring_baseline = json.loads((extracted / "results/ring_sweep.json").read_text())
105     commands = [
106         [sys.executable, "verify_geometry.py"],
107         [sys.executable, "intersection.py"],
108         [sys.executable, "second_point.py"],
109         [
110             sys.executable,
111             "experiments.py",
112             "--cases",
113             "500",
114             "--output",
115             "results/final_experiments.json",
116         ],
117         [sys.executable, "enhancements.py"],
118         [sys.executable, "ring_sweep.py"],
119         [sys.executable, "verify_terminal.py"],
120         [sys.executable, "verify_refinements.py"],
121         [sys.executable, "verify_coverage_limits.py"],
122         [sys.executable, "compare_routes.py"],
123         [sys.executable, "compare_probes.py"],
124         [sys.executable, "make_assets.py"],
125         [sys.executable, "make_enhancement_assets.py"],
126         [
127             "latexmk",
128             "-xelatex",
129             "-interaction=nonstopmode",
130             "-halt-on-error",
131             "paper.tex",
132         ],
133         [
134             "latexmk",
135             "-xelatex",
136             "-interaction=nonstopmode",
137             "-halt-on-error",
138             "ai_usage.tex",
139         ],
140     ]
141     for command in commands:
142         started = time.monotonic()
143         cwd = extracted / "report" if command[0] == "latexmk" else extracted
144         completed = subprocess.run(
145             command,
146             cwd=cwd,
147             text=True,
148             stdout=subprocess.PIPE,
149             stderr=subprocess.STDOUT,
150             timeout=240,
151             check=False,
152             env={**os.environ, "MPLBACKEND": "Agg"},
153         )
154         records.append(
155             {
156                 "command": [Path(command[0]).name, *command[1:]],
157                 "returncode": completed.returncode,
158                 "seconds": time.monotonic() - started,
159             }
160         )
161     if completed.returncode:

```

```

162         raise RuntimeError(
163             f"Extracted archive command failed: {command}\n{completed.stdout[-12000:]}"
164         )
165     replay = json.loads((extracted / "results/final_experiments.json").read_text())
166     assert baseline["total_runs"] == replay["total_runs"] == 4430
167     assert len(baseline["runs"]) == len(replay["runs"])
168     for old, new in zip(baseline["runs"], replay["runs"]):
169         for key in old:
170             if key != "real_duration_s":
171                 assert old[key] == new[key], (old["seed"], key, old[key], new[key])
172     enhanced_replay = json.loads((extracted / "results/enhancement_experiments.json").read_text())
173     assert enhanced_baseline["total_runs"] == enhanced_replay["total_runs"] == 3080
174     for old, new in zip(enhanced_baseline["runs"], enhanced_replay["runs"], strict=True):
175         assert {k: v for k, v in old.items() if k != "real_duration_s"} == {
176             k: v for k, v in new.items() if k != "real_duration_s"}
177     ring_replay = json.loads((extracted / "results/ring_sweep.json").read_text())
178     assert ring_baseline == ring_replay
179     for filename in ("paper", "ai_usage"):
180         log = (extracted / "report" / f"{filename}.log").read_text(errors="replace")
181         assert "Missing character:" not in log and "Overfull" not in log
182         assert (
183             "undefined references" not in log and "undefined citations" not in log
184         )
185         assert len(pymupdf.open(extracted / "report" / f"{filename}.pdf")) > 0
186     return {
187         "status": "passed",
188         "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
189         "manifest_hashes_verified": True,
190         "all_4430_behavioral_runs_reproduced_exactly": True,
191         "all_3080_enhancement_runs_reproduced_exactly": True,
192         "ring_sweep_reproduced_exactly": True,
193         "excluded_nondeterminism": "wall-clock real_duration_s only",
194         "commands": records,
195     }
196
197
198 def main():
199     parser = argparse.ArgumentParser()
200     parser.add_argument("--verify", action="store_true")
201     args = parser.parse_args()
202     for metadata_name in ("metadata.json", "enhancement_metadata.json"):
203         metadata = json.loads((ROOT / "report/generated" / metadata_name).read_text())
204         for name, expected in metadata["input_sha256"].items():
205             actual = hashlib.sha256((ROOT / name).read_bytes()).hexdigest()
206             if actual != expected:
207                 raise ValueError(f"Stale generated asset input: {name}")
208     for prefix, name in (("paper", "paper"), ("ai", "ai_usage")):
209         audit = json.loads((ROOT / "report/visual" / f"{prefix}_audit.json").read_text())
210         actual = hashlib.sha256((ROOT / "report" / f"{name}.pdf").read_bytes()).hexdigest()
211         if audit["sha256"] != actual or audit["out_of_page_spans"] or audit["replacement_characters"]:
212             raise ValueError(f"Missing or stale PDF audit: {name}")
213         if prefix == "paper" and not (audit["abstract_keywords_on_first_page"]
214                                     and 0 < audit["body_pages_excluding_abstract"] <= 30):
215             raise ValueError("Paper abstract/body page rule violated")
216     environment = {
217         "python": sys.version,
218         "environment": "conda py314",
219         "packages": {
220             name: importlib.metadata.version(name)
221             for name in ("numpy", "scipy", "matplotlib", "pymupdf")
222         },
223     }
224     (ROOT / "results/environment_manifest.json").write_text(
225         json.dumps(environment, indent=2) + "\n"
226     )
227     files = {name: ROOT / name for name in SOURCES}
228     files.update({"results/" + name: ROOT / "results" / name for name in RESULTS})

```

```

229 files["AI工具使用详情.pdf"] = ROOT / "report/ai_usage.pdf"
230 for path in (ROOT / "report").iterdir():
231     if path.suffix in (".tex", ".pdf"):
232         files[path.relative_to(ROOT).as_posix()] = path
233 for folder in ("generated", "figures"):
234     for path in (ROOT / "report" / folder).iterdir():
235         if path.is_file() and not path.name.startswith("."):
236             files[path.relative_to(ROOT).as_posix()] = path
237 for filename in ("paper_audit.json", "ai_audit.json"):
238     path = ROOT / "report" / "visual" / filename
239     files[path.relative_to(ROOT).as_posix()] = path
240 missing = [name for name, path in files.items() if not path.is_file()]
241 if missing:
242     raise RuntimeError(f"Deliverable proof is missing: {missing}")
243 anonymous_pattern = re.compile(r"/Users/[A-Za-z0-9._-]+/")
244 for name, path in files.items():
245     if path.suffix in (".py", ".txt", ".tex", ".json") and anonymous_pattern.search(
246         path.read_text(encoding="utf-8")
247     ):
248         raise ValueError(f"Personal absolute path in anonymous material: {name}")
249 if path.suffix == ".pdf":
250     document = pymupdf.open(path)
251     if anonymous_pattern.search(
252         "\n".join(page.get_text() for page in document)
253     ):
254         raise ValueError(f"Personal absolute path in PDF: {name}")
255 manifest = {
256     "status": "VERIFIED_RESEARCH_VERSION_OFFICIAL_TESTS_DEFERRED",
257     "not_ready_for_contest_submission": True,
258     "deferred_by_user": "Official simulator practice/formal runs and six encrypted logs",
259     "human_review": "No actual contestant review record; AI review is not human review",
260     "python_environment": "conda py314",
261     "python_version": sys.version.split()[0],
262     "files": {
263         name: {
264             "sha256": hashlib.sha256(path.read_bytes()).hexdigest(),
265             "bytes": path.stat().st_size,
266         }
267         for name, path in sorted(files.items())
268     },
269 }
270 destination = ROOT / "delivery"
271 destination.mkdir(exist_ok=True)
272 archive = destination / "B题研究支撑材料.zip"
273 with zipfile.ZipFile(
274     archive, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9
275 ) as bundle:
276     for name, path in sorted(files.items()):
277         bundle.write(path, name)
278         bundle.writestr(
279             "MANIFEST.json", json.dumps(manifest, ensure_ascii=False, indent=2) + "\n"
280         )
281 with zipfile.ZipFile(archive) as bundle:
282     assert bundle.testzip() is None
283 shutil.copy2(ROOT / "report/paper.pdf", destination / "B题研究论文.pdf")
284 shutil.copy2(ROOT / "report/ai_usage.pdf", destination / "AI工具使用详情.pdf")
285 for path in (
286     archive,
287     destination / "B题研究论文.pdf",
288     destination / "AI工具使用详情.pdf",
289 ):
290     assert path.stat().st_size <= 20 * 2**20, path.name
291 status = {
292     "status": manifest["status"],
293     "official_tests": "DEFERRED_BY_USER",
294     "human_review": "OUTSTANDING",
295     "source_files": len(files),

```

```

296     "archive_bytes": archive.stat().st_size,
297     "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
298 }
299 if args.verify:
300     proof = verify_archive(archive)
301     (destination / "复现验证.json").write_text(
302         json.dumps(proof, ensure_ascii=False, indent=2) + "\n"
303     )
304     status["extracted_archive_reproduction"] = "passed"
305     (destination / "交付状态.json").write_text(
306         json.dumps(status, ensure_ascii=False, indent=2) + "\n"
307     )
308     print(json.dumps(status, ensure_ascii=False, indent=2))
309
310
311 if __name__ == "__main__":
312     main()

```

Listing 17: 区间评分自适应测点候选

```

1  """Unexecuted research candidate: observation-only, bounded active probing.
2
3  No action or belief update occurs here. ``None`` means use the unchanged
4  localizer/optical cover; a selected point is NOT a clearing certificate.
5  """
6
7  from __future__ import annotations
8
9  from math import acos, cos, dist, hypot, isfinite, nextafter, pi, radians, sin
10
11  from geometry import (
12      BEARING_ERROR_DEG,
13      MAX_RADIO_RADIUS,
14      Point,
15      _clip_halfplane,
16      _padding,
17      enclosing_circle,
18  )
19  from second_point import safe_candidates
20
21
22  def _bearing_bound(polygon, point, radius, bins, error_deg):
23      """Upper-bound every positive-bearing posterior using overlapping wedges.
24
25      Each angular bin is enlarged by the measurement error; unlike clip_bearing,
26      the scoring relaxation has NO projected range cap. A bounding-box circle
27      encloses each relaxed polygon. The original MEC is also an enclosing disk.
28      """
29      half_bin = 180.0 / bins
30      width = radians(error_deg + half_bin)
31      scale = max(
32          MAX_RADIO_RADIUS,
33          abs(point[0]),
34          abs(point[1]),
35          *(max(abs(x), abs(y)) for x, y in polygon),
36      )
37      # A padded narrow wedge is a translated exact wedge. Its apex shift is
38      # epsilon/sin(alpha); this larger pad also covers that displacement.
39      padding = 4.0 * _padding(scale) / sin(radians(error_deg))
40      worst, nonempty = 0.0, 0
41      for index in range(bins):
42          angle = 2.0 * pi * index / bins
43          low, high = angle - width, angle + width
44          outer = _clip_halfplane(
45              polygon, point, (sin(low), -cos(low)), 0.0, padding
46          )
47          outer = _clip_halfplane(

```

```

48         outer, point, (-sin(high), cos(high)), 0.0, padding
49     )
50     if not outer:
51         continue
52     nonempty += 1
53     xmin = min(p[0] for p in outer)
54     xmax = max(p[0] for p in outer)
55     ymin = min(p[1] for p in outer)
56     ymax = max(p[1] for p in outer)
57     box_radius = nextafter(
58         0.5 * hypot(xmax - xmin, ymax - ymin) + padding, float("inf")
59     )
60     worst = max(worst, min(radius, box_radius))
61 # An unexpected empty partition must not look like perfect information.
62 return (worst if nonempty else radius), nonempty
63
64
65 def _loss_risk(samples, readings, point, problem):
66     """Geometric risk index, NOT an identified no-signal probability."""
67     total = 0.0
68     station = readings[-1][0]
69     for source in samples:
70         query_distance = dist(point, source)
71         # A positive station implies range >= its distance, even in Q4.
72         lower_range = max(1000.0, *(dist(s, source) for s, _ in readings))
73         if query_distance > lower_range:
74             total += 1.0
75             continue
76         if problem == 3:
77             continue
78         old_distance = dist(station, source)
79         if old_distance <= 1e-9 or query_distance <= 1e-9:
80             # Direction is undefined here; do not manufacture guaranteed signal.
81             total += 1.0
82             continue
83         cosine = sum(
84             (station[i] - source[i]) * (point[i] - source[i]) for i in (0, 1)
85         ) / (old_distance * query_distance)
86         total += acos(max(-1.0, min(1.0, cosine))) / pi
87     return total / len(samples)
88
89
90 def score_probes(
91     polygon,
92     readings,
93     current_position,
94     problem,
95     *,
96     probes_remaining=1,
97     rejected_points=(),
98     bearing_bins=72,
99     error_deg=BEARING_ERROR_DEG,
100     probe_scale=0.22,
101     movement_weight=0.15,
102     loss_weight=1.0,
103     include_lens=True,
104 ):
105     """Return a deterministic score table; consume only public observations.
106
107     polygon: nonempty ordered convex conservative vertices, as Track.polygon.
108     readings: positive (station, svd_deg) pairs for ONE known channel only.
109     rejected_points: previous unsuccessful probes/clears; excludes locations,
110         never source hypotheses. The caller retains all underlying history.
111     probes_remaining: caller-owned remaining bound; this function cannot debit it.
112
113     Legal coordinates are finite and within +/-2,000,000 m per protocol, NOT
114     restricted to the radius-1800 source arena. Scores are dimensionless.

```

```

115 Empty output signals fallback. Malformed/contradictory inputs raise, rather
116 than resetting a belief or silently substituting a made-up polygon.
117 """
118 if problem not in (3, 4):
119     raise ValueError("Only Q3 and Q4 localization is supported")
120 if not isinstance(probes_remaining, int) or probes_remaining < 0:
121     raise ValueError("probes_remaining must be a nonnegative integer")
122 if not isinstance(bearing_bins, int) or bearing_bins < 4:
123     raise ValueError("bearing_bins must be an integer >= 4")
124 if not isfinite(error_deg) or not BEARING_ERROR_DEG <= error_deg < 90:
125     raise ValueError("error_deg must include the public rounding allowance")
126 if error_deg + 180.0 / bearing_bins >= 90.0:
127     raise ValueError("Expanded bearing bins must be narrower than 180 degrees")
128 if not isfinite(probe_scale) or not 0 < probe_scale <= 1:
129     raise ValueError("probe_scale must lie in (0, 1]")
130 if any(not isfinite(w) or w < 0 for w in (movement_weight, loss_weight)):
131     raise ValueError("Score weights must be finite and nonnegative")
132 poly = [tuple(p) for p in polygon]
133 history = [(tuple(s), float(a)) for s, a in readings]
134 position = tuple(current_position)
135 rejected = [tuple(p) for p in rejected_points]
136 points = poly + [position] + rejected + [s for s, _ in history]
137 if any(len(p) != 2 or not all(isfinite(v) for v in p) for p in points):
138     raise ValueError("All points must contain two finite coordinates")
139 if any(not isfinite(a) for _, a in history):
140     raise ValueError("Readings must have finite public bearing angles")
141 center, radius = enclosing_circle(poly) # Empty belief must raise.
142 if not history or probes_remaining == 0 or radius <= 19.9:
143     return []
144
145 station, bearing = history[-1]
146 dx, dy = center[0] - station[0], center[1] - station[1]
147 length = hypot(dx, dy)
148 axis = (dx / length, dy / length) if length > 1e-8 else (
149     cos(radians(bearing)), sin(radians(bearing))
150 )
151 normal = (-axis[1], axis[0])
152 offset = min(120.0, max(25.0, probe_scale * radius))
153 candidates = [("mec_center", center), ("positive_station", station)]
154 for name, direction in (("normal", normal), ("axis", axis)):
155     for sign in (-1, 1):
156         candidates.append((
157             f"mec_{name}_{sign:+d}",
158             tuple(center[i] + sign * offset * direction[i] for i in (0, 1)),
159         ))
160 if include_lens:
161     candidates.extend(
162         (f"q2_lens_{index}", tuple(row["point"]))
163         for index, row in enumerate(safe_candidates(station, bearing, error_deg))
164     )
165
166 samples = list(dict.fromkeys(poly + [center]))
167 exclusions = [s for s, _ in history] + rejected
168 table, seen = [], set()
169 for label, point in candidates:
170     if point in seen:
171         continue
172     seen.add(point)
173     reason = None
174     if any(not isfinite(v) or abs(v) > 2_000_000 for v in point):
175         reason = "outside_protocol_coordinates"
176     elif any(dist(point, old) <= 0.5 for old in exclusions):
177         reason = "previous_observation_or_rejected_location"
178     row = {
179         "candidate": label,
180         "point": point,
181         "eligible": reason is None,

```

```

182         "rejection": reason,
183         "score": None,
184     }
185     if reason is None:
186         bound, bins_used = _bearing_bound(
187             poly, point, radius, bearing_bins, error_deg
188         )
189         movement = dist(position, point)
190         risk = _loss_risk(samples, history, point, problem)
191         row.update({
192             "prior_mec_radius_m": radius,
193             "positive_bearing_radius_upper_m": bound,
194             "movement_m": movement,
195             "no_signal_risk_index": risk,
196             "radius_term": bound / radius,
197             "movement_term": movement_weight * movement / 1000.0,
198             "loss_term": loss_weight * risk,
199             "worst_including_uninformative_no_signal_m": radius,
200             "nonempty_bearing_bins": bins_used,
201             "bearing_bins": bearing_bins,
202             "error_deg": error_deg,
203             "risk_samples": len(samples),
204             "score": (
205                 bound / radius
206                 + movement_weight * movement / 1000.0
207                 + loss_weight * risk
208             ),
209         })
210         table.append(row)
211     return table
212
213
214 def choose_probe(
215     polygon, readings, current_position, problem, **options
216 ) -> Point | None:
217     """Choose minimum score, then minimum travel, then generation-order tie.
218
219     The caller MUST debit its existing finite probe budget, apply only actual
220     positive bearings via Planner.measure, and retain the original fallback.
221     Use score_probes with identical arguments to obtain the full score table.
222     """
223     table = score_probes(polygon, readings, current_position, problem, **options)
224     eligible = [row for row in table if row["eligible"]]
225     if not eligible:
226         return None
227     best = min(eligible, key=lambda row: (row["score"], row["movement_m"]))
228     return best["point"]

```

Listing 18: 骨架路由候选

```

1  """Deterministic routing and task-priority candidates.
2
3  These helpers are deliberately separate from :mod:`planner`: they can improve
4  travel order without changing the certified waypoint set or its discovery
5  obligations. No simulator/source-truth state is read here.
6  """
7
8  from __future__ import annotations
9
10 import math
11 from collections.abc import Sequence
12 from numbers import Real
13
14
15 def _point(value: object) -> tuple[float, float] | None:
16     if isinstance(value, (str, bytes)) or not isinstance(value, Sequence) or len(value) != 2:
17         return None

```

```

18     x, y = value
19     if not isinstance(x, Real) or not isinstance(y, Real):
20         return None
21     x, y = float(x), float(y)
22     return (x, y) if math.isfinite(x) and math.isfinite(y) else None
23
24
25 def optimize_waypoints(points: Sequence[object], start: tuple[float, float] = (0.0, 0.0)) -> list[int]:
26     """Return a deterministic nearest-neighbour + 2-opt index order.
27
28     Every valid input point occurs exactly once. Invalid point data, or an
29     invalid start, uses the exact identity order as a safe, obligation-
30     preserving fallback; it never silently removes or fabricates a waypoint.
31     The 2-opt pass only reverses contiguous order segments, so it changes
32     travel length but cannot change the waypoint certificate.
33     """
34     try:
35         n = len(points)
36     except (TypeError, AttributeError):
37         return []
38     fallback = list(range(n))
39     origin = _point(start)
40     if origin is None:
41         return fallback
42     coords = [_point(p) for p in points]
43     if any(p is None for p in coords):
44         return fallback
45     xy = [p for p in coords if p is not None]
46     remaining = set(range(n))
47     route: list[int] = []
48     current = origin
49     while remaining:
50         index = min(remaining, key=lambda i: ((xy[i][0] - current[0]) ** 2 + (xy[i][1] - current[1]) ** 2, i))
51         route.append(index)
52         remaining.remove(index)
53         current = xy[index]
54
55     def edge(a: tuple[float, float], b: tuple[float, float]) -> float:
56         return math.hypot(a[0] - b[0], a[1] - b[1])
57
58     # First-improvement, lexicographically scanned, strict 2-opt is stable.
59     # Repeat until no improving exchange remains.
60     changed = True
61     while changed:
62         changed = False
63         for i in range(-1, n - 2): # edge from start (i=-1) or route[i]
64             a = origin if i == -1 else xy[route[i]]
65             ai = i + 1
66             for j in range(ai + 1, n):
67                 b = xy[route[ai]]
68                 c = xy[route[j]]
69                 d = xy[route[j + 1]] if j + 1 < n else None
70                 old = edge(a, b) + (edge(c, d) if d is not None else 0.0)
71                 new = edge(a, c) + (edge(b, d) if d is not None else 0.0)
72                 if new + 1e-12 < old:
73                     route[ai : j + 1] = reversed(route[ai : j + 1])
74                     changed = True
75                     break
76             if changed:
77                 break
78     return route

```

Listing 19: 路由候选对照实验

```

1 """Paired route-order candidate comparison on declared synthetic worlds."""
2 from __future__ import annotations
3

```

```

4 import json
5 import statistics
6 from pathlib import Path
7
8 from environment import LocalSimulator
9 from planner import Planner
10
11
12 def run(seed, problem, anchor_policy):
13     env = LocalSimulator(seed, problem=problem, scenario="random", error_mode="smooth")
14     planner = Planner(env, problem=problem, strategy="integrated", anchor_policy=anchor_policy)
15     planner.run()
16     result = env.summary()
17     assert result["fraction_cleared"] == 1
18     return result
19
20
21 def main():
22     output = {}
23     records = []
24     for problem in (3, 4):
25         pairs = [(run(950000 + k, problem, "nearest"), run(950000 + k, problem, "two_opt")) for k in range(200)]
26         records.extend({"problem": problem, "seed": 950000+k,
27                        "nearest": a, "two_opt": b for k, (a, b) in enumerate(pairs)})
28     deltas = [a["average_time_s"] - b["average_time_s"] for a, b in pairs]
29     output[f"Q{problem}"] = {
30         "cases": len(pairs),
31         "all_nearest_cleared": sum(a["fraction_cleared"] == 1 for a, _ in pairs),
32         "all_two_opt_cleared": sum(b["fraction_cleared"] == 1 for _, b in pairs),
33         "nearest_mean_s_per_source": statistics.mean(a["average_time_s"] for a, _ in pairs),
34         "two_opt_mean_s_per_source": statistics.mean(b["average_time_s"] for _, b in pairs),
35         "mean_saved_s_per_source": statistics.mean(deltas),
36         "median_saved_s_per_source": statistics.median(deltas),
37         "faster_cases": sum(d > 0 for d in deltas),
38         "mean_distance_saved_m": statistics.mean(a["distance_m"] - b["distance_m"] for a, b in pairs),
39     }
40     result = {"evidence": "local_synthetic_route_candidate_not_official", "seed_range": [950000, 950199], "summary": output, "runs": records}
41     Path("results/route_candidate_comparison.json").write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n")
42     print(json.dumps(result, ensure_ascii=False, indent=2))
43
44
45 if __name__ == "__main__":
46     main()

```

Listing 20: 自适应测点对照实验

```

1 """Paired comparison for the interval-scored adaptive probe candidate."""
2 from __future__ import annotations
3
4 import json
5 import statistics
6 from pathlib import Path
7
8 from environment import LocalSimulator
9 from planner import Planner
10
11
12 def run(seed, problem, probe_policy):
13     env = LocalSimulator(seed, problem=problem, scenario="random", error_mode="smooth")
14     planner = Planner(env, problem=problem, strategy="integrated", probe_policy=probe_policy)
15     planner.run()
16     summary = env.summary()
17     assert summary["fraction_cleared"] == 1
18     return {**summary, "adaptive_probes": planner.adaptive_probes}

```

```

19
20
21 def aggregate(rows):
22     delta = [a["average_time_s"] - b["average_time_s"] for a, b in rows]
23     return {
24         "cases": len(rows),
25         "fixed_all_cleared": sum(a["fraction_cleared"] == 1 for a, _ in rows),
26         "adaptive_all_cleared": sum(b["fraction_cleared"] == 1 for _, b in rows),
27         "fixed_mean_s_per_source": statistics.mean(a["average_time_s"] for a, _ in rows),
28         "adaptive_mean_s_per_source": statistics.mean(b["average_time_s"] for _, b in rows),
29         "mean_saved_s_per_source": statistics.mean(delta),
30         "median_saved_s_per_source": statistics.median(delta),
31         "faster_cases": sum(d > 0 for d in delta),
32         "fixed_mean_failed_clears": statistics.mean(a["failed_clears"] for a, _ in rows),
33         "adaptive_mean_failed_clears": statistics.mean(b["failed_clears"] for _, b in rows),
34         "mean_adaptive_probes": statistics.mean(b["adaptive_probes"] for _, b in rows),
35     }
36
37
38 def main():
39     summary = {}
40     records = []
41     for problem in (3, 4):
42         pairs = [(run(960000 + k, problem, "fixed"), run(960000 + k, problem, "adaptive")) for k in range(200)]
43         summary[f"Q{problem}"] = aggregate(pairs)
44         records.extend({"problem": problem, "seed": 960000 + k, "fixed": fixed, "adaptive": adaptive}
45                        for k, (fixed, adaptive) in enumerate(pairs))
46     result = {"evidence": "local_synthetic_adaptive_candidate_not_official", "seed_range": [960000, 960199], "
47              summary": summary, "runs": records}
48     Path("results/adaptive_candidate_comparison.json").write_text(json.dumps(result, ensure_ascii=False, indent
49                               =2) + "\n")
50     print(json.dumps({"evidence": result["evidence"], "summary": summary}, ensure_ascii=False, indent=2))
51
52 if __name__ == "__main__":
53     main()

```

Listing 21: 超时终端状态回归

```

1 """Finite Planner terminal/deadline regression using a protocol-shaped fake adapter."""
2 from __future__ import annotations
3
4 import json
5 from pathlib import Path
6
7 from planner import Planner
8
9
10 class DeadlineAdapter:
11     def __init__(self, remaining=1200, max_virtual=10, near_channel=None):
12         self.remaining = remaining
13         self.max_virtual = max_virtual
14         self.virtual = 0.0
15         self.channel = 1
16         self.paths = []
17         self.near_channel = near_channel
18
19     def act(self, path, position=None, channel=None):
20         self.paths.append(path)
21         if path == "/enter":
22             return {"accepted": True, "virtual_time_s": 0.0,
23                    "remaining_real_duration_s": self.remaining,
24                    "max_virtual_duration_s": self.max_virtual}
25         if path == "/exit":
26             if self.virtual >= self.max_virtual:
27                 raise AssertionError("Planner sent /exit after automatic virtual close")
28             return {"accepted": True, "virtual_time_s": self.virtual,

```

```

29         "exit_reason": "user_exit"}
30     if path == "/clear":
31         self.virtual += 5
32         return {"accepted": True, "virtual_time_s": self.virtual, "clear_result": "success"}
33     switch = int(path == "/measure" and channel != self.channel)
34     self.virtual += 5 + switch
35     self.channel = channel
36     return {"accepted": True, "virtual_time_s": self.virtual,
37            "measure_result": "near" if channel == self.near_channel else "no_signal"}
38
39
40 def main():
41     automatic = DeadlineAdapter(remaining=1200, max_virtual=10)
42     result_auto = Planner(automatic, problem=3).run()
43     assert result_auto["terminal_reason"] == "virtual_timeout"
44     assert result_auto["certificate_complete"] is False
45     assert "/exit" not in automatic.paths
46
47     real = DeadlineAdapter(remaining=1, max_virtual=360000)
48     result_real = Planner(real, problem=3).run()
49     assert result_real["terminal_reason"] == "TimeoutError"
50     assert result_real["certificate_complete"] is False
51     assert "/exit" not in real.paths
52     near = DeadlineAdapter(max_virtual=119, near_channel=20)
53     policy = Planner(near, problem=3)
54     for record in policy.scan_records.values():
55         record.update(range(1,7))
56     near_result = policy.run()
57     assert near_result["channel_certificates"][20] == "detected"
58     assert not near_result["certificate_complete"] and "/clear" not in near.paths
59     assert "/exit" not in near.paths
60     completed = DeadlineAdapter(max_virtual=119)
61     policy = Planner(completed, problem=3)
62     for record in policy.scan_records.values():
63         record.update(range(1,7))
64     completed_result = policy.run()
65     assert completed_result["certificate_complete"]
66     assert completed_result["termination"] == "complete_after_virtual_timeout"
67     assert "/exit" not in completed.paths
68     final_clear = DeadlineAdapter(max_virtual=5)
69     policy = Planner(final_clear, problem=3)
70     policy.action("/enter")
71     policy.cleared.update(range(1,16))
72     policy.clear(16, (0.0, 0.0))
73     assert policy._result()["certificate_complete"] and "/exit" not in final_clear.paths
74     result = {"evidence": "local_protocol_shaped_terminal_regression_not_official",
75              "automatic_virtual_timeout": result_auto,
76              "preflight_real_deadline": result_real,
77              "never_exited_after_terminal": True,
78              "near_at_limit_not_absent": True,
79              "final_absence_certificate_at_limit": True,
80              "sixteenth_clear_at_limit_complete": True,
81              "never_claimed_certificate": True}
82     Path("results/terminal_verification.json").write_text(json.dumps(result, indent=2) + "\n")
83     print(json.dumps(result, indent=2))
84
85
86 if __name__ == "__main__":
87     main()

```

Listing 22: 环半径与滚动调度开发比较

```

1  """Development-only anchor geometry and rolling route screen, no official calls."""
2  from __future__ import annotations
3
4  import json
5  import statistics

```

```

6 from pathlib import Path
7
8 from environment import LocalSimulator
9 from planner import Planner
10
11
12 def case(seed, problem, route, ring_radius=None):
13     env = LocalSimulator(seed, problem=problem)
14     policy = Planner(env, problem=problem, strategy="integrated", anchor_policy=route,
15                     ring_radius=ring_radius)
16     result = policy.run()
17     actual = env.summary()
18     assert result['certificate_complete'] and actual['fraction_cleared'] == 1
19     return actual
20
21
22 def main():
23     results = {}
24     for problem in (3, 4):
25         configs = [('nearest', None), ('rolling', None)]
26         if problem == 3:
27             configs += [('nearest', 1200.), ('rolling', 1200.)]
28         baseline = None
29         for route, radius in configs:
30             rows = [case(1000000+k, problem, route, radius) for k in range(150)]
31             if baseline is None:
32                 baseline = rows
33             differences = [a['average_time_s']-b['average_time_s'] for a,b in zip(baseline,rows)]
34             results[f'Q{problem}_{route}_{radius}'] = {
35                 'cases':len(rows), 'mean_s_per_source':statistics.mean(r['average_time_s'] for r in rows),
36                 'mean_distance_m':statistics.mean(r['distance_m'] for r in rows),
37                 'mean_measures':statistics.mean(r['measures'] for r in rows),
38                 'mean_saved_s_per_source':statistics.mean(differences),
39                 'faster_cases':sum(d>0 for d in differences), 'all_cleared':True,
40                 'runs':rows}
41             Path('results/anchor_development.json').write_text(json.dumps(results,indent=2)+'\n')
42             print(json.dumps({k:{f:v for f,v in row.items() if f!='runs'} for k,row in results.items()},indent=2))
43
44
45 if __name__ == '__main__':
46     main()

```

Listing 23: 增强策略样本外实验

```

1 """Frozen held-out comparison of accepted geometric/rolling enhancements."""
2 from __future__ import annotations
3
4 import json
5 import math
6 import statistics
7 from pathlib import Path
8
9 from coverage import ring_cover_radius
10 from experiments import aggregate, run_case
11
12
13 def paired_summary(control, enhanced):
14     assert [r['seed'] for r in control] == [r['seed'] for r in enhanced]
15     differences = [a['average_time_s']-b['average_time_s'] for a,b in zip(control, enhanced)]
16     average = statistics.mean(differences)
17     stderr = statistics.stdev(differences)/math.sqrt(len(differences))
18     return {'cases':len(differences), 'mean_saved_s_per_source':average,
19           'mean_reduction_pct':100*average/statistics.mean(r['average_time_s'] for r in control),
20           'approx_95pct_ci_s':[average-1.96*stderr, average+1.96*stderr],
21           'faster_cases':sum(d>0 for d in differences),
22           'mean_distance_saved_m':statistics.mean(a['distance_m']-b['distance_m'] for a,b in zip(control,
23           enhanced))}

```

```

23
24
25 def main():
26     rows, summary, comparison, stress = [], {}, {}, {}
27     for problem in (3,4):
28         paired = []
29         for policy in ('integrated','enhanced'):
30             group = [run_case(1100000+k,problem,policy) for k in range(500)]
31             rows.extend(group)
32             summary[f'Q{problem}_{policy}'] = aggregate(group)
33             paired.append(group)
34         comparison[f'Q{problem}'] = paired_summary(*paired)
35         for scenario in ('minimum_radius','outward_boundary','clustered'):
36             for mode in ('smooth','extreme','iid_location'):
37                 groups = []
38                 for policy in ('integrated','enhanced'):
39                     group = [run_case(1110000+k,problem,policy,scenario,mode) for k in range(30)]
40                     rows.extend(group)
41                     groups.append(group)
42                 key = f'Q{problem}_{scenario}_{mode}'
43                 stress[key] = {'integrated':aggregate(groups[0]), 'enhanced':aggregate(groups[1]),
44                               'paired':paired_summary(*groups)}
45     result = {'evidence':'held_out_local_synthetic_not_official',
46              'selection':'Only development seeds1000000..1000149 used; final seeds excluded from choice',
47              'main_seed_range':[1100000,1100499], 'stress_seed_range':[1110000,1110029],
48              'total_runs':len(rows), 'summary':summary, 'comparison':comparison, 'stress':stress,
49              'contracted_ring':{'radius_m':1200., 'exact_cover_radius_m':ring_cover_radius(1200.),
50                                'radio_margin_m':1000-ring_cover_radius(1200.)}, 'runs':rows}
51     Path('results/enhancement_experiments.json').write_text(json.dumps(result,indent=2)+'\n')
52     print(json.dumps({k:v for k,v in result.items() if k not in ('runs','stress')},indent=2))
53
54
55 if __name__ == '__main__':
56     main()

```

Listing 24: Q3 环半径筛选与新世界确认

```

1  """Development and held-out sweep for the safe Q3 ring radius."""
2  from __future__ import annotations
3
4  import json
5  import math
6  import statistics
7  from pathlib import Path
8
9  from coverage import ring_cover_radius
10 from experiments import run_case
11
12 CANDIDATES = (1200.0, 1250.0, 1300.0, 1350.0, 1400.0,
13               1450.0, 1500.0, 1550.0)
14 DEV_SEEDS = range(1000000, 1000150)
15 HELD_OUT_SEEDS = range(2300000, 2300500)
16
17
18 def summarize(rows):
19     values = sorted(row["average_time_s"] for row in rows)
20     return {
21         "cases": len(rows),
22         "all_cleared_cases": sum(row["fraction_cleared"] == 1 for row in rows),
23         "mean_average_time_s": statistics.mean(values),
24         "median_average_time_s": statistics.median(values),
25         "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1],
26         "mean_distance_m": statistics.mean(row["distance_m"] for row in rows),
27         "mean_measures": statistics.mean(row["measures"] for row in rows),
28     }
29
30

```

```

31 def run_radius(radius, seeds):
32     cover = ring_cover_radius(radius)
33     if cover > 1000.0:
34         raise ValueError(f"unsafe candidate radius {radius}: cover={cover}")
35     rows = []
36     for seed in seeds:
37         row = run_case(seed, 3, "enhanced", ring_radius=radius,
38                        anchor_policy="nearest")
39         row.pop("real_duration_s") # Exclude wall-clock noise from geometry comparison.
40         rows.append(row)
41     return rows
42
43
44 def paired_summary(control, candidate):
45     assert [row["seed"] for row in control] == [row["seed"] for row in candidate]
46     deltas = [a["average_time_s"] - b["average_time_s"]
47               for a, b in zip(control, candidate, strict=True)]
48     mean = statistics.mean(deltas)
49     standard_error = statistics.stdev(deltas) / math.sqrt(len(deltas))
50     return {
51         "cases": len(deltas),
52         "mean_saved_s_per_source": mean,
53         "approx_95pct_ci_s": [mean - 1.96 * standard_error,
54                               mean + 1.96 * standard_error],
55         "faster_candidate_cases": sum(delta > 0 for delta in deltas),
56         "all_control_cleared_cases": sum(row["fraction_cleared"] == 1 for row in control),
57         "all_candidate_cleared_cases": sum(row["fraction_cleared"] == 1 for row in candidate),
58         "deltas_s_per_source": deltas,
59     }
60
61
62 def main():
63     development = {}
64     for radius in CANDIDATES:
65         rows = run_radius(radius, DEV_SEEDS)
66         development[str(radius)] = {
67             "ring_radius_m": radius,
68             "exact_cover_radius_m": ring_cover_radius(radius),
69             "summary": summarize(rows),
70             "runs": rows,
71         }
72     ordered = sorted(development,
73                      key=lambda key: development[key]["summary"]["mean_average_time_s"])
74     winner, runner_up = ordered[0], ordered[1]
75     held_out = {}
76     held_out_rows = {}
77     for radius in sorted({1200.0, float(winner), float(runner_up)}):
78         rows = run_radius(radius, HELD_OUT_SEEDS)
79         held_out_rows[str(radius)] = rows
80         held_out[str(radius)] = {
81             "ring_radius_m": radius,
82             "exact_cover_radius_m": ring_cover_radius(radius),
83             "summary": summarize(rows),
84             "runs": rows,
85         }
86     control = "1200.0" if winner != "1200.0" else runner_up
87     comparison = paired_summary(held_out_rows[control], held_out_rows[winner])
88     comparison.update({"control_ring_radius_m": float(control),
89                      "candidate_ring_radius_m": float(winner),
90                      "control_role": "incumbent" if control == "1200.0" else "development_runner_up"})
91     result = {
92         "evidence": "held_out_local_synthetic_not_official",
93         "selection": "Eight radii compared on existing development seeds; fresh confirmation seeds replace
94                      exploratory reuse of 1100000..1100499. No retuning after confirmation.",
95         "total_runs": len(CANDIDATES) * len(DEV_SEEDS) + len(held_out_rows) * len(HELD_OUT_SEEDS),
96         "candidate_radii_m": list(CANDIDATES),
97         "development_seed_range": [DEV_SEEDS.start, DEV_SEEDS.stop - 1],

```

```

97     "held_out_seed_range": [HELD_OUT_SEEDS.start, HELD_OUT_SEEDS.stop - 1],
98     "development": development,
99     "development_winner_m": float(winner),
100    "development_runner_up_m": float(runner_up),
101    "held_out": held_out,
102    "held_out_comparison": comparison,
103 }
104 Path("results/ring_sweep.json").write_text(
105     json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
106 )
107 print(json.dumps({
108     "total_runs": result["total_runs"],
109     "development_winner_m": float(winner),
110     "development_runner_up_m": float(runner_up),
111     "held_out_seed_range": result["held_out_seed_range"],
112     "held_out": {key: value["summary"] for key, value in held_out.items()},
113     "comparison": {key: value for key, value in comparison.items()
114                    if key != "deltas_s_per_source"},
115 }, indent=2))
116
117
118 if __name__ == "__main__":
119     main()

```

Listing 25: 增强几何及区间评分交叉验证

```

1  """Independent checks for ring contraction, 2-opt and interval probe bounds."""
2  from __future__ import annotations
3
4  import itertools
5  import json
6  import math
7  import random
8  from pathlib import Path
9
10 from active_policy_candidate import score_probes
11 from coverage import directional_waypoints, omni_waypoints, ring_cover_radius
12 from geometry import clip_bearing, enclosing_circle, initial_polygon
13 from route_policy_candidate import optimize_waypoints
14
15
16 def length(points, order, start):
17     sequence = [start] + [points[i] for i in order]
18     return sum(math.dist(a, b) for a, b in itertools.pairwise(sequence))
19
20
21 def main():
22     rng = random.Random(417072)
23     route_cases, interval_cases = 0, 0
24     for size in (0,1,2,7,31):
25         for _ in range(10):
26             points = [(rng.uniform(-2500,2500), rng.uniform(-2500,2500)) for _ in range(size)]
27             start = (rng.uniform(-1000,1000), rng.uniform(-1000,1000))
28             route = optimize_waypoints(points, start)
29             assert sorted(route) == list(range(size))
30             current, pending, greedy = start, set(range(size)), []
31             while pending:
32                 i = min(pending, key=lambda i: (math.dist(current, points[i]), i))
33                 greedy.append(i); pending.remove(i); current = points[i]
34                 assert length(points, route, start) <= length(points, greedy, start) + 1e-7
35             route_cases += 1
36     ring = omni_waypoints(1200.)
37     rho = ring_cover_radius(1200.)
38     for _ in range(2000):
39         r, phi = 1800*math.sqrt(rng.random()), rng.uniform(0, math.tau)
40         source = (r*math.cos(phi), r*math.sin(phi))
41         assert min(math.dist(source, p) for p in ring) <= rho + 1e-7

```

```

42     attained = min(math.dist((1800*math.cos(math.pi/6),1800*math.sin(math.pi/6)),p) for p in ring)
43     assert abs(attained-rho)<1e-7 and rho<1000
44     ring_tours = []
45     for d in (1200.0, 1550.0, 900.0 * math.sqrt(3.0)):
46         centers = omni_waypoints(d)[1:]
47         minimum = min(length(centers, order, (0.0, 0.0))
48                        for order in itertools.permutations(range(6)))
49         assert abs(minimum - 6 * d) < 1e-7
50         ring_tours.append({"radius_m": d, "permutations": 720,
51                           "minimum_length_m": minimum, "analytic_length_m": 6 * d})
52     vertex_cases = 0
53     for spacing in (900.0, 950.0, 990.0, 999.0):
54         vertices = directional_waypoints(spacing)
55         for source in vertices:
56             if math.hypot(*source) > 1800.0 + 1e-8:
57                 continue
58             for k in range(6):
59                 neighbor = (source[0] + spacing * math.cos(k * math.pi / 3),
60                             source[1] + spacing * math.sin(k * math.pi / 3))
61                 assert min(math.dist(neighbor, v) for v in vertices) < 1e-7
62             vertex_cases += 1
63     max_violation = -float('inf')
64     for bearing in (0., 89., 179., 359.99):
65         poly = clip_bearing(initial_polygon(),(0.,0.),bearing)
66         table = score_probes(poly,[(0.,0.),bearing]),(0.,0.),3,include_lens=False)
67         for row in table:
68             if not row['eligible']:
69                 continue
70             for angle in range(0,360,2):
71                 post = clip_bearing(poly,row['point'],angle)
72                 if not post:
73                     continue
74                 violation = enclosing_circle(post)[1] - row['positive_bearing_radius_upper_m']
75                 max_violation = max(max_violation,violation)
76                 assert violation <= 1e-6
77                 interval_cases += 1
78     result = {'evidence':'numerical_sanity_plus_analytic_geometry_not_official',
79              'route_cases':route_cases, 'ring_samples':2000,
80              'ring_radius_m':1200.,'exact_ring_cover_radius_m':rho,
81              'attained_boundary_radius_m':attained, 'radio_margin_m':1000-rho,
82              'ring_tour_exhaustive':ring_tours,
83              'source_vertex_six_neighbor_cases':vertex_cases,
84              'candidate_posterior_cases':interval_cases,'max_interval_bound_violation_m':max_violation}
85     Path('results/refinement_verification.json').write_text(json.dumps(result,indent=2)+'\n')
86     print(json.dumps(result,indent=2))
87
88
89 if __name__ == '__main__':
90     main()

```

Listing 26: 31 格点不可删反例验证

```

1  """Generate exact, auditable coverage-limit witnesses (Python 3.14).
2
3  No simulator is used: every assertion is a direct Euclidean calculation on the
4  31 points returned by ``coverage.directional_waypoints``. The JSON file is a
5  reproducible calculation artifact, not a hand-entered table.
6  """
7  from __future__ import annotations
8
9  import json
10 import math
11 import sys
12 from pathlib import Path
13
14 ROOT = Path(__file__).resolve().parent
15 sys.path.insert(0, str(ROOT))

```

```

16 import coverage
17
18 R = coverage.ARENA_RADIUS
19 Q = coverage.MIN_RADIO_RADIUS
20 S = coverage.DEFAULT_SPACING
21 EPS_CANDIDATES = (1.0,)
22
23 # Six nearest-neighbour directions in axial (i,j) coordinates.
24 DIRS = ((1, 0), (0, 1), (-1, 1), (-1, 0), (0, -1), (1, -1))
25
26 def xy(ij: tuple[int, int]) -> tuple[float, float]:
27     i, j = ij
28     return (S * (i + j / 2.0), S * math.sqrt(3.0) * j / 2.0)
29
30 def norm(p: tuple[float, float]) -> float:
31     return math.hypot(*p)
32
33 def sub(a: tuple[float, float], b: tuple[float, float]) -> tuple[float, float]:
34     return (a[0] - b[0], a[1] - b[1])
35
36 def dot(a: tuple[float, float], b: tuple[float, float]) -> float:
37     return a[0] * b[0] + a[1] * b[1]
38
39 def add(a: tuple[float, float], b: tuple[float, float]) -> tuple[float, float]:
40     return (a[0] + b[0], a[1] + b[1])
41
42 def scale(c: float, a: tuple[float, float]) -> tuple[float, float]:
43     return (c * a[0], c * a[1])
44
45
46 def actual_lattice() -> dict[tuple[int, int], tuple[float, float]]:
47     pts = coverage.directional_waypoints(S)
48     axial: dict[tuple[int, int], tuple[float, float]] = {}
49     for p in pts:
50         # Recover integer coordinates and verify the reconstruction, rather than
51         # silently substituting a separately hard-coded 31-point set.
52         j = round(p[1] / (S * math.sqrt(3.0) / 2.0))
53         i = round(p[0] / S - j / 2.0)
54         ij = (i, j)
55         assert math.dist(p, xy(ij)) < 1e-8
56         axial[ij] = p
57     assert len(axial) == 31
58     return axial
59
60 def witness_for(vij: tuple[int, int], points: dict[tuple[int, int], tuple[float, float]]) -> dict:
61     v = points[vij]
62     candidates = []
63     for k in range(6):
64         d1, d2 = DIRS[k], DIRS[(k + 1) % 6]
65         uij = (vij[0] + d1[0], vij[1] + d1[1])
66         wij = (vij[0] + d2[0], vij[1] + d2[1])
67         if uij not in points or wij not in points:
68             continue
69         m = scale(0.5, add(points[uij], points[wij]))
70         nvec = sub(v, m)
71         h = norm(nvec)
72         n = scale(1.0 / h, nvec)
73         for eps in EPS_CANDIDATES:
74             g = add(m, scale(eps, n))
75             if norm(g) >= R:
76                 continue
77             rows = []
78             for ij, p in points.items():
79                 distance = norm(sub(p, g))
80                 forward = dot(sub(p, g), n)
81                 rows.append((ij, distance, forward))
82             reachable = [row for row in rows if row[1] <= Q + 1e-8 and row[2] >= -1e-8]

```

```

83         if len(reachable) == 1 and reachable[0][0] == vij:
84             candidates.append((eps, m, n, g, rows, uij, wij))
85             break
86     assert candidates, f"no elementary witness for {vij}"
87     eps, m, n, g, rows, uij, wij = min(candidates, key=lambda z: (z[0], z[1]))
88     target_distance = norm(sub(v, g))
89     second = min(norm(sub(p, g)) for ij, p in points.items() if ij != vij)
90     reachable = sorted((dist, ij) for ij, dist, fw in rows if dist <= Q + 1e-8 and fw >= -1e-8)
91     assert reachable[0][1] == vij and len(reachable) == 1
92     return {"solevertex": list(vij), "target": list(vij), "edge_neighbors": [list(uij), list(wij)],
93           "midpoint": list(m), "epsilon": eps, "source": list(g), "n": list(n),
94           "normal": list(n), "source_radius": norm(g),
95           "target_distance": target_distance, "second_detector_distance": second,
96           "reachable": [[list(ij), dist] for dist, ij in reachable],
97           "min_other_forward": min(fw for ij, dist, fw in rows if ij != vij),
98           "midpoint_to_nearest_other_positive": min(norm(sub(p, m)) for ij, p in points.items()
99               if ij not in (uij, wij, vij) and dot(sub(p, m), n) > 1e
100               -8)}
101
102 def main() -> None:
103     points = actual_lattice()
104     witnesses = [witness_for(ij, points) for ij in sorted(points)]
105     for w in witnesses:
106         assert w["source_radius"] < R and w["target_distance"] <= Q
107         assert len(w["reachable"]) == 1
108         assert w["second_detector_distance"] > 5.0
109     optimal_d = math.hypot(R, Q)
110     half_angle = math.atan(Q / R)
111     attained_angle = math.acos(R / optimal_d)
112     assert abs(attained_angle - half_angle) < 1e-12
113     for i in range(1001):
114         d = R + i * Q / 1000
115         cos_limit = max(R / d, (d*d + R*R - Q*Q) / (2*d*R))
116         assert math.acos(min(1.0, cos_limit)) <= half_angle + 1e-12
117     result = {"parameters": {"arena_radius": R, "receive_radius": Q,
118                             "spacing": S, "point_count": len(points)},
119             "q3_boundary_angle_lower_bound": {"q": 900.0, "minimum_points": 7,
120             "half_angle_radians": math.asin(900.0 / R),
121             "reason": "six arcs have total angular capacity 2*pi only at equality; equality ring misses
122             origin"},
123             "q1000_static_seven_irreducible": {"minimum_fixed_subset": 7,
124             "boundary_arc_capacity_of_five": 10.0 * math.asin(Q / R),
125             "origin_to_ring": 900.0 * math.sqrt(3.0)},
126             "q4_global_outer_boundary_lower_bound": {"minimum_outer_detectors": 7,
127             "half_angle_radians": math.atan(Q / R),
128             "optimal_detector_radius": optimal_d,
129             "attained_half_angle_radians": attained_angle,
130             "checked_detector_radii": 1001,
131             "statement": "lower bound for arbitrary placements at an outward boundary source; not a 31-
132             point global optimum claim"},
133             "witnesses": witnesses}
134     out = ROOT / "results" / "coverage_limits.json"
135     out.parent.mkdir(exist_ok=True)
136     out.write_text(json.dumps(result, indent=2, sort_keys=True) + "\n", encoding="utf-8")
137
138 if __name__ == "__main__":
139     main()

```

Listing 27: 增强策略图表生成

```

1  """Generate enhancement-only paper assets from saved local experiment evidence.
2
3  Run from any directory with conda run -n py314 python <path>/make_enhancement_assets.py.
4  No planner, simulator, hidden world, or source file is loaded or executed.
5  """
6  from __future__ import annotations
7

```

```

8 import hashlib
9 import json
10 import math
11 import os
12 import statistics
13 import tempfile
14 from collections import defaultdict
15 from io import BytesIO
16 from pathlib import Path
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 from matplotlib.lines import Line2D
23 from matplotlib.patches import Circle
24
25 ROOT = Path(__file__).resolve().parent
26 GENERATED = ROOT / "report" / "generated"
27 FIGURES = ROOT / "report" / "figures"
28 WIDTH = 16 / 2.54
29 POLICIES = ("integrated", "enhanced")
30 SCENARIOS = ("minimum_radius", "outward_boundary", "clustered")
31 MODES = ("smooth", "extreme", "iid_location")
32 COLORS = ("0072B2", "D55E00")
33
34
35 def require(condition, message):
36     if not condition:
37         raise ValueError(message)
38
39
40 def close(actual, expected, name, tolerance=1e-8):
41     require(math.isfinite(actual) and math.isfinite(expected), f"{name}: nonfinite value")
42     require(math.isclose(actual, expected, rel_tol=1e-10, abs_tol=tolerance),
43             f"{name}: {actual!r} != {expected!r}")
44
45
46 def number(value):
47     require(math.isfinite(value), "Nonfinite table value")
48     return f"{value:.2f}"
49
50
51 def load(name):
52     return json.loads((ROOT / "results" / name).read_text(encoding="utf-8"))
53
54
55 def atomic_bytes(path, content):
56     require(path.parent in (GENERATED, FIGURES), "Output outside owned directories")
57     path.parent.mkdir(parents=True, exist_ok=True)
58     fd, temporary = tempfile.mkstemp(prefix="." + path.name + ".", dir=path.parent)
59     try:
60         with os.fdopen(fd, "wb") as stream:
61             stream.write(content)
62             os.replace(temporary, path)
63     finally:
64         Path(temporary).unlink(missing_ok=True)
65
66
67 def table(name, headers, rows, widths, left_columns=(0, 1)):
68     """Fixed-width booktabs tabular only; the parent paper owns all captions."""
69     require(len(headers) == len(widths), "Invalid table widths")
70     require(sum(widths) + 0.14 * (len(widths) - 1) <= 15.9, "Table exceeds 16 cm")
71     columns = []
72     for index, width in enumerate(widths):
73         align = r"\raggedright" if index in left_columns else r"\raggedleft"
74         columns.append(">{" + align + r"\arraybackslash}p{" + str(width) + "cm}")

```

```

75 specification = "@{" + r"@{\hspace{0.14cm}}".join(columns) + "@{"
76 lines = [r"\begin{tabular}" + specification + "}", r"\toprule",
77          " & ".join(headers) + r" \\", r"\midrule"]
78 for row in rows:
79     require(len(row) == len(headers), "Ragged table")
80     lines.append(" & ".join(str(item) for item in row) + r" \\")
81 lines.extend((r"\bottomrule", r"\end{tabular}"))
82 atomic_bytes(GENERATED / name, ("\n".join(lines) + "\n").encode("utf-8"))
83
84
85 def save_figure(fig, name):
86     buffer = BytesIO()
87     fig.savefig(buffer, format="pdf", metadata={"CreationDate": None, "ModDate": None,
88                                               "Creator": "make_enhancement_assets.py"})
89     atomic_bytes(FIGURES / name, buffer.getvalue())
90     plt.close(fig)
91
92
93 def check_run(row):
94     for key in ("n_sources", "n_cleared", "measures", "switches", "clear_attempts", "failed_clears"):
95         require(type(row[key]) is int and row[key] >= 0, f"Invalid {key}")
96     n = row["n_sources"]
97     require(n > 0 and row["n_cleared"] <= n, "Invalid per-world denominator")
98     require(row["clear_attempts"] == row["n_cleared"] + row["failed_clears"],
99            "Clear-attempt decomposition mismatch")
100    require(row["distance_m"] >= 0, "Negative distance")
101    close(row["fraction_cleared"], row["n_cleared"] / n, "Clear fraction")
102    close(row["average_time_s"], row["virtual_time_s"] / n, "Per-world average")
103    total = (row["distance_m"] / 5 + 5 * row["measures"] + row["switches"]
104            + 3 * row["failed_clears"] + 5 * row["n_cleared"])
105    # Each accepted measurement/clear rounds its movement charge to microseconds.
106    rounding_bound = 0.5e-6 * (row["measures"] + row["clear_attempts"]) + 1e-8
107    close(total, row["virtual_time_s"], "Independent cost identity", tolerance=rounding_bound)
108
109
110 def metrics(rows):
111     require(bool(rows), "Empty group")
112     values = sorted(row["average_time_s"] for row in rows)
113     return {"cases": len(rows),
114            "all_cleared_cases": sum(r["n_cleared"] == r["n_sources"] for r in rows),
115            "mean_average_time_s": statistics.mean(values),
116            "median_average_time_s": statistics.median(values),
117            "p90_average_time_s": values[math.ceil(0.9 * len(values)) - 1]}
118
119
120 def check_summary(actual, summary, name):
121     for key, value in actual.items():
122         close(value, summary[key], f"{name}/{key}")
123
124
125 def paired(control, enhanced):
126     require([r["seed"] for r in control] == [r["seed"] for r in enhanced], "Unpaired seeds")
127     for a, b in zip(control, enhanced):
128         require(all(a[key] == b[key] for key in
129                    ("problem", "scenario", "error_mode", "n_sources")), "Unpaired world metadata")
130     delta = [a["average_time_s"] - b["average_time_s"] for a, b in zip(control, enhanced)]
131     mean = statistics.mean(delta)
132     return delta, {"cases": len(delta), "mean_saved_s_per_source": mean,
133                  "mean_reduction_pct": 100 * mean / metrics(control)["mean_average_time_s"],
134                  "faster_cases": sum(value > 0 for value in delta)}
135
136
137 def load_enhancement():
138     data = load("enhancement_experiments.json")
139     require(data["main_seed_range"] == [1100000, 1100499], "Unexpected held-out seeds")
140     require(data["stress_seed_range"] == [1110000, 1110029], "Unexpected pressure seeds")
141     groups = defaultdict(list)

```

```

142     for row in data["runs"]:
143         check_run(row)
144         require(row["strategy"] in POLICIES, "Unexpected enhancement policy")
145         groups[(row["problem"], row["scenario"], row["error_mode"], row["strategy"])] += 1
146     expected = {(q, scenario, mode, policy) for q in (3, 4) for policy in POLICIES
147                 for scenario, mode in [("random", "smooth")] +
148                 [(s, m) for s in SCENARIOS for m in MODES]}
149     require(set(groups) == expected, "Missing or extra enhancement groups")
150     for key, rows in groups.items():
151         rows.sort(key=lambda r: r["seed"])
152         start, end = data["main_seed_range"] if key[1] == "random" else data["stress_seed_range"]
153         require([r["seed"] for r in rows] == list(range(start, end + 1)),
154                 f"{key}: incomplete or duplicate seeds")
155     require(len(data["runs"]) == data["total_runs"] == 3080, "Run total mismatch")
156     require(set(data["summary"]) == {f"Q{q}_{p}" for q in (3, 4) for p in POLICIES},
157             "Unexpected main summary groups")
158     require(set(data["comparison"]) == {"Q3", "Q4"}, "Unexpected main comparisons")
159     require(set(data["stress"]) == {f"Q{q}_{s}_{m}" for q in (3, 4)
160                                     for s in SCENARIOS for m in MODES}, "Expected 18 pressure groups")
161     for q in (3, 4):
162         controls = groups[q, "random", "smooth", "integrated"]
163         enhancements = groups[q, "random", "smooth", "enhanced"]
164         for policy in POLICIES:
165             check_summary(metrics(groups[q, "random", "smooth", policy]),
166                           data["summary"][f"Q{q}_{policy}"], f"Q{q}/{policy}")
167             check_summary(paired(controls, enhancements)[1], data["comparison"][f"Q{q}"], f"Q{q}/paired")
168         for scenario in SCENARIOS:
169             for mode in MODES:
170                 key = f"Q{q}_{scenario}_{mode}"
171                 for policy in POLICIES:
172                     check_summary(metrics(groups[q, scenario, mode, policy]),
173                                   data["stress"][key][policy], key + "/" + policy)
174                     check_summary(paired(*(groups[q, scenario, mode, p] for p in POLICIES))[1],
175                                   data["stress"][key]["paired"], key + "/paired")
176     return data, groups
177
178
179 def enhancement_tables(data, groups):
180     performance_rows, cost_rows, macros = [], [], []
181     for q, word in ((3, "Three"), (4, "Four")):
182         control, enhanced = (groups[q, "random", "smooth", p] for p in POLICIES)
183         for policy, rows in zip(POLICIES, (control, enhanced)):
184             stats = metrics(rows)
185             label = "原集成" if policy == "integrated" else "增强"
186             performance_rows.append((f"Q{q}", label,
187                                     number(stats["mean_average_time_s"]),
188                                     number(stats["median_average_time_s"]),
189                                     number(stats["p90_average_time_s"]),
190                                     f'{stats["all_cleared_cases"]}/{stats["cases"]}')
191                                     # Average each world's cost/N; never divide pooled costs by pooled N.
192                                     components = [statistics.mean(value(r) / r["n_sources"] for r in rows)
193                                                   for value in (lambda r: r["distance_m"] / 5,
194                                                             lambda r: 5 * r["measures"], lambda r: r["switches"],
195                                                             lambda r: 3 * r["failed_clears"], lambda r: 5 * r["n_cleared"])]
196                                     rounding_bound = statistics.mean(
197                                         (0.5e-6 * (r["measures"] + r["clear_attempts"]) + 1e-8) / r["n_sources"]
198                                         for r in rows)
199                                     close(sum(components), stats["mean_average_time_s"], "Mean cost identity", tolerance=rounding_bound)
200             )
201             cost_rows.append((f"Q{q}", label, *(number(v) for v in components),
202                               number(stats["mean_average_time_s"])))
203             comparison = paired(control, enhanced)[1]
204             macros.extend(((f"EnhancedQ{word}Time", number(metrics(enhanced)["mean_average_time_s"]),
205                             (f"EnhancedQ{word}Gain", number(comparison["mean_reduction_pct"]))))
206             macros.extend(((f"EnhancementRuns", str(len(data["runs"])),
207                             (f"ContractedRadius", number(data["contracted_ring"]["radius_m"]))))
208     atomic_bytes(GENERATED / "enhancement_numbers.tex",

```

```

208         ("% Gains are percentage reductions of paired-group mean T/N.\n" +
209          "\n".join("\newcommand{\key}{" + key + "}" + value + "}" for key, value in macros)
210          + "\n").encode("utf-8"))
211 table("enhancement_table.tex", ("问题", "策略", "均值/s", "中位数/s", "$P_{90}$/s", "全清/世界"),
212      performance_rows, (0.9, 1.7, 2.6, 2.6, 2.6, 2.6))
213 table("enhancement_costs.tex", ("问题", "策略", "$L/(5N)$", "$5M/N$", "$S/N$", "$3F/N$", "$5K/N$", "合计/s"),
214      ,
215      cost_rows, (0.9, 1.5, 2.1, 1.8, 1.7, 1.7, 1.7, 2.1))
216 scenario_labels = {"minimum_radius": "最小半径", "outward_boundary": "边界朝外", "clustered": "聚集"}
217 mode_labels = {"smooth": "平滑", "extreme": "极端", "iid_location": "位置独立"}
218 pressure_rows = []
219 for q in (3, 4):
220     for scenario in SCENARIOS:
221         for mode in MODES:
222             control, enhanced = (groups[q, scenario, mode, p] for p in POLICIES)
223             comparison = paired(control, enhanced)[1]
224             both_clear = sum(a["n_cleared"] == a["n_sources"] and b["n_cleared"] == b["n_sources"]
225                             for a, b in zip(control, enhanced))
226             pressure_rows.append((f"Q{q}", scenario_labels[scenario], mode_labels[mode],
227                                  number(metrics(control)["mean_average_time_s"]),
228                                  number(metrics(enhanced)["mean_average_time_s"]),
229                                  number(comparison["mean_saved_s_per_source"]),
230                                  f'{comparison["faster_cases"]}/{len(control)}',
231                                  f'{both_clear}/{len(control)}'))
232 table("enhancement_stress.tex", ("问题", "场景", "误差", "原集成/s", "增强/s", "节省/s", "更快/对", "双全清/对"),
233      pressure_rows, (0.8, 1.7, 1.7, 2.1, 2.1, 2.1, 1.6, 2.0), left_columns=(0, 1, 2))
234
235 def candidate_table():
236     static = load("route_candidate_comparison.json")
237     adaptive = load("adaptive_candidate_comparison.json")
238     require(set(static["summary"]) == set(adaptive["summary"]) == {"Q3", "Q4"},
239            "Unexpected candidate problems")
240     static_start, static_end = static["seed_range"]
241     adaptive_start, adaptive_end = adaptive["seed_range"]
242     require(type(static_start) is int and type(static_end) is int and static_end >= static_start,
243            "Invalid static seed range")
244     require(type(adaptive_start) is int and type(adaptive_end) is int and adaptive_end >= adaptive_start,
245            "Invalid adaptive seed range")
246     static_count, adaptive_count = static_end - static_start + 1, adaptive_end - adaptive_start + 1
247     static_groups = defaultdict(list)
248     for pair in static.get("runs", []):
249         require(pair["problem"] in (3, 4), "Unexpected static problem")
250         nearest, two_opt = pair["nearest"], pair["two_opt"]
251         check_run(nearest)
252         check_run(two_opt)
253         require(nearest["seed"] == two_opt["seed"] == pair["seed"]
254                and nearest["problem"] == two_opt["problem"] == pair["problem"],
255                "Static pair metadata mismatch")
256         static_groups[pair["problem"]].append(pair)
257     adaptive_groups = defaultdict(list)
258     for pair in adaptive.get("runs", []):
259         require(pair["problem"] in (3, 4), "Unexpected adaptive problem")
260         for policy in ("fixed", "adaptive"):
261             row = pair[policy]
262             check_run(row)
263             require(row["seed"] == pair["seed"] and row["problem"] == pair["problem"],
264                    "Adaptive pair metadata mismatch")
265             adaptive_groups[pair["problem"]].append(pair)
266     rows = []
267     for q in (3, 4):
268         pairs = sorted(static_groups[q], key=lambda p: p["seed"])
269         require([p["seed"] for p in pairs] == list(range(static_start, static_end + 1)),
270                "Static seed mismatch")
271         nearest = [p["nearest"] for p in pairs]
272         two_opt = [p["two_opt"] for p in pairs]

```

```

273     deltas = [a["average_time_s"] - b["average_time_s"]
274               for a, b in zip(nearest, two_opt)]
275     s = {"cases": static_count,
276         "all_nearest_cleared": sum(a["fraction_cleared"] == 1 for a in nearest),
277         "all_two_opt_cleared": sum(b["fraction_cleared"] == 1 for b in two_opt),
278         "nearest_mean_s_per_source": statistics.mean(a["average_time_s"] for a in nearest),
279         "two_opt_mean_s_per_source": statistics.mean(b["average_time_s"] for b in two_opt),
280         "mean_saved_s_per_source": statistics.mean(deltas),
281         "faster_cases": sum(d > 0 for d in deltas)}
282     for key, value in s.items():
283         if key in static["summary"][f"Q{q}"]:
284             close(value, static["summary"][f"Q{q}"][key], f"Static Q{q}/{key}")
285     close(s["nearest_mean_s_per_source"] - s["two_opt_mean_s_per_source"],
286          s["mean_saved_s_per_source"], "Static mean difference")
287     rows.append((f"Q{q}", "静态2-opt", number(s["nearest_mean_s_per_source"]),
288                number(s["two_opt_mean_s_per_source"]), number(s["mean_saved_s_per_source"]),
289                f'{s["faster_cases"]}/{static_count}',
290                f'{s["all_nearest_cleared"]}/{s["all_two_opt_cleared"]}'))
291     pairs = sorted(adaptive_groups[q], key=lambda p: p["seed"])
292     require([p["seed"] for p in pairs] == list(range(adaptive_start, adaptive_end + 1)),
293            "Adaptive seed mismatch")
294     fixed, adapted = [p["fixed"] for p in pairs], [p["adaptive"] for p in pairs]
295     fixed_metrics, adapted_metrics = metrics(fixed), metrics(adapted)
296     delta, comparison = paired(fixed, adapted)
297     actual = {"cases": len(pairs), "fixed_all_cleared": fixed_metrics["all_cleared_cases"],
298             "adaptive_all_cleared": adapted_metrics["all_cleared_cases"],
299             "fixed_mean_s_per_source": fixed_metrics["mean_average_time_s"],
300             "adaptive_mean_s_per_source": adapted_metrics["mean_average_time_s"],
301             "mean_saved_s_per_source": comparison["mean_saved_s_per_source"],
302             "median_saved_s_per_source": statistics.median(delta),
303             "faster_cases": comparison["faster_cases"]}
304     check_summary(actual, adaptive["summary"][f"Q{q}"], f"Adaptive Q{q}")
305     rows.append((f"Q{q}", "自适应探测", number(actual["fixed_mean_s_per_source"]),
306                number(actual["adaptive_mean_s_per_source"]), number(actual["mean_saved_s_per_source"]),
307                f'{actual["faster_cases"]}/{adaptive_count}',
308                f'{actual["fixed_all_cleared"]}/{actual["adaptive_all_cleared"]}'))
309     table("candidate_table.tex", ("问题", "候选", "对照/s", "候选/s", "节省/s", "更快/对", "全清数\\newline 对照/
    候选"),
310          rows, (0.8, 2.4, 2.1, 2.1, 2.1, 1.9, 2.6))
311
312
313 def coverage_limits_table():
314     certificate = load("coverage_limits.json")
315     witnesses = certificate["witnesses"]
316     require(len(witnesses) == certificate["parameters"]["point_count"] == 31,
317            "Witness count mismatch")
318     target_distances = [row["target_distance"] for row in witnesses]
319     other_distances = [row["second_detector_distance"] for row in witnesses]
320     rows = [
321         ("Q3 q=900 lower bound", "7", "six boundary arcs at equality; origin uncovered"),
322         ("Q3 fixed seven-set at q=1000", "7", "five ring arcs leave a boundary gap"),
323         ("Q4 outer-boundary lower bound", "7", "arbitrary placements; outer detectors only"),
324         ("Q4 current lattice witnesses", str(len(witnesses)),
325          f"target d={min(target_distances):.3f}--{max(target_distances):.3f} m"),
326         ("Q4 nearest other lattice point", f"{min(other_distances):.3f} m",
327          "every witness remains outside near threshold"),
328     ]
329     table("coverage_limits_table.tex", ("Statement", "Value", "Interpretation"), rows,
330          (4.1, 2.0, 8.7), left_columns=(0, 2))
331
332
333 def ring_sweep_table():
334     data = load("ring_sweep.json")
335     require(data["development_seed_range"] == [1000000, 1000149], "Ring development seeds")
336     require(data["held_out_seed_range"] == [2300000, 2300499], "Ring fresh confirmation seeds")
337     candidates = data["candidate_radii_m"]
338     require(candidates == [1200., 1250., 1300., 1350., 1400., 1450., 1500., 1550.],

```

```

339         "Unexpected frozen ring candidates")
340 count = 0
341 for phase, seed_key in (("development", "development_seed_range"),
342                         ("held_out", "held_out_seed_range")):
343     start, end = data[seed_key]
344     for key, group in data[phase].items():
345         runs = group["runs"]
346         require([r["seed"] for r in runs] == list(range(start, end + 1)), "Ring paired seeds")
347         count += len(runs)
348         radius = float(key)
349         envelope = max(radius / math.sqrt(3),
350                        math.sqrt(1800**2 + radius**2 - math.sqrt(3) * 1800 * radius))
351         close(envelope, group["exact_cover_radius_m"], "Ring envelope")
352         require(envelope < 1000, "Unsafe ring candidate")
353         for row in runs:
354             check_run(row)
355             require(row["problem"] == 3 and row["strategy"] == "enhanced"
356                    and row["scenario"] == "random" and row["error_mode"] == "smooth",
357                    "Ring world contract")
358             close(row["ring_radius_m"], radius, "Ring run geometry")
359             require(row["certificate_complete"] and row["timing_independently_verified"]
360                    and row["n_cleared"] == row["n_sources"], "Incomplete ring run")
361             check_summary(metrics(runs), group["summary"], "Ring summary")
362         require(count == data["total_runs"] == 2200, "Ring run denominator")
363         ordered = sorted(data["development"], key=lambda k:
364                          data["development"][k]["summary"]["mean_average_time_s"])
365         winner, runner_up = ordered[:2]
366         close(float(winner), data["development_winner_m"], "Ring development winner")
367         close(float(runner_up), data["development_runner_up_m"], "Ring runner-up")
368         control = "1200.0" if winner != "1200.0" else runner_up
369         comparison = data["held_out_comparison"]
370         close(comparison["candidate_ring_radius_m"], float(winner), "Ring candidate")
371         close(comparison["control_ring_radius_m"], float(control), "Ring control")
372         deltas, calculated = paired(data["held_out"][control]["runs"], data["held_out"][winner]["runs"])
373         require(deltas == comparison["deltas_s_per_source"], "Ring raw paired deltas")
374         require(comparison["faster_candidate_cases"] == calculated["faster_cases"], "Ring win count")
375         close(calculated["mean_saved_s_per_source"], comparison["mean_saved_s_per_source"], "Ring mean")
376         se = statistics.stdev(deltas) / math.sqrt(len(deltas))
377         for observed, expected in zip(comparison["approx_95pct_ci_s"],
378                                     (statistics.mean(deltas) - 1.96 * se,
379                                      statistics.mean(deltas) + 1.96 * se), strict=True):
380             close(observed, expected, "Ring paired CI")
381 rows = []
382 for radius in candidates:
383     key = str(float(radius))
384     group = data["development"][key]
385     held = data["held_out"].get(key)
386     rows.append((f"{radius:.0f}", number(group["exact_cover_radius_m"]),
387                number(group["summary"]["mean_average_time_s"]),
388                number(held["summary"]["mean_average_time_s"]) if held else "--",
389                number(comparison["mean_saved_s_per_source"]) if key == winner else "--",
390                "保留" if key == winner else ("次优" if key == runner_up else "")))
391 table("ring_sweep_table.tex", ("环半径/m", "解析覆盖/m", "开发T/N", "新样本T/N", "相对对照节省/s", "选择"),
392      rows, (1.5, 2.2, 2.0, 2.1, 2.6, 1.0), left_columns=(5,))
393 macros = {
394     "RingSelectedTime": number(data["held_out"][winner]["summary"]["mean_average_time_s"]),
395     "RingControlTime": number(data["held_out"][control]["summary"]["mean_average_time_s"]),
396     "RingSaving": f'f{{comparison["mean_saved_s_per_source"]:.3f}}',
397     "RingLowerCI": f'f{{comparison["approx_95pct_ci_s"][0]:.3f}}',
398     "RingUpperCI": f'f{{comparison["approx_95pct_ci_s"][1]:.3f}}',
399     "RingFaster": str(comparison["faster_candidate_cases"]),
400 }
401 atomic_bytes(GENERATED / "ring_numbers.tex", "".join(
402     "\\newcommand{\\" + key + "}{\" + value + \"}\\n" for key, value in macros.items()).encode())
403
404
405 def geometry_figure(data, groups):

```

```

406 certificate = load("coverage_limits.json")
407 arena = certificate["parameters"]["arena_radius"]
408 radio = certificate["parameters"]["receive_radius"]
409 legacy = certificate["q1000_static_seven_irreducible"]["origin_to_ring"]
410 contracted = data["contracted_ring"]
411 radius = contracted["radius_m"]
412 for policy, expected in (("integrated", legacy), ("enhanced", radius)):
413     for row in groups[3, "random", "smooth", policy]:
414         close(row["ring_radius_m"], expected, "Ring geometry/run mismatch")
415 def exact_cover(r):
416     return max(r / math.sqrt(3), math.sqrt(arena**2 + r**2 - math.sqrt(3) * arena * r))
417 q = exact_cover(radius)
418 close(q, contracted["exact_cover_radius_m"], "Contracted cover radius")
419 close(radio - q, contracted["radio_margin_m"], "Contracted radial distance margin")
420 fig, axes = plt.subplots(1, 2, figsize=(WIDTH, 4.5))
421 fig.subplots_adjust(left=0.10, right=0.99, bottom=0.37, top=0.85, wspace=0.30)
422 for ax, r, color, title in zip(axes, (legacy, radius), COLORS, ("Integrated", "Contracted")):
423     centers = [(0., 0.)] + [(r * math.cos(k * math.pi / 3), r * math.sin(k * math.pi / 3))
424                        for k in range(6)]
425     for x, y in centers:
426         ax.add_patch(Circle((x / 1000, y / 1000), radio / 1000,
427                             facecolor=color, edgecolor=color, alpha=0.12, linewidth=0.8))
428     if title == "Contracted":
429         ax.add_patch(Circle((x / 1000, y / 1000), q / 1000, fill=False,
430                             edgecolor=color, linestyle="--", linewidth=0.7))
431     ax.add_patch(Circle((0, 0), arena / 1000, fill=False, color="black", linewidth=1.1))
432     ax.scatter([x / 1000 for x, _ in centers], [y / 1000 for _, y in centers],
433               color=color, s=16, zorder=4)
434     boundary = (arena * math.cos(math.pi / 6), arena * math.sin(math.pi / 6))
435     ax.plot([r / 1000, boundary[0] / 1000], [0, boundary[1] / 1000], color="black", linewidth=1.1)
436     ax.plot(boundary[0] / 1000, boundary[1] / 1000, "kx", markersize=5)
437     ax.set(xlim=(-2.7, 2.7), ylim=(-2.7, 2.7), xlabel="x (km)", ylabel="y (km)",
438           title=f"{title}\nr = {r:.2f} m")
439     ax.set_xticks((-2, 0, 2))
440     ax.set_yticks((-2, 0, 2))
441     ax.set_aspect("equal")
442     ax.grid(alpha=0.15)
443     ax.text(0.5, -0.27, f"q = {exact_cover(r):.2f} m\nmargin = {radio-exact_cover(r):.2f} m",
444            transform=ax.transAxes, ha="center", va="top", fontsize=10)
445     handles = [Line2D([], [], color="black", label="Source arena"),
446                Line2D([], [], color=COLORS[0], alpha=0.5, label=f"Radio: {radio:.0f} m"),
447                Line2D([], [], color=COLORS[1], linestyle="--", label=f"Cover: {q:.2f} m (right)")]
448     fig.legend(handles=handles, loc="lower center", bbox_to_anchor=(0.5, 0.005),
449               ncol=1, frameon=False, fontsize=10)
450     save_figure(fig, "enhancement_geometry.pdf")
451
452
453 def ecdf(ax, values, **kwargs):
454     ordered = sorted(values)
455     require(bool(ordered), "Empty ECDF")
456     ax.step([ordered[0]] + ordered, [0.] + [(i + 1) / len(ordered) for i in range(len(ordered))],
457            where="post", **kwargs)
458
459
460 def performance_figure(groups):
461     fig, axes = plt.subplots(2, 2, figsize=(WIDTH, 5.8), sharey=True)
462     fig.subplots_adjust(left=0.11, right=0.98, top=0.93, bottom=0.14, hspace=0.56, wspace=0.34)
463     for column, q in enumerate((3, 4)):
464         control, enhanced = (groups[q, "random", "smooth", p] for p in POLICIES)
465         delta, _ = paired(control, enhanced)
466         ax = axes[0, column]
467         ecdf(ax, delta, color=COLORS[column], linewidth=1.5)
468         ax.axvline(0, color="0.4", linestyle="--", linewidth=0.8)
469         ax.set(title=f"Q{q}: paired savings", xlabel="Integrated - enhanced (s/source)")
470         ax = axes[1, column]
471         for rows, color, label in zip((control, enhanced), COLORS, ("Integrated", "Enhanced")):
472             ecdf(ax, [r["average_time_s"] for r in rows], color=color, linewidth=1.5, label=label)

```

```

473     ax.set(title=f"Q{q}: time distributions", xlabel="T/N (s/source)")
474 for ax in axes.flat:
475     ax.set_ylim(0, 1.02)
476     ax.set_yticks((0, 0.5, 1))
477     ax.grid(alpha=0.18)
478     ax.tick_params(axis="x", labelrotation=0)
479 for ax in axes[:, 0]:
480     ax.set_ylabel("Empirical probability")
481 handles, labels = axes[1, 0].get_legend_handles_labels()
482 fig.legend(handles, labels, loc="lower center", ncol=2, frameon=False, bbox_to_anchor=(0.5, 0.015))
483 save_figure(fig, "enhancement_performance.pdf")
484
485
486 def main():
487     plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
488                         "axes.titlesize": 10, "axes.labelsize": 10,
489                         "xtick.labelsize": 10, "ytick.labelsize": 10,
490                         "legend.fontsize": 10, "pdf.fonttype": 42,
491                         "savefig.transparent": False, "figure.facecolor": "white"})
492 data, groups = load_enhancement()
493 enhancement_tables(data, groups)
494 candidate_table()
495 coverage_limits_table()
496 ring_sweep_table()
497 geometry_figure(data, groups)
498 performance_figure(groups)
499 print(f"Generated enhancement assets from {len(data['runs'])} saved runs; "
500       "raw candidate pairs and ring sweep validated. No simulator executed.")
501 input_names = ("enhancement_experiments.json", "route_candidate_comparison.json",
502               "adaptive_candidate_comparison.json", "coverage_limits.json",
503               "ring_sweep.json", "refinement_verification.json", "anchor_development.json")
504 output_names = ("enhancement_numbers.tex", "enhancement_table.tex", "enhancement_costs.tex",
505                "enhancement_stress.tex", "candidate_table.tex", "coverage_limits_table.tex",
506                "ring_sweep_table.tex", "ring_numbers.tex", "enhancement_geometry.pdf", "
507                enhancement_performance.pdf")
508 metadata = {
509     "evidence": "held_out_local_synthetic_not_official",
510     "policy": {"Q3": "enhanced ring radius 1200m selected by a development-only sweep, nearest anchor",
511               "Q4": "enhanced 990m triangular lattice, rolling remaining-anchor route",
512               "probe": "fixed probe and bounded recovery; adaptive candidate is negative control"},
513     "input_sha256": {"results/" + name: hashlib.sha256((ROOT / "results" / name).read_bytes()).hexdigest()
514                     for name in input_names},
515     "outputs": list(output_names),
516     "main_seed_range": data["main_seed_range"],
517     "stress_seed_range": data["stress_seed_range"],
518     "total_runs": data["total_runs"],
519     "ring_sweep_runs": 2200,
520     "ring_confirmation_seed_range": [2300000, 2300499],
521     "metric": "per-world T/N; paired savings are control minus enhanced",
522     "official_calls": 0,
523     "reproducibility": "Generated after validating every raw run, paired seed and cost decomposition",
524 }
525 atomic_bytes(GENERATED / "enhancement_metadata.json",
526             (json.dumps(metadata, ensure_ascii=False, indent=2) + "\n").encode("utf-8"))
527
528 if __name__ == "__main__":
529     main()

```

Listing 28: 覆盖上界理论片段

```

1 % Analytic coverage limits; numerical values are generated by
2 % verify_coverage_limits.py and written to results/coverage_limits.json.
3 \subsection{What is, and is not, proved by the coverage limits}
4 \label{sec:coverage-limits}
5 Let  $R=1800$  m and let  $q$  denote the receive radius. A detector at
6 radius  $d$  subtends on  $|x|=R$  an angular half-width  $\alpha$  satisfying

```

```

7 \[
8 \cos\alpha=\{R^2+d^2-q^2\over 2Rd\}.
9 \]
10 Maximising over  $d$  gives  $\alpha\leq\arcsin(q/R)$ . Consequently, at the
11 conservative  $q=900$ -m, six detectors have total boundary-arc capacity at
12 most  $2\pi$ . Equality requires every detector to attain the same extremum
13 and the six arcs to tile without overlap; the centres then lie on the ring
14  $d=900\sqrt{3}$ , whose distance from the origin exceeds  $900$ . Thus the
15 origin is uncovered in the equality case, and seven points are necessary for
16 this  $q=900$  certificate. The origin plus the six-ring construction proves
17 sufficiency.
18
19 At the minimum guaranteed receive radius  $q=1000$ -m, the same
20 \emph{fixed} seven-point set is irreducible: deleting a ring point leaves five
21 ring points, with boundary capacity at most  $10\arcsin(5/9)<2\pi$ ; deleting the
22 origin leaves it uncovered because  $900\sqrt{3}>1000$ . This is a
23 fixed-subset statement, not a claim that every arbitrary six-point placement
24 fails at  $q=1000$ .
25
26 For Q4, put  $s=990$ -m and use the actual 31 points returned by
27 \texttt{directional\_waypoints}. The script calls this function, reconstructs
28 integer axial coordinates, and enumerates every elementary triangle adjacent
29 to every one of the 31 points. For each target vertex  $v$ , it takes the
30 midpoint  $m$  of the opposite edge and tests sources
31  $g=m+\text{varepsilon}(v-m)/\text{lVert } v-m/\text{rVert}$  for a systematically enumerated
32 positive margin. It retains a witness only when  $\text{lVert } g/\text{rVert}<R$ ,
33  $\text{lVert } v-g/\text{rVert}\leq 1000$ , and all 31 points have been checked against the
34 closed forward half-plane. The resulting JSON records the source,
35 normal, source radius, target distance, and second-detector distance for all
36 31 targets. The elementary geometry explains the separation: after the
37 small inward shift, the two edge endpoints have negative projection, while
38 other points in the positive side start at the lattice distance
39  $\sqrt{7}s/2$  from  $m$  (the script verifies this from coordinates), so the
40 chosen target is the sole reachable detector. This proves indispensability
41 of this particular 31-point lattice, not global optimality among arbitrary
42 placements. The witness can be embedded in a legal 10--16-source world by
43 placing additional sources on distinct channels; no claim about a 31-point
44 global minimum is made.
45
46 Finally, an outward source on the arena boundary has angular half-width at
47 most  $\beta=\arctan(q/R)$ , hence at  $q=1000$  at least
48  $\lceil\pi/\beta\rceil=7$  outer detectors are needed. This is a global
49 outer-detector lower bound only; it does not imply that 31 total arbitrary
50 placements are optimal. Boundary points and forward tests are closed, so
51 zero projection counts as reception, while the positive source shift makes
52 the selected witness strict. The same certificate covers sources on the
53 boundary and at lattice vertices; for the latter the six nearest neighbours
54 are within  $R+s$  and one has positive projection for every direction.

```

Listing 29: 默认有限时间证明片段

```

1 \subsection{默认配置的有限动作与虚拟时间上界}
2 \label{sec:default-finite-bound}
3 本节限于本文最终增强策略：问题三用原点及半径1200的六点环、最近邻调度；问题四用990米31点骨架、滚动2-opt余点排序；均
   用固定横向候选，每任务至多6次测向。假定接口正常、误差模型成立且无现实时间中断。证明只用骨架范数界，不依赖访问顺
   序；不声称全局最优，不涵盖自适应候选或任意参数配置。距离单位均为米。
4
5 \paragraph{可行域尺寸与216单元证书。}
6 记真源为 $g$ ，最近正南向点为 $s$ ， $\alpha=1.005^\circ$ 。示向轴坐标满足 $0\leq x\leq 1500$ 、 $|y|\leq x\tan\alpha$ 。计入显
   式外扩，令 $\text{varepsilon}=10^{-7}$ ，取
7 \[
8 X=1500+2\text{varepsilon},\text{qqquad}
9 W=2\bigl((1500+\text{varepsilon})\tan\alpha+\text{varepsilon}/\cos\alpha\bigr).
10 \]
11 于是纵向长度至多 $XX$ ，横向宽度至多 $W<53$ ， $\sqrt{X^2+W^2}<1501$ 。最小包围圆圆心 $z$ 在 $P$ ，故 $\operatorname{diam}(P)$ 
    $<1501$ 且 $|z-s|<1501$ 。正测点范数至多3300、初始外切多边形范数小于1801，使相关\texttt{\_padding}为 $10^{-9}<$ 
    $\text{varepsilon}$ ；此为几何与显式外扩的解析界，不是全部浮点运算的形式化认证。

```

```

12
13 单点域直接用一圆覆盖。否则设直径端点差为$(a,b)$、$d=\sqrt{a^2+b^2}>0$。任意点差满足$|\Delta x|\leq d$、$|\Delta y|\leq W$、$|b|\leq W$，故直径法向投影差$|-b\Delta x+a\Delta y|/d\leq 2W$。计入投影外扩，直径对齐矩形仍长小于1501、宽小于106，对角线小于1505。令$h_-=\sqrt{2}(19.9-8\varepsilon)$，不大于代码单元边长上限，则
14 \begin{equation}
15 J\leq\left\lceil\frac{1501}{h_-}\right\rceil\right\rceil
16 \left\lceil\frac{106}{h_-}\right\rceil
17 =54\times 4=216
18 \end{equation}
19 个半径19.9的圆盘覆盖整个矩形。蛇形相邻中心距至多$\sqrt{2}\times 19.9$，循环移位至多增加一条矩形内跳跃，故
20 \begin{equation}
21 L_{\rm snake}\leq(216-1)\sqrt{2}\times 19.9+\sqrt{1501^2+106^2}<7570.
22 \end{equation}
23 每个覆盖中心$q_{\rm opt}$与$g$均在同一矩形中，故$|q_{\rm opt}-g|<1505$。
24
25 \paragraph{恢复点必须以最近正测点为中心估计。}
26 固定候选$q_0=z\pm\delta n$满足$n\perp(z-s)$、$\delta\leq 120$，因此$|q_0-s|\leq A:=\sqrt{1501^2+120^2}<1506$。
27 在获得新正示向前，所有恢复仍使用同一个$s$。设$R_k$为相应轴向反射，则
28 \[
29 q_{k+1}-s=\frac{1}{2}R_k(q_k-s),\quad
30 |q_{k+1}-s|=\frac{1}{2}|q_k-s|.
31 \]
32 反射只保持到$s$的距离，不能把失败点的源距与1500取平均。故连续恢复有$|q_k-s|\leq 2^{-k}A$；新正示向更新$s$后重用首点
    界。由$|s-g|\leq 1500$，全部局部测向点满足
33 \begin{equation}
34 |q-g|\leq B:=1500+A<3006,\quad
35 |q|\leq 1800+B<4806.
36 \end{equation}
37 圆心源距小于1501，覆盖中心源距小于1505，两者范数亦小于4806。测向间连线至多$2B<6020$；额外圆心清除接入至多$B+1501<4520$；光学接入至多$B+1505<4520$，若紧接圆心试清则至多$1501+1505<4520$。
38
39 \paragraph{任务进出、动作计数与总界。}
40 每任务至多6次测向，各提前返回分支均成功清除或转入有限光学覆盖；真源在覆盖域内，故每个正常完成任务必清除一新源，任务数至多16。成功终点距该源至多20、范数至多1820；共享补测及其$\texttt{near}$清除不移动。这处理了从$\texttt{near}$另一源清除点进入下一任务的情形。
41
42 骨架点范数至多$990\sqrt{7}<2620$。集成策略整次骨架扫描不插入局部移动，故至多31次骨架抵达，出发于原点、骨架或成功清除点，每段小于5240。任务首个动作由范数至多2620的任务外位置进入范数小于4806的局部点，入口小于7500。此后至多5条测向连线、一次额外圆心清除移动（保证清除或小区域试清）、一次光学接入及一条蛇形，故
43 \begin{equation}
44 L\leq 31\times 5240+
45 16\bigl(7500+5\times 6020+4520+4520+7570\bigr).
46 \label{eq:corrected-travel-bound}
47 \end{equation}
48 若首动作即圆心清除或光学首点，入口已支付该段，多余预算仅为放宽。任务至下一骨架的退出段已计入31次骨架抵达；无需返回原锚点。
49
50 骨架至多$31\times 20$次测向；共享只在任务结束后调用，至多16次、每次至多16源；局部至多$16\times 6$次。放宽共享预算得到
51 \[
52 M\leq 31\times 20+16\times 16\times 6
53 \leq 31\times 20+31\times 16\times 16\times 6=1212.
54 \]
55 每源无信号后试清至多6次、小区域试清至多一次、覆盖至多216次，另留一次给非覆盖成功清除（骨架、共享、局部$\texttt{near}$或保证清除）；成功后不再处理，故$C\leq 16(216+6+1+1)=3584$。移动速度为5米/秒，测向舍换频至多6秒，清除至多5秒，因而
56 \begin{equation}
57 \begin{split}
58 T\leq\frac{31\times 5240+
59 16(7500+5\times 6020+4520+4520+7570)}{5}\quad
60 &+1212\times 6+3584\times 5
61 <100\times 3600\ \mathrm{s}.
62 \end{split}
63 \end{equation}
64 这是虚拟时间保守界，不是预期成绩；现实20分钟限制须由客户端期限机制另行遵守。

```

Listing 30: 样本外增强分析片段

```
1 \section{有证据的搜索增强：环收缩与滚动路径}
```

2 \label{sec:enhancement}

3 \subsection{优化几何，而非减少证明义务}

4 原七点骨架的900米覆盖保证留有100米接收余量，但这一余量不必全部保留。机器狗的移动速度仅为5米/秒，约1559米的骨架环会形成较长的空频道排查路径；可在仍小于1000米的覆盖半径内，把检测环向圆心收缩。该改变没有减少七个检测位置，也没有推测某频道不存在，而是为每个新位置重新给出完整覆盖证书。

5

6 取 $R=1800$ ，原点加六个角间隔 60° 的环点，环半径为 d 。源的极径为 r ，选择角差不超过 30° 的环点，则在该方向最坏角度处，到最近检测点的距离为

7 \begin{equation}\label{eq:ring-envelope}

8 $f_d(r)=\min\left\{r,\sqrt{r^2+d^2-\sqrt{3}dr}\right\}$.

9 \end{equation}

10 两项相等时 $r=d/\sqrt{3}$ ；交点之前原点更近，之后环点更近。在 $[d/\sqrt{3},R]$ 上，根号内是凸二次函数，所以最大值在端点取得。只要 $d/\sqrt{3}\leq R$ ，整个圆域的精确覆盖半径为

11 \begin{equation}\label{eq:contracted-cover}

12 $\rho_{\text{ring}}(d)=\max\left\{\frac{d}{\sqrt{3}},\right.$

13 $\left.\sqrt{R^2+d^2-\sqrt{3}Rd}\right\}$.

14 \end{equation}

15 这不仅给出上界：相邻环点的角平分线在 $r=d/\sqrt{3}$ 及 $r=R$ 处分别达到两项，因此是精确值，而不是网格采样估计。

16

17 要求 $\rho_{\text{ring}}(d)\leq q$ ，其中 q 是选用的安全接收半径，得到

18 \begin{equation}

19 $d\leq\sqrt{3}q,\quad$

20 $\frac{\sqrt{3}R-\sqrt{4q^2-R^2}}{2}\leq d\leq$

21 $\frac{\sqrt{3}R+\sqrt{4q^2-R^2}}{2}$.

22 \end{equation}

23 当 $q=900$ 时只有原构型的 $d=900\sqrt{3}$ ；当允许 q 向物理下界1000靠近时，可选更小的 d 。本文不取恰好触及1000米的极限值，而固定 $d=1200$ 米，计算得到

24 \[

25 $\rho_{\text{ring}}(1200)=968.901572\ \mathrm{m},$

26 $\quad 1000-\rho_{\text{ring}}(1200)=31.098428\ \mathrm{m}.$

27 \]

28 该余量远大于双精度和坐标舍入误差，也允许进行明确的覆盖灵敏度分析。若检测中心位置扰动不超过 ϵ 米，三角不等式使覆盖半径至多增加 ϵ ；故在 $\epsilon<31.09$ 米时仍可保持物理距离覆盖。这不是题目要求的定位误差容许范围，而是布点构型的稳定性结论。

29

30 \begin{figure}[H]\centering

31 \includegraphics[width=\textwidth]{figures/enhancement_geometry.pdf}

32 \caption{七点覆盖的余量--行程权衡：1200米收缩环保持完整接收覆盖，不删去任何频道的检测义务。}

33 \end{figure}

34

35 \subsection{环半径的行程下界与候选筛选}

36 \label{sec:ring-sweep}

37 覆盖半径最小并不等价于任务时间最小。对原点与六个正六边形环点，若只考虑从原点出发、访问全部六个环点且不要求返回的骨架路径，则其精确最短长度为

38 \begin{equation}\label{eq:ring-tour}

39 $L_{\text{skeleton}}^{\min}(d)=6d.$

40 \end{equation}

41 证明很直接：起点到第一个环点的距离为 d ，其后五条环点间连线的长度均不小于相邻点间距 d ，故总长至少 $6d$ ；沿圆周依次访问恰好达到该下界。这个结论不含局部定位支线，也不适用于提前确认16源后省略剩余点的轨迹，但清楚解释了环收缩为何能减少 骨架层 的移动成本。

42

43 在本文支持的区间 $1200\leq d\leq 900\sqrt{3}$ 上，式 $\text{eqref{eq:contracted-cover}}$ 由边界项控制，且

44 \[

45 $\rho_{\text{ring}}'(d)=\frac{d-\sqrt{3}R/2}{\sqrt{R^2+d^2-\sqrt{3}Rd}}\leq 0,$

46 $\quad (L_{\text{skeleton}}^{\min})'(d)=6.$

47 \]

48 因此扩大检测环会改善最坏接收余量，却同时提高完整骨架行程下界；两者构成可解释的权衡，而非某一个指标越小越好。相较原半径， $d=1200$ 把纯骨架下界由 9353.07 米降到 7200 米，但完整任务仍可能因发现顺序与定位支线改变而增时，不能把这一下界差额直接当作实际节省。

49

50 为避免只比较两个半径就把1200米称为最优，本轮在既有开发种子1000000至1000149上比较8个候选半径，保持其余策略不变。表 $\text{ref{tab:ring-sweep}}$ 给出150个世界的开发均值；1200米胜出，1250米为次优。早期扩展比较曾复用1100000组数据，本表改用未参与此前筛选的2300000至2300499进行最终确认，冻结候选后不再调参。保存的2200次执行包括 8×150 次开发运行与 2×500 次新世界配对运行，不是2200个独立世界。

51 \begin{table}[H]\centering\small

52 \caption{Q3安全环半径筛选；时间单位为秒/源，破折号表示未进入样本外复核}\label{tab:ring-sweep}

53 \input{generated/ring_sweep_table.tex}

54 `\end{table}`

55 1200米与1250米在500个新检验世界均全清；前者平均为`\RingSelectedTime{}`秒/源，后者为`\RingControlTime{}`秒/源。配对节省均值为`\RingSaving{}`秒/源，其近似95%置信区间为 $[\text{\RingLowerCI}, \text{\RingUpperCI}]$ ，`\RingFaster/500`个世界更快。故保留1200米作为当前默认，不引入新策略分支。此结论仅针对声明的候选集与合成分布，不声称连续半径、所有源分布或全部在线策略中的全局最优。

56

57 `\subsection{从静态2-opt改为剩余任务的滚动重排}`

58 固定骨架路线的静态2-opt只能减少骨架间纯移动，不能预知局部定位支线结束后机器狗在哪里。因此本文改为每次准备选择骨架任务时，从`\emph{已接受动作确定的当前位置}`出发，在未完成骨架点集合上重新建立最近邻开放路径，再做2-opt交换；只执行新的第一个骨架点，下一决策时重新规划。

59

60 设当前路径包含边 (a,b) 和 (c,d) ，交换区间方向可使其变为 (a,c) 和 (b,d) ，路径变化为

61 `\begin{equation}\label{eq:two-opt-delta}`

62 `\Delta L=\lvert a-c\rvert+\lvert b-d\rvert-\lvert a-b\rvert-\lvert c-d\rvert.`

63 `\end{equation}`

64 仅当 $\Delta L < 0$ 且超过数值容差才接受。若区间到达开放路径终点，不计不存在的终端边。交换只反转索引子序列，故点集和逐频道义务不变；算法不允许因路程较远而删除圆外点。一次扫描 $O(n^2)$ 个交换，严格下降在有限排列集合内终止。本文只给出每次2-opt扫描的解析复杂度；实验结果未单独记录其CPU时间或总扫描次数。

65

66 这一滚动方案的安全性时间与时间最优性必须分开：安全性仍由完整骨架和光学后备证明，2-opt仅是当次余程的局部搜索。下一次测向发现新源后，未来真实路线可能不同；因此每次的短路不保证每个世界总时间都缩短。Q3收缩环后仅采用最近点调度，因为开发样本中叠加滚动优化的收益很小；Q4采用滚动余点排序，因为开发比较显示其能减少实际长距离骨架路程。

67 `\subsection{覆盖点数的下界与格点不可删性}`

68 `\begin{group}\sloppy`

69 Q3的900米覆盖构造和边界弧下界见式`\eqref{eq:omni}`及第`\ref{sec:cover-lower}`节；它们共同给出该保守半径下的精确七点结论，不是1000米物理接收下界下任意布局的全局最优性。

70

71 Q4的31点结论同样只针对当前\$990\$米三角格。逐点见证脚本在每个格点附近构造唯一可接收的定向源，因此删除当前集合中的任一点会破坏该证书；这不是对任意替代布局的全局点数下界。

72 再考察圆周上的径向向外发射源。探测点范数为 d 、极角差为 ϕ 时，前向条件为 $d \cos \phi \geq R$ ；接收还要求 $d^2 + R^2 - 2dR \cos \phi \leq q^2$ 。 $d < R$ 或 $d > R + q$ 均不可能接收，故单点最大覆盖角半宽为

73 `\begin{equation}\label{eq:boundary-angle-bound}`

74 `\alpha_4=\max_{R \leq d \leq R+q} \arccos\!\left\{-\left(\frac{R}{d}, \frac{d^2+R^2-q^2}{2dR}\right)\right\}=\arctan`

75 `\frac{q}{R}.`

76 `\end{equation}`

76 第一项随 d 递减，第二项在 $d \geq R$ 递增，交点 $d=\sqrt{R^2+q^2}$ 给出等号。 $q=1000, R=1800$ 时， $\alpha_4=0.50710$ 弧度，故至少需要 $\lceil \pi / \alpha_4 \rceil = 7$ 个圆外探测位置。该特殊边界下界与当前31点不可删性不同，不能推出31点全局最少。

77 `\begin{table}[H]\centering\small`

78 `\caption{覆盖下界及当前固定集合的不可删性证据}\label{tab:coverage-limits}`

79 `\input{generated/coverage_limits_table.tex}`

80 `\end{table}`

81 `\endgroup`

82

83 `\subsection{隔开发和检验的数据分工}`

84 环收缩和滚动路线的开发只使用1000000至1000149的150个种子；该组比较原环/收缩环、最近点/滚动策略，不作为最终效率宣称的检验集。配置冻结后，1100000至1100499用于每问500个随机世界的配对比较；1110000起的30个种子用于每问三种压力几何与三种误差场。所有比较中，基础集成和增强使用同一世界、同一固定地点误差及同一停止证书。主增强比较为`\EnhancementRuns{}`次运行，环半径筛选另有2200次运行；两者均单独保存在对应JSON中，不与原4430次基础对照混为同一分母。

85

86 `\begin{table}[H]\centering\small`

87 `\caption{冻结配置后的样本外比较；每问500个同种子世界，均全清}\label{tab:enhancement-main}`

88 `\input{generated/enhancement_table.tex}`

89 `\end{table}`

90 Q3增强平均为`\EnhancedQThreeTime{}`秒/源，比同世界基础集成方案降低`\EnhancedQThreeGain\%`；Q4增强平均为`\EnhancedQFourTime{}`秒/源，降低`\EnhancedQFourGain\%`。对应逐世界节省的`\textbf{配对均值近似95\%置信区间}`分别为 $[\$37.04, \$41.55]$ 秒/源和 $[\$9.69, \$18.25]$ 秒/源；这不是逐世界差值的取值区间。两者下端均大于零。Q3在478/500个世界更快，Q4在328/500个世界更快，故不宣称增强对所有实例单调占优。

91

92 `\begin{figure}[H]\centering`

93 `\includegraphics[width=\textwidth]{figures/enhancement_performance.pdf}`

94 `\caption{样本外配对差异。正节省代表增强更快，零左侧案例必须保留而不能筛掉。}`

95 `\end{figure}`

96

97 `\subsection{收益来自哪里：按每世界每源分解}`

98 表`\ref{tab:enhancement-costs}`先把每个世界的各项成本除以该世界源数，再对世界求平均。未舍入数据满足计费恒等式；本表逐项保留两位小数后，分项之和与总计可相差0.01秒/源，另有逐动作微秒舍入误差。不能改用所有世界总时间除以总源数，那会

增加大源数世界的权重。

```

99 \begin{table}[H]\centering\small
100 \caption{增强前后分项成本，单位为秒/源}\label{tab:enhancement-costs}
101 \input{generated/enhancement_costs.tex}
102 \end{table}
103 Q3收缩环主要减少纯移动，允许稍多的检测来换取更短的路程；这说明最少测向次数不是本题的正确目标。Q4滚动路线保留同样的31点
    集合，收益来自定位后实际出发位置对后续路线的反馈。只有完整计费式支持这些解释；不以失败清除次数下降单独证明总效率
    提升。
104
105 \subsection{压力分布下的效果边界}
106 \begin{table}[H]\centering\small
107 \caption{新增压力检验：每组30对世界，表中节省为基础减增强}\label{tab:enhancement-stress}
108 \input{generated/enhancement_stress.tex}
109 \end{table}
110 各组的全清首先检验覆盖和终止不变量；正负时间差则反映效率对几何、固定误差场和方向的依赖。不能把普通随机世界的平均改善直
    接推广为所有压力分布的改善，也不能因某组增时而取消必要的圆外检测。压力场景用于揭示边界，不用于反复调参直至所有行
    都变为正数。
111
112 \subsection{改进结论和实施选择}
113 本文把“能证明不漏检”与“在样本中平均更快”视为两个独立闸门。候选只有同时保留原有证书、在所有列示运行中完成证书，并在
    与对照相同的完整计费式下表现稳定，才进入默认策略；否则保留为可复核的负对照，而不是删除其结果。
114 \begin{table}[H]\centering\small
115 \caption{候选机制的准入矩阵}
116 \begin{tabular}{>\raggedright\arraybackslashp{2.7cm}>\raggedright\arraybackslashp{4.1cm}>\raggedright\arraybackslashp{4.9cm}>\raggedright\arraybackslashp{1.3cm}}\toprule
117 机制 & 不变量或安全条件 & 证据与代价 & 决策\\midrule
118 Q3收缩环 & 式\eqref{eq:contracted-cover}给出全圆域覆盖， $\rho_{\text{ring}}(1200)<1000\$$  & 150个开发世界筛选，500个样
        本外配对世界；额外保留31.10米余量 & 采用\\
119 Q4滚动路径 & 完整31点集合和逐频道检测义务不变 & 500个样本外世界中滚动重排只改变访问顺序；不减少圆外点 & 采用\\
120 静态2-opt & 点集不变，故安全但不能预见定位支线终点 & 200个同种子世界的完整 $IT/N\$$ 比较中均值增时 & 不采用\\
121 区间自适应测点 & 正示向评分有保守上界，失败时仍回到光学兜底 & Q4失败次数下降但新增测向和移动，完整时间未稳定改善 & 不
        采用\\
122 定向负观测裁剪 & 需维护 $(g,R,n)$ 联合可行集，不能只删位置圆盘 & 当前实现不做未经证明的高维判空 & 不启用\\bottomrule
123 \end{tabular}
124 \end{table}
125 该矩阵规定的是策略选择条件，而不是强制先扫完骨架再定位。在线程序仍在未完成骨架和已知源之间选择下一任务，保留所有欠缺的
    频道义务；任何局部评分都不能绕过完整覆盖或把失败清除改写成成功。“优化损失时间”与“证书失效”必须分别处理。
126
127 复现时先运行几何验证和终端回归，再运行基础实验与增强实验，随后由资产脚本从JSON重建表格、图形和轨迹，最后编译论文并检查
    正文页数。该顺序避免人工复制数字，也使每个图表都能追溯到固定代码、输入哈希和种子范围。由于官方测试尚未执行，本节
    所有“采用”只表示研究程序中的默认实现，不表示已通过正式竞赛测试。

```

A.2 匿名文件清单与结果接口

文件或目录	用途
results/final_experiments.json	4430 次最终合成运行
results/enhancement_experiments.json	3080 次样本外增强运行
results/ring_sweep.json	Q3 环半径开发筛选及样本外配对证据
results/coverage_limits.json	全部 31 点不可删构造
results/refinement_verification.json	收缩环与角区间上界验证
results/geometry_verification.json	几何交叉核验与回归
results/intersection_verification.json	原始交会区域三种状态
results/http_verification.json	幂等重试与计时验证
results/http_deadline_verification.json	慢速响应绝对期限验证
results/second_point_example.json	第二测点候选评分

<code>results/recovery_development.json</code>	恢复策略开发对照
<code>results/visual_review.json</code>	PDF 逐页视觉检查记录
<code>results/enhancement_review.json</code>	增强策略独立审阅记录
<code>results/paper_review.json</code>	非官方自动审阅记录
<code>results/final_review.json</code>	新结论与新世界证据的最终 独立审查
<code>report/generated/</code>	自动数字、表格与轨迹证据
<code>report/figures/</code>	基础及增强矢量图
<code>report/ai_usage.tex</code>	AI 使用详情源文件
<code>reproduce.txt</code> 、 <code>requirements.txt</code>	命令、环境及依赖
<code>MANIFEST.json</code>	支撑包内文件校验值
<code>results/route_candidate_comparison.json</code>	路由候选负结果
<code>results/adaptive_candidate_comparison.json</code>	自适应测点候选负结果与子 指标
<code>results/terminal_verification.json</code>	虚拟/现实期限终端回归
<code>theory_additions.txt</code> 、 <code>paper_expansion.txt</code>	独立理论草稿
<code>report/generated/coverage_limits_table.tex</code>	不可删性汇总表

图表由`make_assets.py`和`make_enhancement_assets.py`读取对应 JSON 生成, 完整代码如上。`audit_pdf.py`渲染全部页面并检查文字边界; `package.py--verify`解压支撑包, 重新运行计算和编译, 不以原目录存在文件代替解压可复现性。

含有官方 `robot_id` 的私有 HTTP 日志不进入匿名包; 如正式流程要求提交官方加密导出, 应保持官方给出的原文件名和原始字节, 不在公开包中复制或改写。官方模拟器尚未运行, 故本附录不提供官方输出。